日本語による論理プログラミングの試み

A Trial of Logic Programing in the Japanese Language

犬童 健良

KENRYO INDO

Abstract

Prolog is a logic programing language that constructs computer programs as Horn-clauses and applies a type of refutation algorithm that incorporates a backtrack mechanism. This study presents a series of frugal and practical applications of Prolog in natural language computer programing, mainly written in Japanese. Two main approaches were used. The first approach proposes Japanese natural language logic programing based on Prolog predicates. By splitting words with particles, queries in Japanese words will be unified to the head of rule that may have system or user defined predicates, including calling a shell command as a procedure attached in that body. In particular, particle *wa* is utilized for both knowledge reference and knowledge assertion. For the second approach, an automated extraction of clause programs from a well-written Japanese document titled "Calculation of inheritance tax" is demonstrated. Furthermore, the first approach is modified by incorporating negation, syllogism, and several semantic aspects. Known/unknown, old/new information, and exhaustive/contrasting are incorporated to the programs for the forms of sentences X *wa* Y and X *ga* Y. This study argued that exhaustive *ga* and contrasting *wa* can be explained formally in terms of the stable conditions for a game theoretical network model that consists of alternative relations of word pairs. Furthermore, contrasting and exhaustive cognitive functions are discussed in a similar manner in resolution of rule clauses and they relate to circumscription in non-monotonic reasoning. Additionally, as related fields, agency relationships are discussed with regard to the cognitive-social context.

和文要旨

Prologはコンピュータプログラムをホーン節として構築し，バックトラックメカニズムを備えた背理法アルゴリズムを適用する論理プログラミング言語である． この論説は，主に日本語で書かれた自然言語コンピュータープログラミングのためのPrologの質素で実用的なアプリケーションが示される． 2つの異なるアプローチが用いられた．第1のアプローチではPrologの述語をベースにした日本語論理プログラミンを提案する．日本語の単語で書かれた質問を助詞で区分することによって，システム/ユーザ定義述語やシェルコマンド呼び出しを本体に含むルールの頭部と単一化する．とくに助詞「は」は知識の参照と知識の主張に活用される．第2のアプローチでは，日本語で書かれた文書から節プログラムを直接抽出する．「相続税の計算」を説明したサンプル文書からの自動抽出がデモンストレーションされた．さらに，第1のアプローチは否定，三段論法，および意味論的概念を組み込んで修正された．既知/未知，旧い/新しい情報，総記と対比が，「XはY」や「XがY」の形の文に対するpyログラムに組み込まれた．総記の「が」と対比の「は」については，語のペアの代替的関係からなるネットワークのゲーム理論的モデルにおける安定性条件を用いて説明された．さらに，総記や対比の認知機能とルール節の導出との類似が論じられ，また非単調推論におけるサーカムスクリプションと関連付けられた．関連分野として，認知的−社会的な文脈におけるエージェンシー関係が論じられた．

1 はじめに

言葉で人とやりとりできるコンピュータを実現する[1]ことは，人工知能（AI）技術の一つの到達点と考えられる．プログラミング言語は，数式や論理式と同じく，人工言語である．JIS用語でプログラミング言語[2]とは，プログラムを表現するための人工言語である． またコンピュータプログラムまたはプログラム[3]とは，アルゴリズムの記述に適した人工言語の規則に従った構文上の単位で，ある機能または仕事の遂行または問題の解決のために必要な宣言と文や命令とから構成されるものである． 高級言語C，Java，Pythonなどは第3世代言語（3GL）に属している．人工言語としてのプログラミング言語に対して，自然言語，すなわち日常的に人間が話したり書いたりする言葉，あるいはそれに近い表現でもってプログラムを作ることができるプログラミング言語は，第4世代言語（4GL）[4]と呼ばれる．

もし人間のプログラマーがプログラミングを自然言語に近い表現によって行うことができれば，アプリケーションシステム開発をより直観的に行えるが，自然言語の曖昧性や不正確性に対処する必要が生じる．[5] 人工言語としての数式，論理式，プログラム言語などは，曖昧性や不正確性から免れて合理的な指向を表現できるメリットを追求した結果である．他方，人間が普段話したり書いたりする言葉から直接にコンピュータプログラムに翻訳することは，現代のコンピュータ技術では自然言語インタフェース（音声アシスタンスやチャットボット）として限定的に実現されている（3節で述べる方法の延長と解釈できる）．他方，現状では自然言語プログラミングは不完全なものにとどまっている．実際，日本語プログラミングと称される複数の実例が知られる（Mind，プロデル，ドリトル，言霊，ひまわり，なでしこなど）．これらは，たんに文字種の制限をなくした第3世代プログラミング言語というだけでなく，基礎的なプログラミング作業の文脈に限っては，日本語の自然な表現に近いコーディングが可能である．しかし通常の書き言葉ないし話し言葉で，自由にコンピュータに指令できるというわけではなく，第3世代プログラミング言語の約束事（構文）に沿って表現しなければならない．

日本語プログラミングと音声アスタントの関係は，より一般的な関係の一つの現れである．一般的に文表現の柔軟性と，その意味の正確さとの間にはトレードオフがある．同様に，プログラミング言語においても，プログラムとみなせる自然言語表現を限定する（第3世代言語の日本語化）か，適用領域を限定するか（DB検索など適用領域を制限する第4世代言語）かの2者択一を緩和し，両条件をバランスさせることが必要である．このトレードオフ制約をどのように克服できるかが日本語プログラミングの鍵である．

9節で詳しく論じるような意味上の問題にかかわらず，助詞に注目すると，前述の日本語プログラミングのトレードオフを緩和できることが知られる（木村・片桐，1988； 馬場・筧，2011）．木村・片桐(1988)は，MIND言語における日本語プログラミングにおいて，助詞「は」を変数宣言とし，助詞を後置した格成分をプログラムに翻訳している．馬場・筧(2011)のプロデル言語は，助詞によって仮引数と実引数を結び付けることで自然な日本語で書けるオブジェクト指向言語として提案されており，航空便座席予約システムの開発への応用例が示されている．Prolog言語でも，助詞で他の語句を結合する日本語プログラミング化が試みられてきた(高野，2014； 小谷，1993； 太細他，1984)．これらの先行研究については3.1節で言及する．この後直ぐ説明するが，Prologでは助詞や動詞をファンクターとみなすことでより簡便な日本語プログラミングが可能である(犬童，2018a)．本論文では，助詞をファンクターとする方法での日本語論理プログラミングを，「XはY」や「XがY」の形をした文の意味的処理の観点から，第4節，第5節，および第9節で，詳しく取り上げる．

素朴に考えると，自然言語の文章を論理形式に翻訳することができれば，さまざなまアプリケーションで

効率的に利用することができるはずだ．[6]しかしながら，論理プログラミングを採用した1980年代の第5世代コンピュータプロジェクトの影響が限定的であったという経験的な事実に照らすと，この予想は楽観的すぎるかもしれない．

そもそも自然言語表現を論理プログラミングへ翻訳する方法は残念ながら唯一ではない．本論文では，理論的な厳密さを追求することはせず，非専門家がすぐに実行できる方法を見出すことに専念する．まず以下の基準を満たす方法を提案する．

（基準σ）シンプルで，かつ一貫したやり方で，機械的に実行可能であること．

（基準τ）元の自然言語の語句，文書のすべての部分を用い，それらの順序関係を損なわないこと．

本論文は論理プログラミングが，自然言語表現の自然さと，プログラミングの汎用性との間のトレードオフをどのように緩和できるかについて，Prologを用いた具体的なプログラム例を通じて探求する．既存のプログラム言語についての，たんなる日本語化にとどまらず，文書から論理プログラムを自動抽出したり，あるいは助詞の使い分けによって生じる微妙な意味の変化を，論理プログラムにおいて扱えるようにしたりすることを目的とする．

論理プログラミング[7]とは，論理式の形でのルール（規則）の集合としてプログラムを構成する方法であり，そのプログラムの実行は，入力データを与えると，それに適用できるルールをシステムが自分自身で見つけて，ルールごとに予め定められたアルゴリズムを適用する．Prologは最初の論理プログラミング言語でありその代表である．[8] 現在利用可能な処理系の多くははエジンバラ記法の流れをくみ，その国際標準はISO/IEC 13211-1, 2 である．[9]

アプリケーションソフトウエアとして見ると，論理プログラミングはルールベースシステム[10]におけるルールを論理で表現して実装したものとみなせるが，他方，論理プログラミング言語はプログラミング言語（言語プロセッサ）であり，特定のアプリケーションに限定されない汎用性をもつ．Prologは論理プログラミング言語として最も古く，かつその代表的なものである(Sterling and Shapiro, 1994; Clocksin, 1997; Bratsko, 1994; Deransart, Ed-Dbali, and Cervoni, 1991)．Prolog[11]の動作原理は背理法の一種（導出原理と呼ばれるその自動処理）であり，クエリーされた条件（ゴール節と呼ばれる）の否定が矛盾を導く（つまりゴールが成功する）ことを証明しようとする．証明に用いる節は，バックトラックと呼ばれる機構によって縦型探索される．導出原理は自然言語処理とは独立に，自動定理証明の研究から提案された技術である．自動定理証明については自然言語プログラミングに直接かかわらないので，紙幅の都合により，ここでは詳しく述べないことにする．

助詞をファンクター（関数記号）とみなすことは，日本語に近い表現でPrologプログラミングを行う簡便な方法である．[12]例えば「太郎 は 日本人 ですか。」「何 が ホニュウ動物 ですか。」といったクラス分類とそのメンバーに関する質問や「おつり は いくらですか。」といった計算を伴う質問に答えることができる．同アプローチを自然言語インタフェースに応用すると，例えば

(q.1) ?- 1000 から 999 を引くと Y です．（ENTERキーを入れる）
Y = 1.

のような質問応答を実行できる．これ以降，Prologに入力する質問の最後のENTERキーは省略する．また各質問事項例について，プロンプト?－の前の括弧内に(q.番号)のように質問の番号を振る．
なお質問例(q.1)で用いたプログラムは以下のようである．

:- op(500, yfx, から).
:- op(500, yfx, を).
:- op(500, yfx, と).
:- op(900, yf, です).

X から Y を 引く と Z :- Z is Y - X.
Z です:- Z.

Prologの知識表現は述語(predicate)を基本とし，それぞれの述語は真(true)か偽(false)の値を持つものと解釈される．またPrologはユーザから質問された述語に対して，それが真であることを（ゴールとして）証明しようとするが，その際，導出と呼ばれる背理法の一種を用いる（2節および付録A参照）．論理プログラミングと呼ばれるのは，このようにプログラムを一階述語論理の形式に則って書き，またそれを機械的に証明しようとするからである．他方，一階述語論理の枠組みからは外れるが，便利な組込述語が用意されているので，これを利用して日本語の別名をつけたプログラムを書いて再利用することもできる．

集めて数える(_述語, O, N):-
findall(_述語, _述語, O),
length(O, N).

上述の「集めて数える/3」という3引数の述語は，データベース述語findall/3に，リスト記法の要素数を数えるlength/2を組み合わせて，任意の述語の形式を，「_述語」という変数に入れて，収集結果をOのリスト形式に出力し，Oの要素の個数を変数Nの値とする．ただし，Prologでの代入は単一化可能なもの（項）に限られる．原則，強制的に代入（破壊的に代入）することができなず，もし単一化できない項を代入しようとすると，その試み（質問，ないしゴール）は失敗し，falseの値が返される．

述語は，数以外の定義域と値域を許す一般化された（陰）関数についての制約条件を表しており，単一化代入はその解と解釈することができる．その特殊な場合として，簡単な連立方程式を解くこともできる．

(q.2) ?- _二次方程式 = X ^2 - 2 * X * Y =:= 3, _解 = (x = X, y = Y), findall(_解, (between(0, 10, X), between(0, 10, Y), _二次方程式), _すべての解), nl.

_すべての解 = [(x=3, y=1)].

上述の質問例は，二次方程式$X^2-2XY-3=0$の解である，0から10までの間の自然数のペアXとYをすべて求めている．ちなみに本論文では用いないが，論理プログラミングの発展的応用の一つとして，より複雑な制約条件の組を解くための技術が研究・開発されており，CLPライブラリとして追加されている処理系もある．

ところで，「クジラはホニュウ動物」や「姉は留守にしております」における助詞「は」は，単独の文として不自然ではなく，主題の「は」と呼ばれる．また「雨が降っている」や「カエルが泳いでいる」は，やはり単独の文として自然であり，中立叙述の「が」と呼ばれる．これらの文の「は」と「が」を入れ替えたとき，非文ではないが，不自然な表現になる．「雨は降っている」は，「雨は降っているが，雪は降っていないのか」といった

言外の比較の意味合いが生じる．これは対比の「は」と呼ばれる．また「クジラがホニュウ動物」や「姉が留守にしております」は不自然であり，クジラだけが，あるいは姉だけがといった他の主語を除外するニュアンスを帯びる．これは総記の「が」と呼ばれる．こうした助詞の言い回しによる意味の変化は非母国語学習者向け教材にも反映されているようである．他方，日本語を母国語とする者にとっては，暗黙知の一種であり，むしろルールとして言語化しにくいものであろう．

助詞「は」と「が」の議論は古くからあったが，概念や用語法には混乱が見られる．「は」に先行する語句は，既知の事項や旧い情報（主題ないし話題）を示し，「が」に先行する語句は未知の事項や新しい情報を，その主語として示すとされた（柴谷・影山・田守，1983）．他に，主語，主格，提題，題目，話題などといった異なる構文上のカテゴリーないし意味上の役割として考察される（三上，1963）．また文の焦点の位置（情報構造）や文脈からの予測可能性（談話的機能）との関連が論じられている(柴谷ら，1983；井上，1989；久野,1973)．さらに「おいしくはない」のように名詞以外の語句の限定する「は」や，「水が飲みたい」の「が」のように目的語を示す「が」もある（詳細は文献に譲る）．「対比」「総記」のような自然言語の使用者がほとんど意識せず使い分けできるにもかかわらず，その説明に窮する（犬童・松田・太田，1989；久野，1973）．総記(exhaustive listing)は，「クジラがホニュウ動物だ」のように，「クジラだけがホニュウ動物」の意味を生じる．また対比(contrasting)は，「風は吹いている」（が雨は止んでいる）のように，実際に言及されていない文を意識させる(Kuroda，1965；久野，1973；井上，1989)．

本論文では，日本語に近い文表現で論理プログラミングをするという目的から，従来の日本語プログラミングにおいて顧みられなかったと思われる，既知と未知，情報の新旧，対比や総記といった自然言語の認知的側面を，Prologによって，忠実に，かつできるだけ簡潔に再現する．しかしながら言語学の諸概念を論じることが主たる目的ではなく，便宜的に助詞を文字列の区切り記号のように扱い，また例題として扱う文の形式も，一部例外（文書からの直接抽出の6～8節）を除くと，高々数単語を助詞で結合したものにすぎない．総記と対比は，言外の語ペアを文中の語が鎖状につながるネットワークにおいて，不安定要因となる枝（関係）を排除したり，追加したりすることによってその安定性を保つ．これは言語学とは別の分野であるサプライチェーンの安定ネットワーク(Ostrovsky，2008)と形式的に一致し，またAI研究におけるサーカムスクリプション(McCarthy，1980)の概念と類似する．本論文の9節において，この安定性条件は，総記や対比の解釈に適用される．

本論文の以降の部分は，大きく分けて3つの部分からなる．最初の部分は5節までであり，前半部分は，論理プログラミングに馴染みのない読者のため，基本的な知識や使い方の説明を兼ねている．Prologにすでに2節でPrologとその日本語化についていくつかの代替的方法を述べた後，3節で簡単な日本語コマンドラインインタフェースを説明し，また4節では助詞をファンクターとした日本語プログラミングの実例を示す．予備知識のある読者は，ざっと目を通した後，5節からお読み頂いてかまわない．5節は，前節の方法で，とくに助詞「は」を用いた場合を，知識の参照と獲得の認知プロセスとみなす．「は」や「が」の言い回しをめぐる問題については，9節で立ち戻る．5節と9節だけを，独立して読むこともできる．

中間部分の6節から8節では，日本語で書かれた文章からのPrologプログラム抽出（プログラム自動合成）をデモンストレーションする．この部分は助詞をファンクターとして用いる前節までの方法の欠点を回避し，日本語の文書を直接論理プログラムに変換する．しかしながら，前半の3～4節の手法を用いていないため，独立に読むことができる．具体的には相続税にかんするサンプル文書から，Prologの節プログラムの表現を自動抽出する．まず6節では文書のアウトラインを見出し語に注目してルール節（確定節）として抽出する．つぎに7節では数式行部分をPrologの算術式に翻訳することを試みる．また8節では抽出された数式内の変数の参照関係を分析する．

最後の2節では，定式化の厳密さや技術的な詳細を追求するのではなく，論理プログラミングとその自然言語プログラミングへの応用を，より広い文脈である人間知能や組織における情報システムの問題に埋め込む．9節では助詞「は」と「が」を巡って，既知と未知，および情報の新旧，総記と対比といった論点をPrologプログラム化によって明確化し，そのメカニズムを説明する．総記と対比の認知過程を，ネットワークモデルの安定性，およびサーカムスクリプションとの関連を論じる．またこれによって，総記や対比が導出の類似物であることを明らかにする．10節では認知心理学における条件文課題研究や，行動経済学におけるバイアスやフレーミングとの関連にも言及する．これら意思決定関連諸分野は，認知科学，行動経済学，意思決定支援システム，そして次の節での経営情報学分野に学際的に広がる．11節は，潜在的な適用領域についての思弁である．仕事の依頼や契約に伴う言語的コミュニケーションを，自然言語とプログラミング言語とを結ぶエージェンシー関係とみなすことを通じて，組織合理性の追求とその情報システムにかんする一般的な捉え方を示唆する．またAI活用に関する5種類の非倫理的リスクや職種ごとの自動化リスクの問題に言及する．最後に12節で結論を述べる．付録AはPrologプログラムの構文や意味，利用できる処理系などについてやや詳しく述べる．

なお本文中や付録のソースコードは，本論文で提案された日本語論理プログラミングにかんする研究ないし教育上のアイディアを，実例を挙げて具体的にデモンストレーションするためのものである．[13] 付録Bは組込述語を日本語化するアプリケーション例を示す．付録Cは相続税の計算についてのサンプル文書から数式を抽出するプログラムを説明する．また数式内の変数間の参照関係を分析して視覚化する．

2 論理プログラミングと日本語 Prolog

本節はPrologの基本的な特徴を述べ，またその日本語での書き方について具体例を用いて説明する．文法や背景など，より詳しい説明については付録Aを見られよ．なお本研究では，プログラムの実行テスト用の処理系として， SWI-Prolog Windows 64bit版を用いる．[14] 本論文の目的に照らして，日本語文字が扱えて，一般ビジネスユーザが多いWindows OS上で比較的安定に稼働するSWI-Prologであれば，小規模の試験的なプログラム開発に十分であると思われる．また簡単のため，一部非ISO標準組込述語を採用する．とくに文字列の分割 split_string/4 と term_string/2 を8節と付録Cで用いる．

Prologプログラムでは事実，ルール，およびゴールの3種類の節を用いる．ゴールは質問とも呼ばれる．節(clause)は，それぞれ末尾にピリオドをつける．節にはルール（rule）と事実(fact)の2種類があり，項(term)と呼ばれる部品を組み合わせて作られる．節自体もまた項の一種である．

ルールは，“:-”（ネック記号）で区切られた頭部と本体を持ち，本体の条件がすべて真であるとき，頭部は真である（正確には「成功する」）という意味をもつ．[15]

```
ルール:-　ルールの本体の条件.
別のルール:-　ルールの本体の条件1，ルールの本体の条件2.
もう一つ別のルール:-　本体の条件1，本体の条件2，本体の条件3.
```

事実節は条件部を持たずに成功する．つまり事実は頭部だけかならなる節であり，条件部がtrueであるルールと同じ意味をもつ．

事実の例です．

Prologシステムをインストール後，メイン画面を立ち上げるには，作業ディレクトリに拡張子’.pl’の付いた空のテキストファイルを作り，ダブルクリックするのが便利である．しかじかのメッセージが表示された後，プロンプト ?- が現れる．コマンドラインからの入力を質問（クエリー）と呼ぶ．質問はゴール節として入力し，最後にピリオドをつける．

ゴール節内の各変数に対して単一化できる代入（変数がないときは空代入）を構成し，それによってゴールの真の事例が見つかると，ゴールは成功し，trueがその値として返される．またそのような代入が見つからなければ，ゴールは失敗し，falseが値として返答される．

(q.3) ?- _ゴールの例 = ここに質問して半角ピリオドとENTERを入力します.
_ゴールの例 = ここに質問して半角ピリオドとENTERを入力します.
(q.4) ?- f(X, b) = f(a, Y).
X = a,
Y = b.
(q.5) ?- 0 is 3 -2.
false.

述語はその引数としての変数をもつことができる．変数名には半角アルファベット大文字で始めるか，あるいは先頭に半角下線を付ける．日本語文字の場合は一部の文字を除いて後者の方法を用いることができる．SWF-Prologでは，全角アルファベット，ギリシャ文字，ローマ数字などの各種文字の大文字で始めることも許されるようである．

変数を用いたルールの例(_引数):- ルールの本体の条件(_引数).
変数を用いたルールの別例(_引数1, _引数2):- ルールの本体の条件(_引数1, _引数2).

(q.6) ?- _各種文字の変数を用いたリスト = [_日本語, Θ, X, Abc, Ⅱ, Ⅳ].
_各種文字の変数を用いたリスト = [_日本語, Θ, X, Abc, (Ⅱ, Ⅳ)].

未代入の変数は，各節ごとに全称と仮定される．Prologの動作は，導出と言って，背理法を自動化したものであり，ゴールの否定が失敗することを証明しようとする三段論法の一種である．例えば，「魚は泳ぐ．メダカは魚である．よってメダカは泳ぐ」という推論は，「メダカは泳ぐ」をゴールとして，仮にその否定「メダカは泳がない」を正しいとすると，「魚は泳ぐ」ことが既知なので，“メダカ＝魚”という単一化が失敗する．そうすると「メダカは魚である」というもう一つの既知の事実と矛盾する．つまりゴールの否定「メダカは泳がない」を追加することは不可能であるので，「メダカは泳がない」が正しいと推論できる．

泳ぐ(_何か) :- 魚(_何か).
魚(メダカ).

(q.7) ?- 泳ぐ(魚).
true.

（練習問題）
以下のプログラムを完成させて，「メダカ は 泳ぐ」という質問に成功するようにしなさい．

魚 は 泳ぐ.
メダカ は 魚 です.

導出原理は，単一化代入を用いてゴールを背理法によって証明する．その探索はバックトラックによって制御され，原則的にすべての可能性を調べつくす（導出木の縦型探索）．単一化やバックトラックは関数プログラミングや制約処理アプローチに標準として組み込まれていない機構であり，Prologの特徴の一つである．[16]
例えば次のように，カンマで区切って複数のゴールを並べることもできる．

(q.8) ?- X = [a, b, c], X = [Y | _].

リストはカギ括弧内をカンマで区切った項の並びで，[A | B] はリストの先頭要素Aとそれ以降の部分Bを表す．下線は匿名変数である．ENTERを入力すると，背理法の一種を用いてゴールの否定を証明しようとする導出原理(resolution principle)，およびゴールに含まれる変数に代入される項は，単一化と呼ばれるアルゴリズムにしたがって自動的に作成される．[17]

X = [a, b, c],
Y = a.
true.

上の例におけるクエリーは，リストXの先頭要素Yを質問している．Prologはルール節の本体内の条件を逐次下位目標としてクエリーする．もし充足可能な代入が見つかればそのゴールが成功したとして代入された値の組を返答し，またもしそのような答えがみつからなければそのゴールは失敗だと答える．
Prologのプログラムは，論理的な意味に沿って読めて，節の並びは直接プログラムの実行順序とは直接関係しないという意味で，「宣言的」プログラミングと呼ばれる．しかし実際には，一階述語論理の意味である真偽値とは独立の副作用（標準入出力やファイルの読み込みと書き出し，節の登録や削除，データベース述語やカットなど）を用いて手続き的な使い方をすることができる．[18]
以下に，よく使われる述語をいくつか日本語化してみよう．

改行します :- nl.
書き出す(_書き出す内容) :- write(_書き出す内容).
つねに成功します:- true.
強制的に失敗します:- fail.
ここから先は失敗すると後戻りすることはできません:- !.

これらの組込述語は以降の例題においても頻繁に用いられる．次の2述語もよく用いられる．

ユニブ(_項, [_ファンクター | _引数リスト]):- _項=.. [_ファンクター | _引数リスト].
節を参照する(_ルールの頭部, _本体):- clause(_ルールの頭部, _本体).

ユニブは左辺の項をファンクターとその引数からなるリストに分解，あるいは右辺のリストを項として左辺に組み立てる双方向の高階述語である． clause/2は内部データベースの節を検索し，存在すれば第1引数にルール頭部，第2引数に本体を返す．ただし事実節の第2引数はtrueを返す．ユニブ，clause，カット，assert，writeなどは，いずれも一階述語論理の外ではたらく副作用である．3節では，これを利用して，簡単な自然言語インタフェースを作る．

3 日本語コマンドラインインタフェース

Prolog 言語は，直接的に日本語でプログラミングすることも可能である．基本的な述語を日本語の別名でルール化したり，ユーザ定義述語を日本語で定義したりすることで，デバッグ時に処理の流れを追跡することが容易になる．またそれは古典的なAIシステムで実現された文章理解と問題解決の融合を，現代のIT 環境で実践的に活用することの可能性を示唆する．本節では，直接的に述語名を日本語文字列として表現することによって，日本語でコマンドラインからPCを操作する例を紹介する．

3.1 積木の世界から自動化へ

現代のコンピュータは，マウスやタッチパッドなどの入力デバイスから，ウィンドウ，フォルダやファイルのアイコンをクリックし，ドラックアンドドロップすることで，直観的に操作でできる．これはグラフィックユーザインタフェース（GUI）と呼ばれる仕組みである．それ以前から利用されていたのは，（Windowsで言う）コマンドプロンプトから，ユーザがキーボードを使って，シェルプログラムのコードを直接入力する，あるいは，マクロプログラムやシェルスクリプトを使って自動処理させるといった方法である．これはコマンドラインユーザインタフェース（CUI）と呼ばれる．

1970年代初頭に公表されたAIシステムSHRDLUは，文章理解と問題解決を融合し，線画として描かれた積木の世界で，コマンドラインからの自然言語文での指令に対し，SHADLUが積木を移動させた(Winograd, 1972)．スタンフォード大のWinogradのウェブページからはソースコードや実行デモを閲覧することができる．同頁とそこからリンクされたWikipedia英語版のSHRDLUの項目によると，SHRDLUはLISP言語と，同時期にWinogradらが開発に携わったMicro-Plannerが使われている．LISPやPLANNERは，Prologのような節形式と導出原理に基づく論理プログラミングに対峙する，もう一つの記号処理的AIアプローチ「知識の手続き的埋め込み」として提案されたものであった．

SHRDLUの積木の世界を簡略化して日本語化したProlog版も知られる(高野，2014)．高野のプログラムでは，積木を動かす指令の文に現れる名詞や動詞の同義語を，意味カテゴリーとしてまとめた単語辞書を用意し，文を助詞によってこれらの意味カテゴリーが結合したものとみなすことによって，ユーザの与えた自然言語表現を積木移動のルールに翻訳する．やはり助詞を用いて単語カテゴリーを結合する方法が用いられており，通常の品詞に基づく文法理論の代わりに，高野(2014)では意味文法と呼ばれる．小谷(1993)ではメタ述語を用いて，助詞によって結合された述語名を日本語化する方法を提案している．小谷の方法では，助詞と名詞などのそれ以外の語句との間のスペースは不要であり，他方，述語引数部分は各引数を日本語化するので，元のProlog構文に近い．太細他(1984)は限定された日本語で表現されたプ

ログラムからPrologプログラムを自動合成するPLINシステムを紹介している(pp.188-195). PLINシステムはメインフレーム上に実装された初期の国産Prologで作成され, 100ステップ程度の小さなアプリケーションである.

現代のスマホやスマートデバイスの音声アシスタンス機能は, 明示的にはユーザによるプログラミングを目的としていないが, 自然言語表現を, 内部的に実行可能なプログラムに適切に対応させるので, 特殊用途の自然言語プログラミングの機構と言えるかもしれない. 音声アシスタンスの商用技術は, クラウド環境下で蓄積されたユーザデータの下で機械学習された音声認識システムと, ユーザと対話的にやりとりする自然言語インタフェースとしてのチャットボットを動かすルールベース/事例ベースの組み合わせと想像されるが, その詳細は不明である. しかし機能としては, SHRDLUにおける積木の世界における積木の移動が, スマホやPCの状態を遷移させることに置換わっただけである.

3.2 シェルコマンドを手続付加して日本語で PC を操作する

Prologでは組込述語shell/1を用いてシェルコマンドを発行することができる. 次の例はカレントディレクトリのファイル一覧をdir.txtというファイルに書き出す.

(q.9) ?- shell('cmd.exe /C dir > dir.txt').

組込述語assert/1を用いて, 日本語コマンドをルールとして登録し, 実行することもできる.

(q.10) ?- assert((カレントディレクトリのファイル一覧を保存:- shell('cmd.exe /C dir > dir.txt'))).
(ENTERキー入力)
true.
(q.11) ?- カレントディレクトリのファイル一覧を保存.
true.

SWI-Prologには, 組込述語として, win_shell, win_execもある. [19]

(q.12) ?- assert((検索サイトを開く :- win_shell(open, 'https://www.google.com/ '))). (ENTERキー入力)
true.
(q.13) ?- 検索サイトを開く.
true.

このように, 特定の用途に対するルール節にshell/1などで紐付けられた手続きを, Prologから日本語で呼びだすことができ, それによって日本語コマンドラインインタフェースを作成できる.

こんにちは世界をファイル保存:- shell('cmd.exe /C echo Hello World! > hello.txt').
メモ帳でこんにちは世界を開く:- shell('notepad.exe hello.txt').
コマンドプロンプトを開く:- win_exec(open, 'cmd.exe').

その自然言語表現を組み合わせて用いることもできる．次のクエリーは，“Hello World!”をテキストファイルに書き出し，メモ帳で開く操作を，続けて実行する．

(q.14) ?- こんにちは世界をファイル保存，メモ帳でこんにちは世界を開く.
true.

(q.15) ?- エクセルのある場所に移動，コマンドプロンプトを開く.
true.

述語変数を付けた表現の例も示しておこう．

```
カレントディレクトリ( P ):- working_directory( P, P ).
三桁毎のカンマ区切りにする( _数, _金額 ):- format( atom( _金額 ), '~D', [ _数 ] ).
```

(q.16) ?- 三桁毎のカンマ区切りにする(1050,　_金額).
_金額 = '1,050'.

3.3 ファイル入出力

またCSVファイルの読み書き，Webページの読み込み，XMLファイルの解析など，処理系に用意された各種のライブラリを利用することもできる．[20]

次のプログラムはcsv_read_file/3を利用し，CSV形式のファイルから各行をカンマ区切りで読み込む．

```
'CSVファイルから行を読む'( CSV, J, A, B ) :-
        atom_concat( CSV, '.csv', File ),
        csv_read_file( File, Data, [ ] ),
        nth1( J, Data, row( A, B ) ).
```

上記プログラムの実行例として，keystrokes.csvというファイルから各行を読む．

(q.17) ?- 'CSVファイルから行を読む'(keystrokes, J, A, B).　（ENTERキー入力）
J = 1,
A = 'BACKSPACE',
B = '{BACKSPACE}' ;
J = 2,
　A = 'BACKSPACE',
　B = '{BS}' .

このように，述語名として目的の処理に即した意味のあるまとまりに対応する文字列を選び，コマンドラインからのシェルコマンド実行をルールの本体に結合した副作用として記述いておくことにより，PC操作を自動化することが可能である．またWSH（バッチファイル）やPowershellのスクリプトプログラムを呼び出して複

雑な作業をさせることもできるだろう.

3.4 組込述語を日本語化する

前節の方法に即して, Prolog内のデータをファイルに書き出すプログラム例を示す.

アルファベットの小文字とその文字コードをファイルに保存:-

```
tell( 'alphabet.txt' ),
atom_codes( az , [ I, J ] ),
between( I, J, K ),
char_code( C, K ),
atomic_list_concat( [ C, K ], ',', A ),
nl,
write( A ),
fail
;
told.
```

上記のプログラムを"csv.pl"などとファイル名を付けて, カレントディレクトリに保存する. ただし日本語化したときのため, 文字のエンコーディングをUTF-8にしておく. [21]プロンプトで次のように実行し,

(q.18) ?- [csv]. (ENTER入力)

次のように質問する.

(q.19) ?- アルファベットの小文字とその文字コードをファイルに保存. (ENTER入力)

'alphabet.txt'という名前のファイルがカレントディレクトリに作成される. メモ帳で開いて確認する.

```
a,97
b,98
…
z,122
```

上記のプログラム中には常套的な組込述語が複数含まれている. ルール本体の最初は, ファイル書き込みの始めを意味する tell/1 であり, その完了を told/0 で記して囲む. toldの直前に置かれたセミコロン;は選言(OR)演算子である. ちなみにセミコロンの代わりにピリオドで区切って, もう一つ"アルファベットの小文字とその文字コードをファイルに保存:- told."のルール節を追加しても同じである. 組込述語 atom_codes/2, char_code/2, between/3, atomic_list_concat/3, nl/0, write/1は, それぞれアトムと文字コードリストの変換, 文字とその文字コード, 整数区間, 改行, 書き出しである. 特殊な組込述語fail/0は失敗であり, バックトラックを強制する.

それぞれの組込述語に，日本語の別名を与えると以下のようになる．

ファイル書出しを開始します(_ファイル名):- tell(_ファイル名).
アトムの文字コードリスト(_アトム, _文字コードのリスト):- atom_codes(_アトム, _文字コードのリスト).
文字のコード(_文字, _文字コード):- char_code(_文字, _文字コード).
区間内の数(_はじめ, _おわり, _数):- between(_はじめ, _おわり, _数).
アトム的リストの結合(_リスト, _区切り文字, _結合した文字列):- atomic_list_concat(_リスト, _区切り文字, _結合した文字列).
改行します :- nl.
書き出す(_書き出す内容) :- write(_書き出す内容).
強制的に失敗します:- fail.
または(前，後):- ';'(前，後).
ファイル書出しを終了します:- told.

上述の方法は一つずつ手作業で組込述語に日本語の別名を与えている．述語名の対応表から，まとめて日本語化したPrologプログラムを作成する例を付録Bとした．日本語で書かれたPrologプログラムの豊富な事例はウェブ上でも見つけることができる[22]

3.5 ルールの分解と抽象化

前節のプログラムは，一つのルール内に組込述語をそのまま用いた条件を複数並べている．実践的に，Prologプログラミングに慣れたユーザは，コマンドラインに直接，複数の組込述語を並べて目的の動作を遂行しようと試行錯誤することがあるだろう．また成功したゴールをエディターにコピーして，改めてルールとして述語名を付け，プログラムファイルとして整形して保存するといった作業フローもよく経験する．しかし，そのままでは可読性が低下し，他者（将来の自分自身を含めて！）がデバッグや修正を行う際，理解のために余分な時間を要することも起きてくる．

ルールを分割し，適切な述語名を改めて付け直すだけでも，可読性を向上し，デバッグや修正に要する時間と手間を節約し，再利用可能性を高めることができるかもしれない．前節のプログラムを例にとると，次のようにルールの処理内容を分割するのがよいと思われる．

```
アルファベットの小文字の文字コード( C, K ):-
        atom_codes( az , [ I, J ] ),
        between( I, J, K ),
        char_code( C, K ).

リスト内の要素をカンマ区切りで書き出す( _リスト ):-
        atomic_list_concat( _リスト, ',', A ),
        nl,
        write( A ).

アルファベットの小文字とその文字コードをカンマ区切りで書き出す( C, K ):-
```

```
        アルファベットの小文字の文字コード( C, K ),
        リスト内の要素をカンマ区切りで書き出す( [ C, K ] ).

条件をバックトラックしながらファイルに保存( _ゴール, _ファイル名 ):-
        tell( _ファイル名 ),
        _ゴール,
        fail
        ;
        told.

アルファベットの小文字とその文字コードをファイルに保存する:-
        _ファイル名 = 'alphabet.txt',
        _ゴール = アルファベットの小文字とその文字コードをカンマ区切りで書き出す( _, _ ),
        条件をバックトラックしながらファイルに保存( _ゴール, _ファイル名 ).
```

このようにルールを分割しておくことによって，ルールが抽象化されて他のルールから呼び出すことも可能になり，述語の再利用可能性が向上する．またプログラムの可読性が向上し，デバッグしやすくなる．

3.6 デバッグコマンドを使う

実際，この例に対して，デバッグ用の述語を使ってみよう．質問されたゴールの導出過程とバックトラックを追跡（トレース）開始する組込述語は，trace/0 である．また spy/1 はスパイポイントを設定し，指定した述語が（下位）ゴールで呼び出された時点からトレースを開始する．

まず「アルファベットの小文字とその文字コードをカンマ区切りで書き出す/2」にスパイポイントを設定しておく．

```
(q.20) ?- spy( アルファベットの小文字とその文字コードをカンマ区切りで書き出す/2 ).
% Spy point on アルファベットの小文字とその文字コードをカンマ区切りで書き出す/2
true.
```

次に「アルファベットの小文字とその文字コードをファイルに保存するする/0」を質問する．トレースが開始されると，デバッガーが現在実行中の下位ゴールを表示する．各行末で，改行（クリープ），s（スキップ），u（上位ゴールに戻る）などを適宜入力する．

```
(q.21) [debug] ?- アルファベットの小文字とその文字コードをファイルに保存する.
 * Call: (10) アルファベットの小文字とその文字コードをカンマ区切りで書き出す(_7912, _7914) ? creep
   Call: (11) アルファベットの小文字の文字コード(_7912, _7914) ? skip
   Exit: (11) アルファベットの小文字の文字コード(a, 97) ? creep
   Call: (11) リスト内の要素をカンマ区切りで書き出す([a, 97]) ? skip
   Exit: (11) リスト内の要素をカンマ区切りで書き出す([a, 97]) ? creep
 * Exit: (10) アルファベットの小文字とその文字コードをカンマ区切りで書き出す(a, 97) ? creep
```

Call: (10) fail ? skip
Fail: (10) fail ? creep
Redo: (11) アルファベットの小文字の文字コード(_7912, _7914) ? skip
Exit: (11) アルファベットの小文字の文字コード(b, 98) ? up
＊Exit: (10) アルファベットの小文字とその文字コードをカンマ区切りで書き出す(b, 98) ? up
Exit: (9) 条件をバックトラックしながらファイルに保存(アルファベットの小文字とその文字コードをカンマ区切りで書き出す(_7912, _7914), 'alphabet.txt') ? up
Exit: (8) アルファベットの小文字とその文字コードをファイルに保存する ? up
true.

Prologの組込述語を日本語述語名に読み替えるアプリケーションを付録Bとした. 述語名の読替え対応表をCSVファイルに作成するだけで, 簡単にユーザ独自の日本語バージョンPrologを作成することができる. 例えば, 子供向けPrologを提供することができるだろう(読者の練習とする). 簡便な方法であるが, プログラムの理解容易性の向上, 非熟達者によるプログラミング生産性向上, プログラミング教育場面での利用可能性などといった効能が期待できる. 次節では助詞をファンクターとして便宜的に活用する代替的なアプローチについて詳しく述べる.

4 日本語論理プログラミング

日本語の利点として, Prologのファンクター定義を用い, 助詞を述語として用いることができ, それによって自然な文に近い節プログラムを作成できる. 実際に, 犬童(2018, January 22)ではスマホ画面からの実行例が示されている. これは前節で述べた日本語文字種による述語と変数の記述からさらに進んで, 類似した本体の構造を持ったルールを抽象化し, 可読性や再利用可能性を向上させることが期待される.

4.1 ファンクター: 優先順位と項の型

文字列を文法規則に則った品詞カテゴリーの並び(文)として認識する自然言語処理プログラムは, パーサと呼ばれる. Prologシステムは確定節文法(DCG)のパーサを標準的に提供する. PRLOGは確定節なのでDCGの書き方とよく似ている(9節も参照). しかしβ部分に副作用を記述できないので, 自然言語インタフェースを作るには少し不自由である. ここではより簡便な代替的方法を紹介する. 助詞や動詞の品詞カテゴリーを文法的な厳密さにこだわらずに, より柔軟に解釈し, Prologのファンクターとして利用する. 例として, 足し算を日本語表現してみよう.

次のルールは, 組込述語pwd/0を手続きとして付加して, カレントディレクトリを表示させる.

(プログラム)

```
:- op( 500, yfx, を ).

カレントディレクトリ を 表示する :- pwd.
```

(実行例)

```
(q.22) ?- カレントディレクトリ を 表示する.  (ENTERキー入力)
```

```
% c:/users/xxxxx/documents/prolog/
true .
```

あるいは，次の行を上記プログラムの後ろに追加しておくことで，プログラム読込み時に自動実行させることができる．

```
:- カレントディレクトリ を 表示する.
```

上述のプログラムでは，助詞をファンクターとして定義するため，組込述語op/3を用いている．SWI-Prologのヘルプファイル[23]によると，第1引数は0から1200までの間の整数で，優先順位(precedence)を表す．優先順位 0 は宣言を除外する．第2引数は型(type)を表し， xf, yf, xfx, xfy, yfx, fy, fx のうちのいずれかである．‘f’はファンクターの位置で， x や y は引数の位置を示す．’y’はその位置に現れるのがファンクターの優先順位と同じかそれ以下でなければならないことを表し，’x’はファンクターの優先順位より厳密に低くなければならないことを意味する．各助詞に対するファンクター定義の方法には厳密な基準がないが，上述のプログラム例では，優先順位500，型yfxを割当てている．これは足し算”+”，引き算”−”，および論理演算子の一部と同じである．

4.2 足し算

文章で足し算をするルールを追加しよう．日本語文字列の変数名は，先頭に半角下線をつける．

```
:- op( 500, yfx, に ).
:- op( 500, yfx, と ).

_数1 を _数2 に 足す と _数3 :-
        ¥+ var( _数1 ),
        ¥+ var( _数2 ),
        _数3 is _数1 + _数2.
```

上述のルールでは，最初の2引数が変数でないことをチェックしてから，次に引き算を実行する．最初の2引数には数値または計算式が入力されている必要があり，変数「_数3」に答えが代入される．

また「○○に」と「○○を」の位置を交換したときも，上述の結果と一致させる．

```
_数1 に _数2 を 足す と _数3 :-
        _数1 を _数2 に 足す と _数3.
```

4.3 引き算

同様に，引き算を次のように表現することができる．

```
:- op( 500, yfx, から ).
```

```
_数1 から _数2 を 引く と _数3 :-
        ¥+ var( _数1 ),
        ¥+ var( _数2 ),
        _数3 is _数1 - _数2.

_数2 を _数1 から 引く と _数3 :-
        _数1 から _数2 を 引く と _数3.
```

プログラムを保存し再読み込みしたら，実際にいくつか計算問題を質問してみよう．

```
(q.23) ?- 3 を -10 に 足す と X.
X = -7.

(q.24) ?- 100 から 22 を 引く と X.
X = 78.

(q.25) ?- 3 に ( -100/10 ) *2 /sqrt(4) を 足す と X.
X = -7.0 .
```

練習として，前出の算術のプログラムと組み合わせた，買い物のおつりの金額を計算させよう．ここではプログラム例を提示するにとどめる．さまざまな変種が考えられる．読者自身で試みられたい．

（プログラム例）

```
:- op( 1000, xfy, '、' ).
:- op( 550, yf, 円 ).
:- op( 1000, xfy, とすると ).
:- op( 500, yfx, おつりは ).

支払金額 を _支払金額 円、請求金額 を _請求金額 円 とすると、
おつりは _いくらですか :-
        _支払金額 から _請求金額 を 引く と _おつりの金額,
        _いくらですか = _おつりの金額 円.
```

（実行例）

```
(q.26) ?- 支払金額 を 10000円、請求金額 を 8760円 とすると、 おつりは _いくらですか.
_いくらですか = 1240 円.
```

4.4 やや複雑な計算，例外処理

より複雑な数式はPrologの組込述語を使えばよいだろう．ここでは全角の等号「＝」をファンクターとして，組込演算子の is/2や=:=/2 と読み替えるのが最も簡単であろう．

最も直接的には,「X ＝ Y :- Y is X」や「X ＝ Y :- Y =:= X」でも可能であるが, 文字列や変数が入力されるとエラーになるケースが発生する. その場合分けはやや煩雑になる. そこで変数判定var/1と例外ハンドリングcatch/3を用いて, 次のように修正するとエラーを回避することができる.

```
:- op( 600, yfx, ＝ ).

X ＝ Y :-
        var( Y ), catch( Y is X, _, fail ), !;
        var( X ), catch( X is Y, _, fail ), !;
        catch( X =:= Y, _, fail ).
```

```
(q.27) ?- 1 + 1 ＝ _答.
_答 = 2.
```

```
(q.28) ?- X ＝ 1 + 2.
X = 3.
```

```
(q.29) ?- X = -3, ( 29 * X  - 13 )^2 ＝ A .
X = -3,
A = 10000 .
```

4.5 言葉で PC を操る

前出のカレントディレクトリの表示プログラムを改良して, 内容取得をファイルに保存, コピー, 移動するといった, 少し複雑な作業を日本語で指示する方法を考えてみよう.

以下のプログラム例では, まず3節の流儀にしたがい,「動作語の辞書/3」「動作語/6」「動作語から実行コマンドへ/4」の3種類の日本語コマンド用ルールを定義する.

```
動作語の辞書( コピーする, copy, [ を, に ] ).
動作語の辞書( 削除する, del, [ を ] ).

動作語( Action, Verb, _ファイル1, P1, _ファイル2, P2 ):-
        動作語の辞書( Action, Verb, [ P1, P2 ] ),
        exists_file( _ファイル1 ),
        ¥+ exists_file( _ファイル2 ).

動作語から実行コマンドへ( _動作, _ファイル1, _ファイル2, _コマンドライン ):-
        _リスト = [ 'cmd.exe', '/C', _動作, _ファイル1, _ファイル2 ],
        _空白 = '¥s',
        atomic_list_concat( _リスト, _空白, _コマンドライン ).
```

次に，これらの日本語コマンド用ルールを基本語彙として用い，助詞「を」と「に」を使って，ファイルを「コピー」する操作を日本語で指示できるルールを定義する．

```
_ファイル1 を _ファイル2 に _動詞 :-
        動作語( _動詞, _動作, _ファイル1, を, _ファイル2, に ),
        動作語から実行コマンドへ( _動作, _ファイル1, _ファイル2, _コマンドライン ),
        shell( _コマンドライン ).
```

またファイルの「削除」を行うルールを，同様に定義することができる．ただし，アリティ（引数の個数）が異なるので，述語としては区別される．

```
動作語( Action, Verb, _ファイル1, P1 ):-
        動作語の辞書( Action, Verb, [ P1 ] ),
        exists_file( _ファイル1 ).

動作語から実行コマンドへ( _動作, _ファイル, _コマンドライン ):-
        _リスト = [ 'cmd.exe', '/C', _動作, _ファイル ],
        _空白 = '¥s',
        atomic_list_concat( _リスト, _空白, _コマンドライン ).

_ファイル を _動詞 :-
        動作語( _動詞, _動作, _ファイル, を ),
        動作語から実行コマンドへ( _動作, _ファイル, _コマンドライン ),
        shell( _コマンドライン ).
```

上述のプログラムを，"anli.pl"というファイル名でカレントディレクトリに保存し，システムに読み込み，次のクエリーを実行する．

```
(q.30) ?- "hello.txt" を "helllo1.txt" に コピーする.
true.
(q.31) ?- "hello.txt" を 削除する.
true.
```

カレントディレクトリ内で，前節で作成したファイル"hello.txt"が別名でコピーされ，元のファイルは削除されたことを確認する．

ちなみに，動作語辞書を用意する理由は，後で「移動」や「ファイル属性の変更」などの，異なる動作にも拡張できるようにするためである．

実際，ファイルの「移動」は，上記のプログラムで定義された動作（の語彙）を組み合わせて表現できる，より複雑な作業である．

まず，「動作語の辞書/3」の2行に次の事実節を追加挿入する．

動作語の辞書(移動する, mv, [を, に]).

次に,「 _ファイル1 を _ファイル2 に _動詞 /3 」のルール節の後ろに, 同じ述語の頭部を持った以下のようなルール節を追加挿入する.

```
_ファイル1 を _ファイル2 に 移動する :-
        _ファイル1 を _ファイル2 に コピーする,
        _ファイル1 を 削除する.
```

```
(q.32) ?- "hello.txt" を "goodbye.txt" に 移動する.  (ENTERキー入力)
true .
```

4.6 文末の表現

ところで, これまでの例題では文末表現が省かれている. 特殊な用途の質問, 例えば金額を尋ねる質問文の文末表現に対して, 特別のルールを与え, 所定の様式で回答を表示させることもできる.

```
_数1 を _数2 に 足す と 'いくらか？' :-
        _数3 is _数1 + _数2,
        atomic_list_concat( [ 答えは, _数3, 'です。'  ], '¥s', A ),
        nl,
        write( A ).
```

より柔軟に, 一般的な文末表現を付加して, より自然な日本語文で質問できるようにするには, 代表的な文末表現について, それぞれファンクター定義しておくことが一つの方法である.

```
:- op( 900, yf, だ ).
:- op( 900, yf, です ).
:- op( 900, yf, である ).
:- op( 900, yf, か ).
:- op( 900, yf, ですか ).
Y だ:- Y.
Y です:- Y.
Y である:- Y.
Y か:- Y.
Y ですか:- Y.
```

試みに「です」体での計算問題を質問してみよう.

```
(q.33) ?-  7 mod 3 に pi を 足す と X です.
```

X = 4.141592653589793 .

4.7 本節のまとめ

以上の例から容易に推察されるように，PC作業で必要となる基本的な作業を，予め動作語彙の辞書として用意し，それらを組み合わせることによって，より複雑な作業を自然言語で指示することができ，それによってPC作業の一部が自動化される．ただし，この方法にはいくつか問題が残される．

課題1　語彙が貧弱で表現できる日本語文が限られていること，

課題2　助詞と単語の間に半角スペースを挿入する必要があるといった，若干の様式上の不自然さ，

課題3　文の意味的内容と演算子優先順位による型とが必ずしも対応せず，複雑な構文が扱えない，

課題4　言い回し（たとえば助詞の使い分け）によるニュアンスの変化などが考慮されていない．

上述の4課題のうち，課題1については，単語DBを用いることによって，少なくとも部分的に，解決できる．例えば，日本語Wordnet(Bond, Isahara, Fujita, Uchimoto, Kuribayashi, & Kanzaki, 2009)は，Prolog版もあり，その語義説明部分はシソーラスとしても利用できる．英語版Wordnetと組み合わせれば，類義語，反意語，換喩などの意味的関連を処理することも可能である．課題2についてはメタ述語を用いる方法(小谷, 1993)では解消される．SWI-Prologにもメタ述語はあるようだが本論文では未検討である．課題3を回避する一つの方法として，6節から8節において，文書そのものからPrologのコードを自動生成する代替案を示す．課題4については，次節および9節で詳しく論じる．

5 「XはY」：既知と未知

日本語の助詞「は」が文の主題を示すこと，あるいは既知の項目や旧い情報を示し，「が」は主語や新しい情報を示すといった観察は，若干混乱しているものの，国語学者や言語学者によって繰り返し主張されてきた観点である．これらは9節であらためて認知科学に関連する話題として触れることとし，本節では，自然言語表現から論理プログラム（節形式）を抽出しようとするとき，助詞「は」を，直接的にユーザの持つ知識をPrologに教える目的で，便宜的に活用する．以降の部分では自然言語表現に近い「XはY」や「XがY」の形の文の意味的処理に対応する論理プログラムを具体的に作成していく．

5.1 内部表現としての関係

例えば「クジラはホニュウ動物」（である/です/だ）という文は，Prologシステム内で，X＝「クジラ」，Y＝「ホニュウ動物」としたときの，関係（X, Y）＝（クジラ，ホニュウ動物）がデータベース内に登録されていて，参照可能な状態にするということだと考えてみよう．つまりこの関係を事実として主張(assertion)するものと考えられる．

もし関係が成立済みであれば既知であり，Prologシステムへのユーザからの質問として入力されたゴール節「クジラはホニュウ動物」は成功する．また関係が未成立のときは，それは未知の内容であるので，ユーザの入力を素直に信じてよいという前提で，Prologシステムはこれを新たに成立した関係としてデータベースに追加する．これは知識の獲得に相当する．それ以外のケースでは，通常の心理的な反応として，無意味として無視する，違和感を覚え拒絶する，もしくは未知の事柄として興味を持って，何らかの追加的な推論や情報取集の動作を開始するかもしれない．

述語「関係/2」は，ユーザの入力から抽出する事実節を，内部的な関係の成立に対応させる．

```
関係( X, Y ) :-
        C =  述語名( X ),
        clause( C, _ ),
        R =.. [ X, Y ],
        clause( R, _ ).
```

上述の「関係/3」の本体の最初の条件では，登録されている述語名Xをclause/2を用いて節検索し，ユニブ =.. でその引数Yと合成した述語Rの形式を作り，再びclause/2を使って検索する．

後の分析に備えて，代替的関係を「関係/2」を用いて定義しておこう．

```
代替的関係( X, Y, Z, W ):-
        関係( Z, W ),
        ( X, Y ) ¥= ( Z, W ).
```

「代替的関係/4」は，ある関係（X, Y）に含まれる要素の一方または両方を取り替えた別の関係（Z, W）である．総記の「が」の場合，代替的関係で，Z≠X, Y＝Wであるものの存在を否定する意味合いが生じる．また対比の場合は，文中のXとYに代替的関係が存在する場合に，Z≠X, Y≠Wである別の代替的関係を探し出そうとする．詳しくは9節で説明するが，これらはいずれも語のネットワークの安定性を保とうとする反応と考えられ，それゆえ2種類のリストの要素の間のマッチング（一対一対応）案から，別のマッチング案への変更が不可能であることを確認する手続きと共通する．

より柔軟な表現方法として，2項関係だけでなくn項の間の多項関係，つまり関係データベースで言うところのタプルとして記述される．ただしタプルは2項関係の集まりとみなすことができる．[24]また2項関係に基づく本論文の情報モデルとデータベースのタプルによる情報表現との関係について，5.10節で再びとりあげる．

5.2 未知の関係

関係(X, Y)が未成立の場合，この後作成する「未知のとき/2」を用いて，新しい知識を獲得する．まずdynamic宣言を用いて，「述語名/1」を動的に変更可能な述語として宣言しておく．

```
:- dynamic(  述語名  / 1 ).

関係を新規に登録する( X, Y ):-
        ¥+ var( X ),
        ¥+ var( Y ),
        R =.. [ X, Y ],
        assert( R ).

未登録のとき述語名を追加する( X ) :-
        C =  述語名( X ),
        clause( C, _ ),
```

```
        !.

未登録のとき述語名を追加する( X ) :-
        C =  述語名( X ),
        assert( C ).

未知のとき( X, Y ) :-
        ¥+  関係( X, Y ),
        関係を新規に登録する( X, Y ),
        未登録のとき述語名を追加する( X ),
        nl,
        write( ″初耳です。″ ).
```

「未知のとき/2」は，未知だった関係を新しく事実節として登録し，同時に「未登録の述語名を追加する/1」を用いて，もし述語名Xが未登録であれば，その述語名も登録し，また登録済みであれば何もせず，最後に「初耳です」と返答する．

また，未登録の述語名Xについて聞かれたとき，知らない（あるいは答えたくない）なら，「知りません」と返答すると便利かもしれない．そこで「未知のとき」に次のようなルールを追加する．

```
未知のとき( X, Y ) :-
        ¥+  述語名( X ),
        var( Y ),
        nl,
        write( ″知りません。″ ).
```

```
(q.34) ?- 猫 は_どのような動物 か.
知りません。
true.
```

ちなみに，疑問詞Xに対する「XはY」は非文法的であるので，Xが変数の場合，質問できない．そこで後出の「XはY」述語では未知のとき非変数をチェックする．「XがY」ではXを疑問詞として質問することが許される．

```
(q.35) ?- _何 が 魚 か .

知りません。
true.
```

これに対処する「未知のとき」の修正は容易であるので読者の練習とする．

5.3 既知の関係

次の「既知のとき/2」は，第2引数の入力値が変数であるかどうかによって，2種類に分かれる．

```
既知のとき( X, Y ) :-
        ¥+ var( Y ),
        関係( X, Y ),
        nl,
        write( ″知っています。″ ).
```

「既知のとき/2」の最初のルールでは，第2引数が確定入力されているか確かめてから，先に定義してあった「関係/2」を質問し，もしそれが成立していれば「知っています」と答える．2番目のルールは，第2引数が変数で入力された場合に適用される．

```
既知のとき( X, Y ) :-
        var( Y ),
        関係( X, Y ).
```

「既知のとき」の2番目は，現在成立している関係を参照しながら，「○○はどういうものですか？」といった疑問文に答えるために使用されることを意図している．

5.4 例外のルール

また通常の既知と未知の両ケースで扱うことができない，例外的なケースでの反応を追加しておく．

```
おかしなこと( _, _ ) :-
        ¥+  関係( X, Y ),
        nl,
        write( ″はぁ？　何ですか。″ ).
```

非文法性，不自然さ，座りの悪さといった異なる程度の「おかしさ」を区別するため，ルールを修正することもできるが，紙幅の都合により省く．

5.5 XはY

以上の「既知のとき」「未知のとき」「おかしなこと」の3ルールを用いて，「XはY」や「XがY」の形の質問（ゴール）に対応するプログラムを作成する．本節では「XはY」の基本ルールを示す．

```
:- op( 600, yfx,  は  ).

X  は  Y:-
        既知のとき( X, Y ).
```

```
X は Y:-
        ¥+ var( X ),
        未知のとき( X, Y ),
        !.

X は Y:-
        おかしなこと( X, Y ),
        !,
        fail.
```

カット！は，必ず成功し，それを通過すると，以降失敗しても同じ頭部をバックトラックさせない．カットは論理的な完全性を破壊するが，うまく使うことで推論を効率化することができる．カットを経由するルールが成功すると，同名の述語を頭部に持つルールが試みられることはない．また前のルールで失敗判定されたという前提で後続の処理を行うことができる．既知の場合は，複数の関係をバックトラックすることを許す．おかしなことは，カット!と失敗failを組み合わせることによって，強制的に現在のルール頭部である「XはY」自体を失敗させる．

ちなみに，「は/2」を4節の「＝/2」の代わりに計算に用いるには，次のルールを先頭に置くとよい．

```
X は Y:-
        X ＝ Y,
        !.
```

以下に計算の実行例を示す．

```
(q.36) ?- 1+1 は 2 .
true .
```

5.6 Prolog に知識を教える

上述のプログラムを，”tisiki.pl”に追加し保存する（節の順序に注意すること）．これを読み込んだ後，「クジラはホニュウ動物」であるという知識をPrologに教える．

```
(q.37) ?- [tisiki].
true.
(q.38) ?- クジラ は ホニュウ動物.　（ENTERキー入力）
初耳です。
true.
```

二度目からは「知っています」と答えるようになる．

(q.39) ?- クジラ は ホニュウ動物. （ENTERキー入力）
知っています。
true.

5.7 「と」および「かつ」による一括学習

追加的に，クジラについての知識をいくつか教えてみよう.

(q.40) ?- クジラ は 海で泳ぐ，クジラ は 魚 ではない. （ENTERキー入力）
初耳です。
初耳です。
true.

同時に複数の関係を覚えさせたい場合，半角カンマで質問を区切って入力できる.

(q.41) ?- 犬 は ホニュウ動物，猫 は ホニュウ動物，ヒト は ホニュウ動物 だ.

上述の例のように，同じ述部が続く場合，助詞「と」を用いて結合できると便利であろう.

(q.42) ?- 犬 と 猫 と ヒト は ホニュウ動物 です.

そのためには，「XはY」の最初のルールとして次のプログラムを追加挿入するだけでよい.

```
X と Y は Z:-
        ¥+ var( Y ),
        ¥+ var( Z ),
        X は Z,
        Y は Z.
```

またできれば複数の述部をまとめて教えることができるとありがたいと思う読者もおられるだろう. 同じ頭部Xに対する複数の述部のY，Y’，Y’’などからなる「XはY」「XはY’」…など，を「かつ」でつなげて一括学習させる.

(q.43) ?- クジラ は ホニュウ動物，クジラ は 海で泳ぐ，クジラ は 魚 ではない.

とする代わりに，以下のように実行したい.

(q.44) ?- クジラ は ホニュウ動物 かつ 海で泳ぐ かつ 魚 ではない.

これは読者の練習問題とする.

5.8 学習内容を確認する

疑問代名詞の代わりに第2引数に変数を用いると，クジラがどのような動物か質問してみよう．

(q.45) ?- クジラ は _どのような動物 か.
_どのような動物か = ホニュウ動物.

別の代入組がまだあるとき，セミコロンを入力すると，バックトラックして別解が表示される．

(q.46) ?- クジラ は _どのような生き物 ですか.
_どのような生き物 = ホニュウ動物 ;
_どのような生き物 = 海で泳ぐ.

Prologの内部データベースでは，これらは述語の形式で事実節として登録されている（また「関係/2」のルールを経由して質問することもできる）．

(q.47) ?- クジラ(X). （ENTERキー入力）
X = ホニュウ動物 ;
X = 海で泳ぐ.

第1引数に変数を使うことはできない．非文法的な例外にあたり，「おかしなこと」として扱われる．

(q.48) ?- _何 は ホニュウ動物. （ENTERキー入力）
はぁ？ 何ですか。
false.

5.9 否定，矛盾，三段論法

もし文末否定表現「ではない/1」「ではありません/1」を，たんじゅんに文全体に組込述語 ¥+ を前置したものとして適用すると，「XはYでない」の形式では，既知のときが失敗し，意図せず肯定表現を学習してしまう．これらは文の述部を否定するものとする必要がある．

文末否定表現(ではない).
文末否定表現(ではありません).

:- op(200, yf, 'ではない').
:- op(200, yf, 'ではありません').

Z ではない:-
　　Z = (X は Y),
　　W = (Y ではない),

```
        Y ¥= X,
        X は W.

Z ではありません:-
        Z ではない.
```

本体の3番目の条件“X ¥ = Y”は,「ホニュウ動物 は ホニュウ動物 ではない」のような自己撞着する文を学習しないために必要である.

```
(q.49) ?- ホニュウ動物 は 魚 ではない .

初耳です。
true.

(q.50) ?- ホニュウ動物( A ).
A = (魚 ではない).
```

否定の獲得により, 矛盾する知識を検出でき, 学習を拒否するようにもできる.

```
(q.51) ?- クジラ は 魚 ですか.

ちがいます。
true.
```

以下のプログラムを「XはY」で使う.

```
矛盾するとき( X, Y, 否定表現以外 ) :-
        ¥+ var( X ),
        ¥+ var( Y ),
        文末否定表現( F ),
        ¥+ Y =.. [ F, _ ],
        関係( X, ( Y ではない ) ),
        nl,
        write( ″ちがいます。″ ).

矛盾するとき( X, Y, 否定表現のとき ) :-
        ¥+ var( X ),
        ¥+ var( Y ),
        文末否定表現( F ),
        Y =.. [ F, Z ],
```

```
        関係( X, Z ),
        nl,
        write( ″ちがいます。″ ).
```

矛盾を検出するには，「XはY」の中でどの位置がよいだろうか．「既知のとき」の中に置くと，失敗して次の「未知のとき」において矛盾した知識が登録されるという問題が生じる．例えば「クジラは魚でない」が既知のことではないので失敗すると，「クジラは魚」という肯定知識が新規に追加されてしまう．

教授法的な助言として，この直後に述べる解答例を読む前に，読者は練習問題としてしばらく考えてから，次のステップに進むようにすべきであろう．一つのシンプルな修正方法としては，「未知のとき」のルールの最初に，矛盾のケースを検出したら新規登録せずに成功するルールを追加挿入する．

```
未知のとき( X, Y ) :-
        矛盾するとき( X, Y, _ ),
        !.
```

ところで，一般に，「XはZ」かつ「ZがY」であるとき，「XはY」であると推論してよい．これは論理学では三段論法と呼ばれる．より抽象的には，二項関係「は」の推移性である（ただし1ステップ推論のみ）．

例えば，「クジラは魚ではない」ことは，読者のよく知っていることだと思う．「クジラはホニュウ動物」であり，また「ホニュウ動物は魚ではない」ことを知っているからである．三段論法は「既知のとき/1」のルールに，以下の条件のパタンを追加することで実現できる（読者の練習とする）．

```
推論できること( X, Y ) :-
        ¥+ var( Y ),
        関係( X, Z ),
        関係( Z, Y ).
```

```
(q.52) ?- ホニュウ動物 は 魚 ではない .
知っています。
true .
```

```
(q.53) ?- クジラ は 魚 ではない .
そう思います。
true .
```

5.10 最初から教え直す

知識を教え直したければ，abolish/1で廃止し，対応する述語名についてもretract/1で削除する．

```
(q.54) ?- abolish( クジラ/1 ), retract( 述語名( クジラ ) ).
true.
```

最初からすべて教え直すには，以下の述語を追加して用いるとよい．

```
教え直す( X ):-
        述語名( X ), abolish( X / 1 ) , retract(  述語名( X ) ).

はじめから教え直す:-  教え直す ( _ ), fail ; true.
はじめから  :-  はじめから教え直す.
```

5.11 フィルターとしての主題化

「XはY」という形の文は，「XがY」における頭部のXが主題化されたものであると考えることができる．しかし「XはY」は「YがX」における述部を主題化したものである場合もある．

5.5節のプログラムを用いると，次に示す例のように不自然な質問も成功してしまう．

```
(q.55) ?-  ホニュウ動物  は  クジラ.
初耳です。
true.
```

たしかに，この質問の前後でPrologの質問応答パタンが変化するから何がしかの情報は得たことになる（この観点での情報の定義は9.1節で詳しく論じられる）．しかし，上述の文はクジラだけがホニュウ動物であるかのように解釈できるため，ホニュウ動物として，クジラ以外に，犬や猫やヒトもいるという常識的な生き物の分類にかんする知識に反する．またそれは「クジラがホニュウ動物」という文が，クジラだけがホニュウ動物であるかのように解釈される（総記）のと同じ内容になる．これは助詞「は」によって文中の成分が，主題化されることと関係している．「クジラはホニュウ動物である」という文から，関係（X，Y）＝（クジラ，ホニュウ動物）が学習されるとして， Yの部分を主題化して質問したのが，「ホニュウ動物はX」であり，その答えがX＝クジラである．「XはY」におけるXを述語とみなしたのは，偶然ではない．述語というものは，そもそもその引数について質問する，そしてその答えとして単一化済みの代入を得るために使うものである．

「XはY」の主たる認知機能は情報のフィルターであると考えられる．標準的なデータベースでは，一般にタプルをn項関係と考え，また情報を検索するためには列名（フィールド）を検索のキーとして指定する必要がある．

次のような簡単なテーブルを考えよう．

ID	名前	役職	上司
s0001	安藤	部長	加藤
s0002	加藤	社長	—
S0003	佐藤	課長	安藤

上のテーブルをTとする．テーブルTは3個のタプルからなる．

T＝{（s0001，安藤，部長，加藤），（s0002，加藤，社長，—），（s0003，佐藤，課長，安藤）}．

ここで「部長は誰か」と質問すると，多くの読者は直ちに「安藤」と答えるだろう．データ数が多いときは，答えを見出すため，表計算ソフトにデータを読み込んで，見出し行にオートフィルターをつけて，「役職」の列をプルダウンし，「部長」を選ぶかもしれない．3節と4節の方法に倣い，以下の疑似コードを完成させることによって，実際に外部のデータベース管理システムにSQLコマンドを発行することもできるかもしれない（読者の練習とする）．

```
_テーブル名 から 役職 が 部長 である行を 抽出 :-
        _SQL文 = ″SELECT ＊ FROM テーブル名 WHERE 役職 ＝ 部長″,
        外部データベースにSQLを発行する( _SQL文 ).
```

不完全ながらデータベースへの日本語フロントエンドが利用可能になった．クジラにかんする知識を一括して再現してみよう．

```
(q.56) ?- はじめから，クジラ は ホニュウ動物，クジラ は 海で泳ぐ，クジラ は 魚 ではない.
(ENTER)
```

次に findall/3 を用いて，関係/2の第1引数をクジラとして，その第2引数を集めることによって，Prologが獲得したクジラにかんする知識を一括してリストとする．

```
(q.57) ?- X = クジラ, findall( Y, 関係(X, Y ), _クジラにかんする知識 ).
_クジラにかんする知識 = [ホニュウ動物，海で泳ぐ，魚 ではない].
```

上述の実行結果は，クジラについての知識が，（クジラ，ホニュウ動物，海で泳ぐ，魚ではない）というタプルないし4項関係として情報表現されることを意味している．これをルールとして再表現すると，

```
クジラ:-
        ホニュウ動物,
        海で泳ぐ,
        魚ではない.
```

と書くことができる．しかしながら，このようにルール化すると，クジラについて質問することはできるが，本体の条件にされた「ホニュウ動物」，「海で泳ぐ」，「魚ではない」のそれぞれとクジラとの間の元の2項関係については質問できない．すなわち，主題化とは元の関係（タプル）のある要素をキーとして選択された代替的関係を集めて，そのキーを頭部に据えたルールを作出する認知的操作に相当する．この操作は表計算ソフトウェアにおけるフィルター機能やピボットテーブル機能における行の抽出・集計，あるいは関係データベースにおけるビューテーブルの作成に似ている．

またホニュウ動物にかんする知識をリストに集めるため，再び findall/3 を用いる．今度は関係/2の第2引数をホニュウ動物として，第1引数を収集する．

```
(q.58) ?- Y = ホニュウ動物, findall( X, 関係( X, Y), _ホニュウ動物にかんする知識 ).
```

_ホニュウ動物にかんする知識 = [クジラ].

現在，ホニュウ動物の真の事例はクジラのみであることが確かめられた．前例と同様に，この抽出結果をルールとして再表現すると次のようである．

ホニュウ動物:-
　　クジラ.

犬や猫やヒトもホニュウ動物であることを追加してみよう．

(q.59) ?- 犬 は ホニュウ動物，猫 は ホニュウ動物，ヒト は ホニュウ動物.

(q.60) ?- Y = ホニュウ動物, findall(X，関係(X, Y), _ホニュウ動物にかんする知識).
Y = ホニュウ動物,
_ホニュウ動物にかんする知識 = [クジラ，犬，猫，ヒト].

ところで，2項関係の情報モデルには(外部)キーが不在という明らかな欠陥がある．データベース検索の比喩では，2項関係(X, Y)におけるXは，テーブル名とフィールド名とその抽出条件がセットになっていないといけない．つまり主題とは情報検索のためのSQLコマンドである．またこれは先ほどの表を使った直観的なデータ検索のイメージに合致する(もちろん主題は「部長」である．その他の部分は文脈から既定値として設定される)．

タプル表現は，Prologの中ではn引数述語として表現される，本論文のアプローチの採用する情報モデルの内部表現のより一般的な形式である．実際，主題化は「XがYのZをWに○○した」とか「XをYのZからWのUへ○○する」といった助詞で標識される複数の格成分について，それぞれ「は」で主題化することができる(三上，1963)．しかし「は」で主題化して，2項関係に再表現された後は，元のタプルを登録する際に使われた助詞の記録は失われる(主題化，つまり主題の有標化の代償として，三上の用語で言うところの「代行」されている格が無標化する)．

そこで2項関係(X, Y)は，情報空間において内部表現された情報，すなわちタプルの中から，主題Xを検索のキーとして，その値となるY(より正確には値Yを含む行)を抽出して得られる結果と解釈することにする．タプル自体が2項であれば，第2項は厳密にいえば抽出された行，タプル(X, Y)自体である．[25]またタプル内の順序に意味がないとすると，(X, (X, Y))または(X, (Y,X))と考えなければならない．ちなみに，SWI-Prologでのタプルは，ユニブによって (X, Y) =.. [',', X, Y] と単一化される．つまり，(X, Y) = ','(X, Y)，(X, Y, Z) = (X, (Y, Z)) = ','(X, ','(Y, Z)) などとなる．順序を考えないタプルないしリスト形式は，dictと呼ばれることがある．

5.12「の」

またルール節はタプルの系列として表現可能である．[26] 事実，「の」という助詞があって，これは無標化された2項関係，つまりタプルの中の任意のペアをとりだして得られる2項関係を作り出せるものであり，我々日本人にとって重宝するものである．

```
:- op( 500, xfy, の ).

(q.61) ?- nl, write_canonical(( a, b, c )), nl, write_canonical(( o の a の b の c )).

',',(a,',',(b,c))
の(o,の(a,の(b,c)))
true.
```

「XのY」の(X, Y)は, 体言のペアか, あるいはXが用言でありかつYが空であり, いずれも全体として体言となる.

また5節で述べた, 「と」や「かつ」による2項関係についても, n項関係, つまりタプルに拡張可能である.

```
(q.62) ?- nl, write_canonical(( a, b, c )), nl, write_canonical(( o と a と b と c )).

',',(a,',',(b,c))
と(と(と(o,a),b),c)
true.

(q.63) ?- nl, write_canonical(( o :- a, b, ¥+ c )), nl, write_canonical(( o は a かつ b かつ c ではない )).

:-(o,',',(a,',',(b,¥+(c))))
は(o,かつ(a,かつ(b,ではない(c))))
true.
```

5.13 本節のまとめ

本節では助詞をファンクターとしてゴールを項に区切り, 疑似的な日本語文を頭部とするルールとして記述する便宜的な日本語論理プログラミングをデモンストレーションした. 助詞をファンクターとみなす方法は, 1節で述べた2基準を部分的に満たす. つまりシンプルであり, かつ一貫したやり方で, ほぼ機械的に実行可能である. また(元の)自然言語の語句の一部分を用いている. この方法の問題点は, 例えば, ファンクターの優先順位と型の割当て方の恣意性, 表現上の空白区切りの不自然さ, 文法的な品詞カテゴリーの無視, そして否定表現である.

本節のアプローチでは, 既知と未知の区別を計算的に明確化した. また本節で提示されたプログラムをベースに, 9節では助詞「は」と「が」の言い回しによって生じる微妙な意味的変化の論理プログラムを作成する. 否定は代替的関係の阻止に関連しているので, その認知的側面についても9節で触れる.

次節では, ファンクターを用いず, 文書から直接節プログラムを抽出する別の方法を論じる.

6 文章からのルール抽出:相続税の計算

前節で導入した助詞をファンクターとして日本語文を述語化する方法は, ほぼ既存のProlog処理系だけ

で日本語論理プログラミングが可能になり，また手続付加によってProlog以外のプログラムの処理を立ち上げ，外部のサービスにアクセスすることも可能である．このため応用としてはさまざまなPC作業を自然言語で指示し自動化する，あるいは逆に作業結果を自然言語に表現しなおすといったことも十分に可能である．欠点としては，いちいち助詞で分かち書きしなければならず，また構文の柔軟性や意味的な処理についても限定されていることである．意味的な処理の問題は9節で再び論じることとして，ここでは日本語文章から直接論理プログラムを抽出する方法を考案する．

空白区切りの不自然さについては，ほぼ自明な解決法がある．成功した日本語の質問を，区切りの空白を削って，述語名として登録し，それを頭部としたルール本体に成功した質問自体を付加すればよい．例えば，

(q.64) ?- Prolog を 使うの は 初めて です.

という質問がもし成功したとすると，次のようなルールが生成されるようにすればよい．

Prologを使うのは初めてです:-
　　Prolog を 使うの は 初めて です.

ただし，大文字で始まる文字列は変数名とみなされるので，述語名を引用符で囲む．また元の区切られた項をこの述語の引数とするべきかもしれない．

'Prologを使うのは初めてです'(Prolog，を，使うの，は，初めて，です):-
　　Prolog を 使うの は 初めて です.

本節では，このような処置による前節の方法の改善念頭に置きつつ，それとは独立に，つまり助詞をファンクターとみなすアドホックな解釈をせずに，(それもまた別のアドホックな代替案かもしれないが)直接的に，日本語の文章から論理プログラムを抽出する具体的な試みをいくつか紹介する．

例として，「法定相続人の数は、相続の放棄をした人がいても、その放棄がなかったものとした場合の相続人の数をいいます。」(国税庁ウェブページ「No.4152 相続税の計算」より)という日本語表現をとりあげよう．

上述の文章に対して，Prologを少し学習したことがある読者であれば，

法定相続人の数(X):-
　　相続の放棄をした人がいても、その放棄がなかったものとした場合の相続人の数(X).

のようなプログラムを書くかもしれない．この書き方はシンプルであり，それゆえ基準 σ を部分的に満たすが，しかし元の文章から一部分だけを抜き出しており，その抽出基準も不明であるため，基準 τ を満たしていない．本論文で提案されるアプローチでは，日本語のテキストデータをPrologのプログラム(確定節として読める表現)に翻訳し，Prolog内部で実行可能にする．仮に文書から確定節への具体的な変換方式が，一つのPrologアプリケーションとして書けたとすると，変換プログラム自体の複雑性という点を度外視することが許されるならば，人間の判断が介入せず，機械的に実行できるシンプルな方法であり，よって基準

σを満たすだろう.

もう一つの基準τと合わせて2基準を同時に満たす方法としては，文書全体を一つのルールとして，各行（各文）を本体の条件として順次並べることが最も簡単である.

（手続きS）元の文章をそのままルールの頭部として，述語名および対応する変数を引数として持たせる. その本体を元の文章を分解した語句を並べて，ルール節とする.

ただし手続きSは一見シンプルすぎて実用に供しそうにないが，特殊な様式で書かれた文書，例えば節見出しと節に振られた番号などがあれば，これらを標識（目印）として，実際に文書のアウトラインを自動抽出することができる.

アウトラインの標識が利用できない文書の場合，基準τを満たす代替的な方法としては，各文をルールとして独立させ，その文自体を頭部として，文を文断片に分解し，「法定相続人の数」「は、」「相続の放棄をした人がいても、その放棄がなかったものとした場合の相続人の数」「をいいます。」の各部分に分け，本体の条件部分として採用する方法が考えられる. すなわち，以下のようなルールを得る.

’法定相続人の数は、相続の放棄をした人がいても、その放棄がなかったものとした場合の相続人の数をいいます’ :-

’法定相続人の数’,

’は’,

’相続の放棄をした人がいても、その放棄がなかったものとした場合の相続人の数’,

’をいいます’.

ここで本体の1番目と3番目の条件に引数を追加して，相続人数を計算できるようにすることができる.

’法定相続人の数は、相続の放棄をした人がいても、その放棄がなかったものとした場合の相続人の数をいいます。’ (_法定相続人の数は、相続の放棄をした人がいても、その放棄がなかったものとした場合の相続人の数をいいます。) :-

法定相続人の数(_法定相続人の数),

は,

相続の放棄をした人がいても、その放棄がなかったものとした場合の相続人の数」(_相続の放棄をした人がいても、その放棄がなかったものとした場合の相続人の数),

をいいます.

また「は、」と「をいいます。」は事実として追加する. このような文の分割処理を，文書全体に適用できるよう拡張したものを，手続きSSとして改めて述べてみよう.

（手続きSS）「文章を句点で区切り，文単位に分けした後，ルール頭部の述語として直接用い，語句に分解してルール本体の条件とし，さらに条件に用いた語句を分解して下位ルールを作り，これを単語や意味を持たない文書断片，もしくは何らかのデータ・PC操作（副作用）を伴う指令内容に至るまで，反復的（再帰的）に分解する. 」

手続SSを自動化するためには，単語DBを用いて単語分解を試みることが考えられる．ただし現時点でこの方法はこの後述べる数式抽出のような特殊な場合に成功例が得られる．単語DBとしてWordnet（日本語と英語）を用いて単語抽出した残りの文章断片が，文全体の骨組みをなす疑似的なファンクターとみなせる．プログラムの詳細は省略するが，以下にその実行結果の例を示す．

(q.65) ?- D = '課税価格をどのように計算しますか', '語リスト分解'(D, Y, W).

D = 課税価格をどのように計算しますか,
Y = [課税，価格，計算],
W = [***, ***, をどのように, ***, しますか] .

また，実際の文書中の数式行では，骨組みの部分は算術演算や括弧を含んでいる．逆に，算術演算や括弧を，半角スペース囲みのProlog算術演算子と括弧に置き換えて整形すると，残りの部分を変数名とした，Prologの数式行が得られる．

7 アウトラインの抽出

本節では日本語で書かれた文書から，その見出し行に注目することでアウトラインをルール節として抽出する．サンプル文書として，国税庁ウェブページに掲載されている「No.4152 相続税の計算」を用いる．デモンストレーション用の述語midashi/0におけるおおよその処理の流れは次のようである．まずサンプル文書全体を文字列データとして入力する．次に文書を行に分割し，行頭におかれた記号や数値の特徴から，行番号付きで見出し行を抽出する．以下はその実行結果である．

(q.66) ?- midashi.
[1]No.4152 相続税の計算。
[4]1 各人の課税価格の計算。
[11]2 相続税の総額の計算。
[28]3 各人ごとの相続税額の計算。
[31]4 各人の納付税額の計算。
true.

ただし文書末尾の括弧書きの「相法」と「参考：関連コード」以下の部分は省略した．抽出された見出し行は，この文書の最も高いレベルのアウトラインを構成する．まず第1行「No.4152 相続税の計算」は，文書全体を一つの述語とみなしたときの述語名であり，次のようなルール節として再表現される．

'No.4152 相続税の計算':-
'1 各人の課税価格の計算',
'2 相続税の総額の計算',
'3 各人ごとの相続税額の計算',

'4 各人の納付税額の計算'.

またこのルールを用いて実際に相続税の計算が可能になるようにするため，ルール「相続税の計算」内の述語引数に，累積的に変数を付与する.

'No.4152 相続税の計算'(X1, X2, X3, X4):-
'1 各人の課税価格の計算'(X1),
'2 相続税の総額の計算'(X1, X2),
'3 各人ごとの相続税額の計算'(X1, X2, X3),
'4 各人の納付税額の計算'(X1, X2, X3, X4).

ちなみに，Prologは，導出の過程で先行する述語で仮の単一化をした変数への代入を，後続の同名変数に仮代入する. すべての同名変数が単一化代入されると，述語自体が成功するが，それまでは先行する変数の代入結果は仮のままであり，バックトラックが生じると却下される（興味深いことにGale & Shapley(1962)の遅延受諾アルゴリズムに類似している）.

次にアウトラインの各項目，すなわち4行目，11行目，28行目，31行目の各見出し行を，それぞれ下位の述語とみなし，次の見出しの直前までの行をその本体とするルール節として再構成する. 例えば，全体の見出し「No.4152 相続税の計算」から第1節の見出し「1 各人の課税価格の計算」までの間には，次の2行が挿入されている.

[平成31年4月1日現在法令等]
相続税の一般的な計算は、次の順序で行います。

全体見出しに続く2行を，次の見出しに相当する節本体の条件の直前に，挿入することによって，確定節を修正すればよい. 修正されたプログラムは以下のようである.

'No.4152 相続税の計算'(X1, X2, X3, X4):-
'[平成31年4月1日現在法令等]',
'相続税の一般的な計算は、次の順序で行います',
'1 各人の課税価格の計算'(X1),
'2 相続税の総額の計算'(X1, X2),
'3 各人ごとの相続税額の計算'(X1, X2, X3),
'4 各人の納付税額の計算'(X1, X2, X3, X4).

挿入された2行は，全体見出しの趣旨説明であり，後続部分の開始の宣言である. それは後続の部分での計算や論理に実質的に影響していると思われない. しかしながら，元の文書の部分を損なわいという原則から，省略せず，暫定的に事実節として扱うものとする. また，この暫定措置をとることで，何らかの実質的な影響が判明した時点で，修正し，事実節からルール節に変えることができる.

次の事実節2行をプログラムに追加する.

’[平成31年4月1日現在法令等]’.
’相続税の一般的な計算は、次の順序で行います’.

全体見出し行と同様に，以降の見出し行に対応する確定節を追加することができることは明らかだろう．1節のプログラムを示しておこう（ただし挿入される事実節への参照が長い文字列になるが）．

’1 各人の課税価格の計算’(X1):-
’まず、相続や遺贈及び相続時精算課税の適用を受ける贈与によって財産を取得した人ごとに、課税価格を次のように計算します’,
’相続または遺贈により取得した財産の総額 ＋ みなし相続などにより取得した財産の価額 － 非課税財産の価額 ＋ 相続時精算課税に係る贈与財産の価額（注1） － 債務及び葬式費用の額 ＝ 純資産価額（赤字のときは0）’,
’純資産価額 ＋ 相続開始前3年以内の贈与財産の価額（注2） ＝ 各人の課税価格（千円未満切捨て）’,
’（注）’,
’1*1 相続時精算課税の特定贈与者・・・中略・・・贈与の時の価額で相続税の課税価格に算入されることになります’,
’2*2（以下省略）’.

最後に，文章からアウトラインを自動抽出するPrologプログラムを作って実行してみると，トップレベルでのアウトラインとして以下の結果を得る．

```
% No.4152　相続税の計算

sozoku_rule( 1,’No.4152 相続税の計算’ ) :-
        sozoku_rule( 2,’[平成31年4月1日現在法令等]’ ),
        sozoku_rule( 3,’相続税の一般的な計算は、次の順序で行います。’ ),
        sozoku_rule( 4,’1 各人の課税価格の計算’ ),
        sozoku_rule( 11,’2 相続税の総額の計算’ ),
        sozoku_rule( 28,’3 各人ごとの相続税額の計算’ ),
        sozoku_rule( 31,’4 各人の納付税額の計算’ ),
        sozoku_rule( 42,’参考： 関連コード’ ),
        sozoku_rule( 41,’（相法11～20の2、21の9～16、33の2、措法70の7の13、相基通16-1～16-3、19-11、20の2-4）’ ).
```

より下位レベルのアウトラインを展開することもできる．

```
% 1　各人の課税価格の計算
```

sozoku_rule(4,'1 各人の課税価格の計算') :-

sozoku_rule(5,'まず、相続や遺贈及び相続時精算課税の適用を受ける贈与によって財産を取得した人ごとに、課税価格を次のように計算します。'),

sozoku_rule(6,'相続または遺贈により取得した財産の総額 ＋ みなし相続などにより取得した財産の価額 － 非課税財産の価額 ＋ 相続時精算課税に係る贈与財産の価額(注1) － 債務及び葬式費用の額 ＝ 純資産価額(赤字のときは0)'),

sozoku_rule(7,'純資産価額 ＋ 相続開始前3年以内の贈与財産の価額(注2) ＝ 各人の課税価格(千円未満切捨て)'),

sozoku_rule(8,'(注)'),

sozoku_rule(9,'1*1 相続時精算課税の特定贈与者･･･中略･･･贈与の時の価額で相続税の課税価格に算入されることになります。'),

sozoku_rule(10,'2*2 相続又は遺贈により財産を取得した相続人等が、相続開始前3年以内にその被相続人からの暦年課税に係る贈与によって取得した財産の価額をいいます。 ').

8 数式の抽出

本節では前節のサンプル文書内から数式表現を自動抽出する. ただしサンプル文書中の図で示された数式については筆者が事前に手作業で文字列に変換してテキストデータに追加してある. プログラムの詳細は付録Cとし, 本節ではその基本的な考え方と主要な結果のみを紹介する.

まず, 文章としての数式表現から, Prologの算術式を含むルールへ翻訳する方法を説明するため, 具体例として, 再び前節のサンプル文書から, 課税遺産総額の計算を説明する箇所を引用する.

「課税価格の合計額から基礎控除額を差し引いて、課税される遺産の総額を計算します。
課税価格の合計額 － 基礎控除額(3,000万円 ＋ 600万円 × 法定相続人の数)＝ 課税遺産総額」

この文章に対応するルールは,「課税遺産総額」に対応する変数(″_課税遺産総額″)を引数として追加して, 次のように書けるだろう.

課税価格の合計額から基礎控除額を差し引いて、課税される遺産の総額を計算します(_課税遺産総額):-

課税価格の合計額 － 基礎控除額(3,000万円 ＋ 600万円 × 法定相続人の数)＝ 課税遺産総額(_課税遺産総額).

また本体の各条件を下位ルールとして分解し, 最後の除算によって課税遺産総額を求める.

課税価格の合計額 － 基礎控除額(3,000万円 ＋ 600万円 × 法定相続人の数) ＝ 課税遺産総額(_課税遺産総額):-

課税価格の合計額(_課税価格の合計額),

基礎控除額(3,000万円 ＋ 600万円 × 法定相続人の数)(_基礎控除額),

_課税遺産総額 =:= _課税価格の合計額 - _基礎控除額.

さらに基礎控除額の部分は，カッコ内にそれを計算する数式表現を含んでいる．これは以下のような独立したルールとして抽出されるべきである．

```
基礎控除額( _基礎控除額 ):-
        3000万円( _3000万円 ),
        600万円( _600万円 ),
        法定相続人の数( _法定相続人の数 ),
        _3000万円 ＋ _600万円 × _法定相続人の数 =:= _基礎控除額.
```

上記2つのルールの末尾で用いられている =:= は両辺の数式を評価して単一化する．通常用いられる is /2 は右辺の計算結果を左辺の変数に代入する．なお，Prologは，基本的に変数に対して破壊的代入をしないので，is/2は例外である．ちなみに，Prologで = 算術演算子ではなく，両辺の項が単一化（ユニフィケーション）できる代入であることを意味する．

上述の変換処理は，Prologプログラムとして書くことができ，それによって元の日本語文書の数式行からの数式を自動抽出することができる（プログラムは付録Cに記載した）．おおよその処理の流れは次のようである．まずサンプル文書全体を文字列データとして入力する．次に文書を行に分割し，内の数式行を等号に注目して抽出する．算術演算子と括弧に着目して単語を分割し組込の算術演算子と半角括弧に置き換える．そのためSWI-Prologの組込述語split_string/4などを利用し，文章中の括弧付き数式表現を文字列置換のプログラムを作成，最終的にterm_string/2で項に変換する．その際，通貨単位（万円）を4節の方法でファンクターとして扱っている（詳細は付録Cを参照）．

付録Cに掲載した「サンプル文書内の数式行を項として再表現する/2」を用いて，数式を抽出すると，以下のように，6, 7, 14, 23, 25, 27, 30, 36, 37, 40の各行，計11行が抽出される．

```
(q.67) ?- サンプル文書内の数式行を項として再表現する( J, X, Y ), nl, write( J: Y ), fail.

6:(相続または遺贈により取得した財産の総額+みなし相続などにより取得した財産の価額-非課税財産の価額+相続時精算課税に係る贈与財産の価額(注1)-債務及び葬式費用の額=:=純資産価額(赤字のときは0))
7:(純資産価額+相続開始前3年以内の贈与財産の価額(注2)=:=各人の課税価格(千円未満切捨て))
14:(各相続人の課税価格の合計=:=課税価格の合計額)
16:(課税価格の合計額-基礎控除額(3000 万円+600 万円*法定相続人の数)=:=課税遺産総額)
23:(課税遺産総額*各法定相続人の法定相続分=:=法定相続分に応ずる各法定相続人の取得金額(千円未満切り捨て))
25:(法定相続分に応ずる各法定相続人の取得金額*税率=:=算出税額)
27:(各法定相続人ごとの算出税額の合計=:=相続税の総額)
30:(相続税の総額*各人の課税価格/課税価格の合計額=:=各相続人等の税額)
36:(各相続人等の税額+相続税額の2割加算-暦年課税分の贈与税額控除-配偶者の税額軽減-未成年者控除-障害者控除-相次相続控除-外国税額控除=:=各相続人の控除後の税額(赤字の場合は0になります))
```

37:(各相続等の控除後の税額-相続時精算課税分の贈与税相当額(外国税額控除前の税額)-医療法人持分税額控除=:=各相続人等の納付すべき税額(※))
40:(赤字となった金額(マイナスは付けません)-相続時精算課税分の贈与税の計算をする際、控除した外国税額=:=還付を受けることができる金額)
false.

こうして得られた疑似数式は，すでにPrologの項として表現済みであるので，後は各項目を変数化すれば実際に計算できるようになる．ただし，カッコ内に注釈や数式が複数回現れることもある．そこで，まず括弧つきの部分を，先行語句をその述語名（ファンクター）とみなして抽出する（プログラムの詳細は付録C参照）．

(q.68) ?- 数式内の注釈やファンクターをルール化する(J, _ルール, _変数), nl, write([J]: _ルール: _変数), fail.

[6]:(相続時精算課税に係る贈与財産の価額(_相続時精算課税に係る贈与財産の価額):-_注1):[_注1]
[6]:(純資産価額(_純資産価額):-_赤字のときは0):[_赤字のときは0]
[7]:(相続開始前3年以内の贈与財産の価額(_相続開始前3年以内の贈与財産の価額):-_注2):[_注2]
[7]:(各人の課税価格(_各人の課税価格):-_千円未満切捨て):[_千円未満切捨て]
[16]:(基礎控除額(_基礎控除額):-_3000 万円+_600 万円*_法定相続人の数):[_3000 万円,_600 万円,_法定相続人の数]
[23]:(法定相続分に応ずる各法定相続人の取得金額(_法定相続分に応ずる各法定相続人の取得金額):-_千円未満切り捨て):[_千円未満切り捨て]
[36]:(各相続人の控除後の税額(_各相続人の控除後の税額):-_赤字の場合は0になります):[_赤字の場合は0になります]
[37]:(相続時精算課税分の贈与税相当額(_相続時精算課税分の贈与税相当額):-_外国税額控除前の税額):[_外国税額控除前の税額]
[37]:(各相続人等の納付すべき税額(_各相続人等の納付すべき税額):-_※):[_※]
[40]:(赤字となった金額(_赤字となった金額):-_マイナスは付けません):[_マイナスは付けません]
false.

またファンクターの部分は同名の変数に置き換えた上で，同名述語をカッコ内の数式（あるいは注釈）を手続き付加した別ルールとして独立に定義し，元の数式行のルールの本体の条件とするなどの処理が必要である（詳細は省く）．

数式内の変数の参照関係を分析する

ここでは，元の文書内での変数出現状況を調べて，0－1値のリストとして抽出するプログラムの実行結果を示す（ソースコードは省略する）．

[0,0,0,0,0,0,0,0,0,0,1];_マイナスは付けません
[0,0,0,0,0,0,0,0,0,0,1];_相続時精算課税分の贈与税の計算をする際、控除した外国税額

[0,0,0,0,0,0,0,0,0,0,1];_赤字となった金額
[0,0,0,0,0,0,0,0,0,0,1];_還付を受けることができる金額
[0,0,0,0,0,0,0,0,0,1,0];_※
[0,0,0,0,0,0,0,0,0,1,0];_医療法人持分税額控除
[0,0,0,0,0,0,0,0,0,1,0];_各相続人等の納付すべき税額
[0,0,0,0,0,0,0,0,0,1,0];_各相続等の控除後の税額
[0,0,0,0,0,0,0,0,0,1,0];_外国税額控除前の税額
[0,0,0,0,0,0,0,0,0,1,0];_相続時精算課税分の贈与税相当額
[0,0,0,0,0,0,0,0,1,0,0];_各相続人の控除後の税額
[0,0,0,0,0,0,0,0,1,0,0];_外国税額控除
[0,0,0,0,0,0,0,0,1,0,0];_暦年課税分の贈与税額控除
[0,0,0,0,0,0,0,0,1,0,0];_未成年者控除
[0,0,0,0,0,0,0,0,1,0,0];_相次相続控除
[0,0,0,0,0,0,0,0,1,0,0];_相続税額の2割加算
[0,0,0,0,0,0,0,0,1,0,0];_赤字の場合は0になります
[0,0,0,0,0,0,0,0,1,0,0];_配偶者の税額軽減
[0,0,0,0,0,0,0,0,1,0,0];_障害者控除
[0,0,0,0,0,0,0,1,1,0,0];_各相続人等の税額
[0,0,0,0,0,0,1,0,0,0,0];_各法定相続人ごとの算出税額の合計
[0,0,0,0,0,0,1,1,0,0,0];_相続税の総額
[0,0,0,0,0,1,0,0,0,0,0];_税率
[0,0,0,0,0,1,0,0,0,0,0];_算出税額
[0,0,0,0,1,0,0,0,0,0,0];_千円未満切り捨て
[0,0,0,0,1,0,0,0,0,0,0];_各法定相続人の法定相続分
[0,0,0,0,1,1,0,0,0,0,0];_法定相続分に応ずる各法定相続人の取得金額
[0,0,0,1,0,0,0,0,0,0,0];_3000万円
[0,0,0,1,0,0,0,0,0,0,0];_600万円
[0,0,0,1,0,0,0,0,0,0,0];_基礎控除額
[0,0,0,1,0,0,0,0,0,0,0];_法定相続人の数
[0,0,0,1,1,0,0,0,0,0,0];_課税遺産総額
[0,0,1,0,0,0,0,0,0,0,0];_各相続人の課税価格の合計
[0,0,1,1,0,0,0,1,0,0,0];_課税価格の合計額
[0,1,0,0,0,0,0,0,0,0,0];_千円未満切捨て
[0,1,0,0,0,0,0,0,0,0,0];_注2
[0,1,0,0,0,0,0,0,0,0,0];_相続開始前3年以内の贈与財産の価額
[0,1,0,0,0,0,0,1,0,0,0];_各人の課税価格
[1,0,0,0,0,0,0,0,0,0,0];_みなし相続などにより取得した財産の価額
[1,0,0,0,0,0,0,0,0,0,0];_債務及び葬式費用の額
[1,0,0,0,0,0,0,0,0,0,0];_注1
[1,0,0,0,0,0,0,0,0,0,0];_相続または遺贈により取得した財産の総額

[1,0,0,0,0,0,0,0,0,0,0];_相続時精算課税に係る贈与財産の価額
[1,0,0,0,0,0,0,0,0,0,0];_赤字のときは0
[1,0,0,0,0,0,0,0,0,0,0];_非課税財産の価額
[1,1,0,0,0,0,0,0,0,0,0];_純資産価額

各行の右端は数式内の変数に対応する項目名，各行のリスト要素は数式行の出現順位ごとに，1がその語が出現する数式行，0は出現なしを意味する．この行列表現によって，数式間の参照関係を知ることができる．

参照関係を視覚化する

前出の0－1表現から，各行の最初の1の桁で他の桁の1があるものが，その項目を定義するルールの頭部に対応し，後続する1の桁が，同項目を条件中で参照している後続ルールであると推定される．この推測を自動化して，参照関係を抽出し視覚化してみると以下のようになる．

```
path((10->0))=[10,0]
10 :_※

path((11->0))=[11,0]
11 :_マイナスは付けません

path((10->0))=[10,0]
10 :_医療法人持分税額控除

path((1->0))=[1,2,8,9,0]
1 :_純資産価額
      |
      -->2 :_各人の課税価格
            |
            -->8 :_各相続人等の税額
                  |
                  -->9 :_各相続人の控除後の税額
…

path((7->0))=[7,8,9,0]
7 :_相続税の総額
      |
      -->8 :_各相続人等の税額
            |
            -->9 :_障害者控除
```

```
path((3->0))=[3,8,9,0]
3 :_課税価格の合計額
      |
      -->8 :_各相続人等の税額
            |
            -->9 :_障害者控除
```

9　総記と対比，情報の新旧

これまでの節では，日本語でPrologプログラミングを行うための実践的アプローチを中心に述べてきた．本節と次節では，自然言語と論理プログラミングの関係をより広い文脈に埋め込んで考察する．本節では認知科学とその関連分野にまたがるいくつかの問題との関連を述べる．

5節のプログラムでは，「XはYか」の形の質問に対する意味的な処理のちがいを既知と未知のケースで具体的に区別した．[27]本節では，主題と対比(久野，1973)，中立叙述と総記(Kuroda，1965；久野，1973)の区別，情報機能（新しい情報と旧い情報）(久野，1973；井上，1989)といった観点を，日本語論理プログラミングとしてどう扱うかが焦点になる．本節では，5節で定義した関係に基づく情報モデルと同節で提案された「XはY」の形の文の論理プログラムを基礎として，総記や対比を語の代替的関係のネットワークの安定性[28]の条件と捉えて，「XがY」と「XはY」の論理プログラムを修正し，その意味合いを反映する． 総記や対比はその意味が成立する関係の集合を調節するはたらきであり，またそれはPrologのしくみ自体にも共通する．これはAIにおけるサーカムスクリプション(McCarthy，1980)との関連が考察される．

9.1 旧い情報？

言語学では，日本語の文には，「XがY」の形の叙述文（つまりXとYが主語-述語(Subject-Predicate)の関係である場合）と，「XはY」の形の判断文（XとYが主題-評言(Theme-Rheme)の関係，ないし話題-説述(Topic-Comment)の関係である場合）があると説明されてきた（柴谷・影山・田守，1982）．例は「雪が降ってきた」は叙述文の例で，「雪は降ってきた」は判断文であると言う．

日本語の場合，「は」が主題を標識するので，主題としては無標である「XがY」と区別できる．三上(1963)は「は」による格成分の主題化を（格の）代行と呼び，また黒田(1965)は「は」の添加による話題化を変形規則として提案した．[29]英語の機能文法では，主題を副詞や補語を文頭部に置いて構文上のちがいによって標識する有標の主題(marked theme)と，代名詞や固有名詞や名詞化された節を文頭部に主語として置く無標の主題(unmarked theme)が区別される(Halliday & Matthiessen，2014，Table 3-1)．つまり，無標（叙述文，あるいは久野(1973)の言う中立叙述）だから主題が欠落しているとは言えないことに注意しなければならない．

また柴谷ら(1982)は，「は」は一般に旧情報と関わっているということを述べている．その根拠としては，文頭を疑問詞にできないという事実を挙げている．また柴谷らは，情報の概念について，旧情報(old information)と既知(known)，新情報(new information)と未知(unknown)はそれぞれ同じとみなしており，明らかに混乱している．Halliday & Matthiessenによる既知(given)と新(new)という情報単位の区別では，基本的に，文の中で情報単位が既知から新へ向かうように配置される．彼らの言う「情報単位」とは，”the tension between what is already known or predictable and what is new or unpredictable”(p.116)，つまり「すでに知られていること，または予測できることと，新しい，あるいは予測できないこととの間の緊張関係」である．し

かし情報単位は文法的単位と厳密に対応するものでもないと言う.

機能言語学に刺激されて, 久野は日本語の文では,「が」は新しい情報, つまり文脈から予測できない先行句を標識するとすれば十分と考えた. つまり総称か文脈指示であっても, 新しい情報をもたらすことがある. 予測可能性を軸に据えたことで説明自体はシンプルになったが, これは新たな混乱も同時に生む. 文脈からの予測が, どのように行われるかが不明であり, 情報の概念自体が曖昧なままである. また総称か文脈指示であれば, 代替的関係が聞き手の知識や先行する談話内に存在する可能性が高く, 予測はよりむずかしくなるはずである. 5節のプログラムでは「XはY」によって関係(X, Y)が未知のとき, 新しい知識を獲得している. 他方5節のプログラムでは「XはY」のXを述語名とみなしたため, Xを変数で与えると, (メタ述語を使わない限り)エラーになるので, 柴谷らが根拠とする経験的事実と一見一致する.

5節で示されたプログラムでは,「は」を2引数の述語として新規の知識の獲得に利用した.「XはY/2」述語には3種類のルールからなり, それぞれ既知のとき, 未知のとき, およびおかしなことの3局面に対応する. また5節で述べたように,「XはY」のXを述語名とすると, Prologでは述語名を変数で質問できないので,「何はホニュウ動物ですか」のような非文が自然に除外される. このためシンプルかつ機械的に論理プログラムに翻訳でき, それによって基準 σ を満たす.

基本的な情報モデルを,「XはY」や「XがY」といったゴールの実行(Prologシステムへの質問)によって新しい知識が, 関係(X, Y)として, 追加されていく過程として考えてみよう(図1).



図1 情報の追加

図1では項目ペアの間の関係は矢線で示されている. しかし一般的な用語の理解として, 情報はたんにメッセージを受信するだけでは生じない. すなわち, メッセージの受信者にとって, 何らかの価値(意味)のある情報として解釈されないかぎり,「ニュース」ではなく, 一時的に注意を引くということはあっても, たいていの場合, 時間がたつにつれて忘却されてしまうだろう. また, 心理学においてプライミング効果やアンカリング効果として知られる現象のように, 情報は必ずしもメッセージを受信した本人が自覚できるとは限らない.

情報の厳密な捉え方として, 確率に基づく情報理論(有本, 1980), 統計的意思決定論(宮沢, 1971), 情報の経済学(Laffont, 1985)の各分野の考え方がある. これらはベイズの定理を基礎とする点で共通しており, 受信されたシグナルによって影響を受けた脳内の情報処理が変化し, 結果的に観察される行動選択ないしその選択確率の変化として観察される.[30]

少なくとも, 知識獲得の局面では, 未知の事項の認知と新しい情報の発生とは並行していると考えられる. 助詞「は」はそのプロセスを媒介する言語刺激の一部である. この観点から,「は」は常に既知の事項や旧い情報を示すわけでは自明である. 仮にPrologを言語理解者のモデルとみなすと, 未知のときのルールが適用される状況では, 未登録の述語が背景知識に追加されるという, 純粋に言語刺激であるもの以外の言語主体の持つ内部状態(認識や価値観)の変化と, それに依存する行動選択(質問への応答)が変化する.(新しい)情報とは, この認知状態の変化, つまり既知と未知の差分, および行動の変化を伴う影響を伴う客観的な刺激であり, その変化分が新しい(情報の)部分である. ちなみに情報はそれ自体が「新

しい」「ニュース」という性質をもっているメッセージと考えられる．そうでないメッセージは無情報である．この観点からは，「旧い情報」という表現は異論理的であり，意味がない．[31]

本論文では，情報を，データと異なり，それ自体が客観的に共通に存在するものではなく，それを解釈する人間やプログラムなどに依存するものであり，認知機能，評価・指令機能，社会機能のすべて，あるいはそのうちのいずれかをもつものと考えることにする．[32] 情報は，複数の可能性のうちの一つ，ないしそのいくつかを特定し，他を除外するのが，その基本的な機能である（認知機能）．情報理論における情報量の概念は，主体の認知活動とは独立に定義されうるので，これに近い．しかし，それだけでは主体にとって何ら影響を及ぼさないもの，つまり無情報，あるいはノイズに過ぎない可能性がある．また曖昧な情報は，除外できる可能性が少ないため，情報量が低い（エントロピーが大きい）．曖昧でない情報は，唯一の可能性を特定し，それ以外の可能性を消去する．情報とは，判断や推論，および行動選択に影響するデータ，ないしその影響のことである（評価・指令機能）．それは行動の変化をデータとして観察した主体にとって，（再び）情報になる．言い換えれば，情報とは間主観的に観察可能な認知機能と同等である（社会的機能）．[33]また次節で述べるように，情報には誤伝，つまり誤った情報，あるいはノイズの含まれる情報（確率的に誤伝であったりそうでなかったりする場合を含む）と，正しい情報，ないしノイズの含まれない情報という区別がある．誤伝には，たんなる書き間違い（タイポ）や意図的な誤伝である嘘・虚偽情報が含まれる．

さらに，誤伝と関係して，前述の有標(marked)と無標(unmarked)の区別が可能である．文の主題は，助詞「は」によって標識される．また文中の（新）情報の位置は「が」によって標識される．前者はあるシグナルの受信によって発生する情報であり，後者はあるシグナルが受信されないときに推論される．例えば，次節の表1は，ある検査の結果をシグナルとして受け取ったときと受け取らなかったときとの反応を分類している．また総記や対比は，シグナルの様式上は中立叙述や主題と変わらないので，相対的に無標であるけれども，それぞれ「それ以外のこと」の不在や存在を意識するように，認知的状態の変化を生じさせる．総記や対比は，言及されていない代替的関係，あるいはそれを表現する文を，心の中の内部的なシグナルとして発生させるという点で，有標である．

9.2 情報と誤伝

論理プログラミングにおけるホーン節の値は真か偽のいずれか（true or false）であり，また否定演算¥+/1は「失敗による否定」である．これらは論理学における真理値や否定とは，必ずしも一致しない．より一般的に，情報について，表1に示す4パタンを考慮する必要がある．[34]

表1　情報についての4パタン

		論理プログラミング	
		真（成功）	偽（失敗）
自然言語	正しい	$1-\beta$	β
	誤り	α	$1-\alpha$

表1は誤伝のある情報構造の最も簡単な場合を示している．表1における行は，日常言語における「～である」や「～とは言えない」の意味と対応し，列はPrologに質問したときの「成功したゴール」と「失敗したゴール」に対応するものと考えて頂きたい．また表1において，α や β は，プログラムが十分に対象世界や

人間の感覚を反映できていないという意味で，誤伝，誤った情報（の確率）に相当する．αはPrologに質問するとゴールとして成功するが，人間のユーザの感覚では誤りであるように思われる事態（の確率）である．例えば，本節冒頭で示された「ホニュウ動物はクジラですか」と質問して，「はい」と答えるようなケースである．またβは人間の側では正しいと思っているが，論理学の直接適用では誤りと判断されたり，あるいはPrologが証明できないゴールであったりする場合（の確率）である．例えば，晴れた日に「今日は天気がいいですね」と質問して，「今日」を普通名詞として扱って「false」が返されるような場合である．しかしながら，意外さ・驚きという点では，むしろαやβが情報の程度を示す．

9.3 主題/対比/中立叙述/総記

図2は，「XはY」の形の文における主題と対比の解釈が切替るパタンを，久野(1973)を参考に要約している．既定でない名詞句は主題になりえないことに注意する．

同様に図3として「XがY」の形の文における中立叙述と総記についての久野の説明を図示する．既定でない習慣的動作の述部を持つ場合，中立叙述になりえないことに注意する．ここでのポイントは，総記と対比における，代替的関係の及ぼす効果の相補性である（表2と表3）．

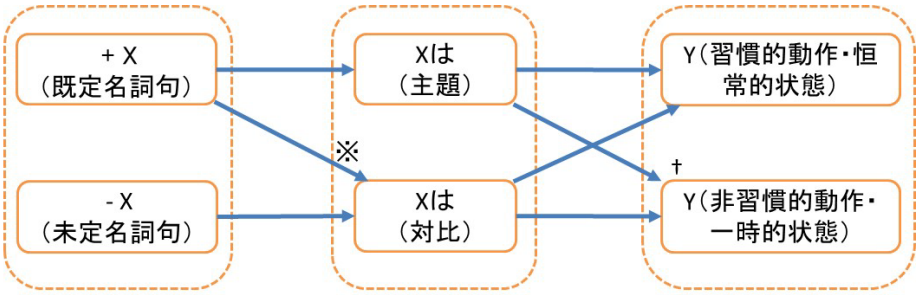


図2　主題と対比

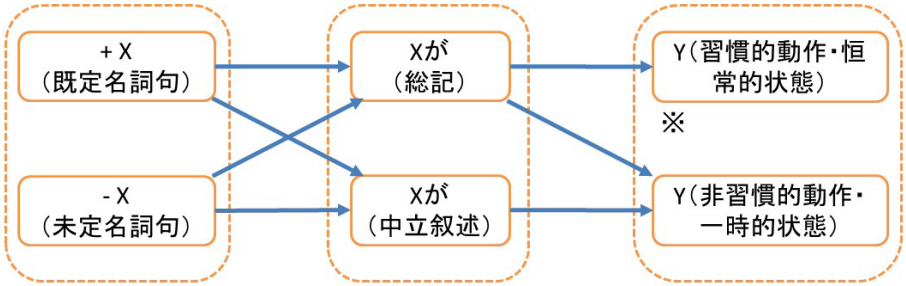


図3　中立叙述と総記

表2　総記と対比

	A＝雨，B＝降っている	
文の形式	AはB	AがB
代替的関係	(C, D)	(C, B)
代替的関係の影響	安定	不安定
解釈	対比	中立叙述

表3　総記と対比

	A＝太郎, B＝悪い	
文の形式	AはB	AがB
代替的関係	(C, D)	(C, B)
代替的関係の影響	安定	不安定
解釈	主題	総記

総記の「AはB」においては，代替的関係（C, B）の存在は中立叙述の阻止要因であり，意味の不自然さをもたらす．対比の「AはB」においては，代替的関係（C, D）はむしろその不自然さを抑制するために必要である．いずれにせよ，外部安定な代替的関係（C, D）が存在すれば，不自然さは抑えられ，総記や対比としての解釈が可能になる．言い替えれば，主題と中立叙述は，外部安定な代替的関係（C, D）を必要とせずに，関係（A, B）が内部安定している状態である．

またAとBの関係を表現する文を，「は」と「が」や二項の位置を入替て作ることによって発生する，中立叙述/総記/主題/対比の発生パタンは，図1と図2，および表2と表3に示された系統性があるものの，言語使用者にとっての主観で判断されるものであり，表1に示したミスマッチの可能性と共に，表4に示すように，不自然さ（「？」で示される）の程度を含めて，個人差が生じうると考えなければならない．

表3　総記と対比：主観的な解釈の例

No.	A	B	AはB	BはA	AがB	BがA
1	クジラ	ホニュウ動物	主題	対比	総記	総記？
2	イルカ	ホニュウ動物	主題	対比	総記	総記？
3	猫	ホニュウ動物	主題	対比	総記	総記？
4	カエル	両生類	主題	対比	総記	総記？
5	メダカ	魚類	主題	対比	総記	総記？
6	雨	降っている	対比	対比	中立叙述	総記？
7	雨	止んでいる	対比	対比？	中立叙述	総記？
8	雪	降っていない	主題	対比？	中立叙述	総記？
9	太郎	悪い	主題	対比	総記	総記？
10	太郎	悪くない	対比	対比？	総記？	総記？

9.4 総記や対比の認知的メカニズム

総記は，先行名詞句を顧客，述部をサービス窓口と考えた「待ち行列」のストーリーを用いると，理解しやすいかもしれない．述部Bをサービス窓口，先行句Aおよびそれに代替するCを顧客と考えたとき，サービスの混雑を避けようとする，もしくは排他的に利用したい場合，そのサービス利用に関して競合（図4での中間位置の図に相当する）が起き，またその結果，現在（＊, B）の＊の位置にあるAがCによってとって代わられる可能性がある．実際，C＝金魚の代わりに，C＝イルカとしてみると，関係（A, B）だけでなく，関係（C, B）も可能で[35]あり，関係の組{（A, B），（C, B）}はマッチングとしては実行できなくなる．そこで，Cはサービスを受けずに退出するか，またはAのサービスが完了するまでの間，待ちに並んでもらう（図4の右

側の図では，Cがダミーのサービスを受けていると解釈する）．

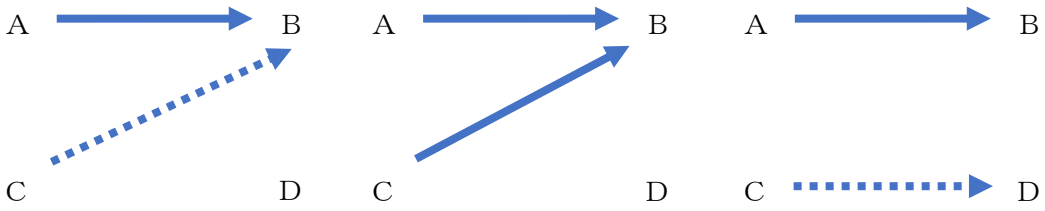


図4　総記における代替的関係：活性化（左），不安定化（中），再安定化（右）

ところで，「猫が寝ている」「猫が目を覚ました」「猫が伸びをした」と言うように，同じXについて中立叙述の「XがY」を連続的につなげて記述したとしても，不自然さを発生させない．これは待ち行列のアナロジーにおいて先入れ先出しでサービスが逐次終了して次の順番待ちの客を処理していることに対応する．同様に主題の「XはY」の場合も，「クジラはホニュウ動物」「クジラは海で暮らす」「クジラは体が大きい」「クジラは魚ではない」といったように，Xの諸性質をより詳しく記述するため，並行してもかまわない（5.7節で述べたように，助詞「と」を用いても記述できる）．

一方，対比の場合，つまり「雨は降っている」「雪は降っていない」「雹は降っていない」・・・というように，最初以外はすべて否定が続くはずである．関係の系列として書き直すと，「（雨，降っている）（雪，降っていない）（雹，降っていない）・・・」という形になる．読者は，これはルール節をタプルの形式で書いたものであるということに気付くべきである（ルール節は肯定リテラルが一つだけのホーン節であることを思い出そう）．つまり，ルール節の頭部をゴールとして質問されて，その本体の条件を下位ゴールとして逐次質問するので，（対比とは，）Prologの導出原理とほぼ同じものということになる．

「ホニュウ動物はクジラ」と「クジラがホニュウ動物」は，引数位置を入れ替えると，いずれも（クジラ，ホニュウ動物）に対して，（トカゲ，ハチュウ類），（スズメ，鳥類），（メダカ，魚類），（タコ，軟体動物），（ウニ，棘皮動物），・・・などといった，X≠クジラ，Y≠ホニュウ動物なる代替的関係（X，Y）を意識することで，対比あるいは総記の解釈として成り立つ（またおそらく前者は後者の主題化である）ことに注意する．

それゆえ，対比の「は」の認知的処理の最初の段階は，おそらく総記の場合と同様に，主題を提示する意図自体を不安定する．それはゴール（X，Y）の否定を追加して，背理法を用いた導出を開始したのと同等の状態である．例えば「雨は降っている」（X，Y）＝（雨，降っている）に対して，もしXの代替要素でX‘，X’‘，…が見つかったとすると，（X’，Y）や（X’’，Y），…を阻止するため，代わりに（雪，降っていない），（雹，降っていない），…を証明しようとする．しかし，主題の「は」についても処理の基本は変わらないはずである．主題の「クジラはホニュウ動物だ」や「彼は優秀だ」に対して，あるいは中立叙述の「雨が降っている」に対して，なぜ対比が生じないのかは直ぐに察しが付く．それらは事実であって，とくにルール本体内の条件を確認する必要はないからである．

こうして，総記や対比の正体がについて，一つの有力な疑いが浮かび上がった．つまり，Prologの導出原理そのものなのではないのか．総記と対比は，いずれも関係（X，Y）と競合ないし並行する代替的関係を意識させ，それを除外ないし両立することが可能であること，すなわち領域条件を確めようとする．[36]

またここから対比が導出と類似物であることが推察される．まず「XはY」のルール表現は，

（X，Y）:-（X’，Y’）;（X’’，Y’’）;…;（X’’’，Y’’’）

である. このルールの意味はマッチングと導出の同値性である. Xの代替集合{ X, X’, X’’, …}とYの代替集合{ Y, Y’, Y’’, …}との間の2部マッチング問題を考えると, 本体の条件に対応するマッチング案を逐次否定し, 残った可能性として頭部を採用する. 導出はルール節A∨¬B∨¬C∨…∨¬Dに対してB, C, …, Dを逐次追加して, 矛盾対¬B∨B＝true を1リテラル消去してAを推論する. A＝(X, Y), B＝(X’, Y’), … , C＝(X’’’, Y’’’)と置くことで, 導出とマッチングは相互に対応する.

総記の意味は, 例えば「クジラがホニュウ動物だ」の場合, まずX＝「ホニュウ動物」以外のZを述部のY＝「ホニュウ動物」と結合できないという主張である. そこで後述する「XがY」のプログラムにおいては, 代替的関係(Z, Y)を検知すると総記の意味が発生したと判断する.

そこで「XがY」のルール表現は, Y’≠YかつY’’≠Y… かつY’’’≠Yとした「XはY」である. あるいは, 誤解がなければ次のように書くこともできる.

(X, Y) :- ¬(X’, Y), ¬(X’’, Y), …, ¬(X’’’, Y)

この後示されるように, この例題の総記については容易に再現される. サンプルプログラムでは代替的関係がほとんど入力されていないから, 単独で「クジラがホニュウ動物だ」と入力しても総記は生じない. 失敗による否定によって, 上述のルールが成功するからだ. しかしたった1個の代替的関係を追加しただけで, 総記の意味を成立させるための認知的努力が別途必要になる.

導出に成功すれば, 主題の「は」や中立叙述の「が」と解釈できる. また導出の途中で失敗すると, バックトラックによって, 対比や総記の意味が成立するかどうか, 別ルールを用いて導出を試みる. ただしバックトラックの際, 参照されるルールはこれらの助詞によって若干異なる. ちなみに, Prologではユーザ自身に聞くことを手続付加として持つ下位質問を発行するには, 組込述語read/1を使うことができる. しかし, 実際の総記や対比の認知過程に近いのは, ユーザにすべてを任せるのではなく, 9.9節に述べるサーカムスクリプションまで半自動的に行い, その結果をユーザに示して受諾か却下の判断を委ねる方法である. 総記では, より狭い領域条件を設定するようにサーカムスクリプションを実行する必要が生じる. また対比の効果は, 主題に関わる代替的関係を適切に拡大するためのサーカムスクリプションである. その結果としての領域条件がどう定まるのかは, 以下に論じるように, ネットワークモデルの安定性(Ostrovsky, 2006)から予測することができる.

さて, 話を元に戻す. 総記と対比では, いずれも排他的にXをYと結びつける関係に注目させるという文使用者の意図そのものが不安定になる. そこで言語使用者が意識的に未言及の別の語と言及済みの語との関係を考えることによって, 調整する必要が生じる. 例えば, 3ノードA, B, Cからなるネットワークに, 当初リンクABだけが利用可能であるとして, もし(Ⅰ)新しいノードDが追加されて, 競合するリンクCBが活性化したり, あるいは(Ⅱ)既存リンクABが消えてしまったりするとしたら, ネットワークは不安定になるが, 不安定の要因となるリンクの除去ないし別のCDへの結合によって再安定化することができる(図4および図5を参照). Ⅰで起きる競合や, Ⅱで生じる消失は, 表1内のαとβの確率で発生する事態に相当しており, いずれも元の関係(A, B)の信頼性を低下させる.

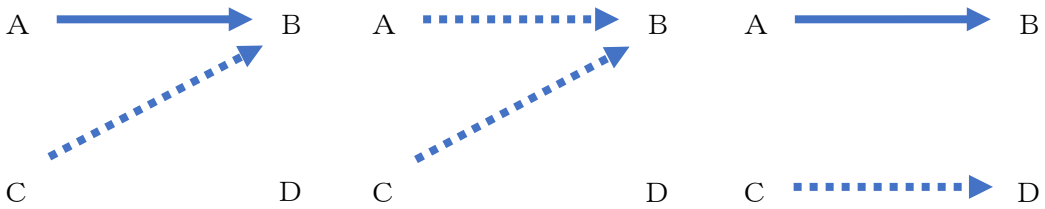


図5　対比における代替的関係：活性化（左），不安定化（中），再安定化（右）

実際，総記や対比は，ネットワーク最適化問題の一種と形式的に類似するだけでなく，それによって説明可能である（犬童,2018）．[37] 隣接する取引者（文理解では単語）ペアがその結合相手を探した結果できる鎖状に結合したネットワークが安定するための条件は，サプライチェーン分析における安定ネットワークの条件である同側代替性と対側補完性とほぼそのまま一致する．[38]

同側代替性：　関係（X，Y）が現時点で成立していると仮定して，（1）もしXと異なる左リスト内のZがYと未結合で，もし同リスト内のZ’が新たに利用可能になったとしても，関係（Z，Y）は新たに成立しない．（2）またもしYと異なる右リスト内のWがXと未結合で，もし同リスト内のW’が新たに利用可能になったとしても，関係（X，W）は成立しない．

対側補完性：　関係（X，Y）が現時点で成立していると仮定して，（1）Xと代替可能なZが新たに利用可能になったとして，元の関係（X，Y）が解消されることはない．（2）またYと代替可能なWが新たに利用可能になったとして，元の関係（X，Y）が解消されることはない．

新しくできてしまう関係とその系列は，鎖ブロックと呼ばれる．既存の関係にとって鎖ブロックはそれを覆してしまう脅威である．同側代替性の定義から明らかなように，総記とは，単語の認知的ネットワークの中で，左側同側代替性（1）を維持しようとする認知的活動である．「クジラがホニュウ動物だ」の「クジラ」以外の「ゾウ」など想像して記憶が活性化すると，未結合の「イルカ」が「ホニュウ動物」に結合してしまう可能性が生じる．このような鎖ブロックを除外できなければ同側代替性の最初の条件（1）は成立しなくなり，したがってネットワークが不安定化する．

また対比は対側補完性の定義にほぼ対応するが，2部ネットワークでは対側補完性と同側代替性は論理的等価であるため，いずれの条件を用いても，対比，総記とも説明できる．実際，前出の例文では，「イルカはホニュウ動物」だと教える前までは，関係（クジラ，ホニュウ動物）が成立していた．また「イルカはホニュウ動物」と知った後，関係（イルカ，ホニュウ動物）が成立している．このとき元の（クジラ，ホニュウ動物）が破棄されることはない（対側補完性による）．ところが，その後で「クジラがホニュウ動物」（総記）とか，「ホニュウ動物はクジラ」（対比，焦点）だと言われれば，（イルカ，ホニュウ動物）を破棄しないと，つじつまが合わなくなってしまう．つまり対側補完性に違反して，ネットワークとして不安定になってしまう．

要するに，安定なネットワークでは，現在のマッチング｛（A，B）｝に言外の要素Cを追加したとき競合が生じて｛（C，B）｝となったり，あるいは関係（C，D）を追加したときパートナーの交換が生じて，｛（A，D），（C，B）｝になったりしない．例えば，A＝「白い雲」とB＝「空に浮かんでいる」が安定な関係をなしているのは，関係（A，B）に競合し，とってかわる可能性のある代替的関係（C，B）をそもそも想起しにくいと言うことと等しい（これを内部安定性と呼ぼう）．内部安定である関係（A，B）を，「AがB」として述べた場合が中立叙述として解釈される．またそれは「AはB」に言い換えたときに，そのような代替的関係を思いつくことが容易ではないがために，対比の効果を生じさせる．また，言外の代替的関係（C，D）が存在していることが，関係（A，B）に外部安定性を与える．例えばA＝「クジラ」，B＝「ホニュウ動物」とすると，｛C＝金魚，D＝魚類｝のように安定した別の代替的関係が存在している場合，対側代替性が成り立つから，｛（A，B），

(C, D)}はマッチングとして安定する. これは図4と図5の右側の図で示されている状態に相当する.

安定ネットワークはリンク結合にかんするノードの自律的な選択の繰り返しの結果生じる均衡, すなわち最適反応としてのリンク付け替え(文献ではT写像と呼ばれる)の不動点である. [39]実際,「AがB」文における総記は, 潜在的な語Dを活性化して, 鎖ブロックCBが生じないか, もし生じているとしたらそれを阻止できるかをチェックしているので, いわば同側代替性の保守に相当する(図4参照). また「AはB」文における対比は, 未言及のCやDを活性化しながら, 言及済みの関係ABが保たれるかどうか確かめようとする. ABから見れば外部の代替的関係CDがあって, それぞれAやBへリンク先を変える動機がないことをチェックできたとすると, CやDの新たな出現によってAB間の既存の結合が脅かされることはないのだと安心する. これは対側補完性の保守に相当する(図5参照).

以上の議論をまとめると, 総記や対比の認知的メカニズムと対応するプログラムの基本仕様は以下のようになる.

総記のメカニズム 例えば,「クジラがホニュウ動物だ」で言及されている関係(A, B)=(クジラ, ホニュウ動物)は, A≠Cなる代替的関係, 例えば(C, B)=(イルカ, ホニュウ動物)が存在すると, {(C, B)}が{(A, B)}の鎖ブロックとなり, 内部安定ではなくなる. その結果, 主観的な不自然さを生じる. しかしこの鎖ブロックを阻止できる外部安定な代替的関係(C, D)(鎖ブロックの鎖ブロック)が見つかれば, 総記の意味に解釈できる.

総記のプログラム その論理プログラムでは, 代替的関係(C, B)を検出したとき中立叙述を阻止する. なおかつもし(C, D), D≠Bが存在していれば総記と解釈し, 存在しないときは「おかしなとき」に委ねる.

対比のメカニズム 例えば,「クジラはホニュウ動物」における関係(A, B)=(クジラ, ホニュウ動物)に対して, もし関係(A, B)が内部安定であれば主題として解釈できる. そうでないとき, 総記と同様に鎖ブロック{(C, B)}が見つかるが, やはり総記の場合と同様に, A≠C, B≠Dなる代替的関係, 例えば(C, D)=(カエル, 両生類)は容易に見つかるため, 対比の効果は仮に生じたとしても不自然さは緩和される. しかし「ホニュウ動物はクジラだ」における関係(A, B)=(ホニュウ動物, クジラ)に対しては, A≠C, B≠Dなる代替的関係を見つけることは簡単ではなくなり, それによって主観的な不自然さを生じさせる.

対比のプログラム 関係(A, B)に対して, もし関係(A, B)が内部安定なら「XはY」を主題と解釈する. そうでないときはA≠C, B≠Dなる(外部安定な)代替的関係(C, D)を探索して, もし見つかれば対比の効果を認め, 見つからないときは「未知のとき」で新規の関係を成立させることなしに,「おかしなとき」に委ねる.

9.5 XがY

以下では前節の最後に述べた「総記のメカニズム」や「対比のメカニズム」にしたがって, それぞれの論理プログラムを作成する. まず本節では総記をアプローチとしては, 以下に「XがYの形の質問応答パタンをいくつか用意し, それを仕様として満足する論理プログラムを, 次節で作成する.

まず以下のゴールを与える. [40]

```
(q.69) ?- はじめから教え直す, クジラ は ホニュウ動物, 金魚 は 魚, カエル は 両生類.
初耳です。
初耳です。
初耳です。
true.
```

以上の背景知識の下で，「何がホニュウ動物か」を聞くと，以下のように返答することが期待される．

(q.70) ?- _何 が ホニュウ動物 ですか.
_何 = クジラ.

また「クジラがホニュウ動物」かどうか聞くと，以下のように回答する．

(q.71) ?- クジラ が ホニュウ動物 ですか.
はい。
true .

上述の応答は，中立叙述として処理され，違和感なく理解されているものと解釈される．「クジラはホニュウ動物」であるかと聞いたときと同じ回答である．しかしここでは背景知識において「それだけが」の意味がすでに成り立っているから疑問に感じないのである．

今度は，ホニュウ動物についての別の事例を教える．

(q.72) ?- イルカ は ホニュウ動物 だ.
初耳です。
true.

しかし，「クジラがホニュウ動物」かどうか聞くと，総記の意味が生じて次のように応答する．

(q.73) ?- クジラ が ホニュウ動物 です .
[総記？]:イルカもホニュウ動物だし
false.

念のため「何がホニュウ動物」であるかを聞くと，バックトラックして複数の事例を返答できる．

(q.74) ?- _何 が ホニュウ動物 ですか.
知っています。
_何 = クジラ ;

知っています。
_何 = イルカ.

9.6 総記の発生をチェックする

総記の効果は，代替的関係が見つかったときに中立叙述を阻止し，見つからなければ中立叙述を成立させることである．[41]

```
総記の発生( X, Y ):-
        代替的関係( X, Y, Z, Y ),
        Z ¥= X,
        atomic_list_concat( [ Z, 'も', Y, 'だし' ], A ),
        nl,
        write( [ '総記？' ] : A ).
```

前出の質問応答パタンを再現するには,「総記の発生条件/2」を使って中立叙述を阻止するルールを「XがY」の最初のルールとして追加挿入するだけでよい. ただし, 受け答えを自然にするため,「既知のとき/2」を以下のように微修正して用いる.

```
既知のとき1( X, Y ) :-
        ¥+ var( X ),
        ¥+ var( Y ),
        関係( X, Y ),
        nl,
        write( ″はい。″ ).

既知のとき1( X, Y ) :-
        var( X ),
        関係( X, Y ).
```

以下に「XがY」のルールを示す.

```
:- op( 600, yfx, が ).

X が Y:-
        総記の発生( X, Y ),
        fail.

X が Y:-
        未知のとき( X, Y ),
        !.

X が Y:-
        既知のとき1( X, Y ).

X が Y:-
        おかしなこと( X, Y ),
        !,
```

fail.

「XがY」のルールは，「XはY」と同じ構成に，総記の発生のルールを追加したものである．少々意外に思われるかもしれないが，これだけで前節の例題の応答パタンが再現される．

9.7 中立叙述：情報の獲得

次に「が」が新しい情報を担うという観点については，どうであろうか．

(q.75) ?- はじめから教え直す.
true.

(q.76) ?- モーツアルト が 作曲家 です.
初耳です。
true.

上述の実行例のように，初出の関係で代替的関係も存在しない「XがY」では，新しい知識は「XはY」の未知のときと同様に受け入れられる．2度目に聞くと，「はい」と答える．

(q.77) ?- モーツアルト が 作曲家 です.
はい。
true .

代替的関係を追加した後に同じ質問を繰り返してみよう．

(q.78) ?- 'ジャン＝ジャック・ルソー' は 作曲家 です.
初耳です。
true.

(q.79) ?- モーツアルト が 作曲家 です.
[総記？]: ジャン＝ジャック・ルソーも作曲家だし
false.

9.8 対比の効果を再現する

対比の効果は，9.3節で述べたように，総記の発生と相補的な関係がある．総記は，代替的関係の存在と関連する．つまり総記と対比は，その原因となる代替的関係の存在と不在となっており，相補的に関連し合っている．一方，対比は，それが意識される状況では，代替的関係が存在しないことによって文の意味が理解しにくくなる現象である．[42]

そこで対比の効果を検出するために，「XはY」のプログラムに以下のルールを追加する．

対比のとき(X, Y) :-

```
        代替的関係( X, Y, Z, Y ),
        ¥+ (
                代替的関係( Z, Y, P, Q ),
                P ¥= X,
                Q ¥= Y
        ),
        nl,
        write( '(対比？)': ( Z, Y )).

対比のとき( X, Y ) :-
        代替的関係( X, Y, X, W ),
        ¥+ (
                代替的関係( X, W, P, Q ),
                P ¥= X,
                Q ¥= Y
        ),
        nl,
        write( '(対比？)': ( X, W )).
```

「対比のとき/2」は，対比の原因となる代替的関係の組合せを検出する2つのルールからなる．成功した場合は，括弧付の「対比？」のメッセージを印字してそれを示す．失敗すると，主題の解釈となる．最初のルールはそもそもXに代替する要素Zが存在しているが，ZにはY以外のWとの代替的関係がないケースであり，対比を検出する．これはZがYとペアリングされてXYが解消されない，それゆえ対側補完性に違反することがないようにするためには，ZがY以外のWとペアリング済であることを確認する必要があるという観点を反映したものである．つまり代替的関係(Z, W)の存在は，対比の原因となる代替的関係(X, Y)に対する阻止条件である．同様に，2つ目のルールは代替的関係(Y, W)に対する阻止条件を確認する．対比の原因となる代替的関係が存在しないときは，2種類の「対比のとき/2」のルールはいずれも成功しない．それは自明に安定するケースである．いずれにせよ，「対比のとき/2」が失敗すると，「XはY」は主題の解釈となる．

また「ホニュウ動物はクジラだ」や「学生は太郎だ」のように，関係(X, Y)の第2項Yを主題化していると考えられるケースについても，対比の意味合いを生じるものとして扱えるようにする．このため，以下のプログラムを前述の「対比のとき」の2ルールの直前に追加挿入しておこう．

```
第2項を主題化する対比( X, Y ) :-
        既知のとき( Y, X ).

対比のとき( X, Y ) :-
        第2項を主題化する対比( X, Y ),
        nl,
        write( '(対比？)': 第2項主題化 ).
```

最後に，「XはY」のルールに対比のケースを追加挿入する．以下のルールを，既知のときの「XはY」ルールと，未知のときの「XはY」ルールとの中間の位置に追加挿入する．

```
X は Y:-
        対比のとき( X, Y ),
        !.
```

上述のルールでは本体末尾にカット!/0が置かれているため，成功すると次のルールである未知のときの処理に進むことはせずに「XはY」自体を完了する．

ところで，ネットワークが安定するということは，追加できる知識に制限があるということを意味している．またそれによって，Prologに対する教え方の順序としての教授法に，良否の区別が生じる．

たんに白紙の状態から「XはY」を繰り返していくのであれば，問題なく教えることができる．

```
(q.80) ?- はじめから教え直す.
ture.
(q.81) ?- 雨 は 降っている, 風 は 止んでいる, 雪 は 降っていない, 雹 は 降っていない,雹 は 止んでいる.
初耳です。
初耳です。
初耳です。
初耳です。
初耳です。
true.
```

既に成立している関係について，同じ質問を繰り返しても違和感（対比）は生じない．

```
(q.82) ?- 雨 は 降っている.
知っています。
true.
(q.83) ?- 風 は 止んでいる, 雪 は 降っていない, 雹 は 降っていない,雹 は 止んでいる.
知っています。
知っています。
知っています。
知っています。
true .
```

しかし天候についての背景知識を与えることによって，対比を生じるようになる．

```
(q.84) ?- はじめから教え直す, 雨 は 天気の種類 です.
```

初耳です。
true.

(q.85) ?- 雨 は 降っている.
(対比？):(雨,天気の種類)
true.

対比を生じたため，知識は伝えることができなかった．そこで天気を伝えるため「が」を使う．

(q.86) ?- 雨 が 降っている.
初耳です。
true.

念のため確認の質問をする．

(q.87) ?- 雨 は 降っている.

知っています。
true .

雨の状態について，伝えることができたようだ．天気の種類も増やそう．

(q.88) ?- 雨 と 風 と 雪 は 天気の種類 です.
知っています。
初耳です。
初耳です。
true .

雨以外の状態についても，対比の違和感を生じることなく，教えることができる．

(q.89) ?- 風 は 吹いていない，雪 は 降っていない .
初耳です。
初耳です。
true.

クジラについて教えた後で，「ホニュウ動物はクジラ」を聞いてみると，意図通りに対比を検出する．

(q.90) ?- クジラ は ホニュウ動物 である.
初耳です。
true.

(q.91) ?- ホニュウ動物 は クジラ だ.
知っています。
(対比？):第2項主題化
true.

ところが，述部のホニュウ動物は，クジラと結合して安定しているため，複数のホニュウ動物をまとめて教えようとすると，うまく行かない．

(q.92) ?- クジラ と 犬 と 猫 は ホニュウ動物 である.

知っています。
(対比？):(クジラ,ホニュウ動物)
(対比？):(クジラ,ホニュウ動物)
true .

天気の種類を教えたときと同様に，先に実例を教えておけば，うまく行く．

(q.93) ?- はじめから教え直す，雨 は 天気の種類 です.
初耳です。
true.
(q.94) ?- 雨 が 降っている.
初耳です。
true.
(q.95) ?- 雨 と 風 と 雪 は 天気の種類 です.
知っています。
初耳です。
初耳です。
true .

9.9 サーカムスクリプション

古典的な論理学とは異なり，我々はある範囲（閉世界）を想定して思考（推論）する．例えば目の前のボートを漕いで向こう岸に渡ろうとするとき，櫓がなかったり，船底に穴があったり，そもそも川の水や向こう岸がなかったりする可能性は，すぐに思いつかないかもしれない．

常識やそれに基づく推論では，明示的に言われていないことについて，半ば無意識的に，否定するか，肯定するか，あるいは無視するかを選択する．5節の「XはY」プログラムは「クジラはホニュウ動物」と教えた後で，「ホニュウ動物はクジラ」を，初出の関係として受け入れる．より正確には，クジラ（ホニュウ動物），ホニュウ動物（クジラ）という2個の事実節として登録する．また関係としての対称性から，これを「クジラ＝ホニュウ動物」という等位関係と解釈できるかもしれない．しかし私たちは，「ホニュウ動物にはクジラだけではなく，猫や犬やヒトも含まれる」ということを知っているため，違和感を覚える．また「クジラがホニュウ動物だ」における総記の意味は，これとほぼ同じ効果を与える．そこで私たちは，この主張を却下したり，

無視したりすることもできるが，「ホニュウ動物はクジラだけだ」と主張する相手に対して，その主張を意味のあるものとして受け入れることができる可能性を留保する代替案もある．すなわち，少なくとも当面の間，クジラ以外のホニュウ動物の事例を除外し，思考の混乱（矛盾）を避けることによって対処する．

このように，常識推論は対象世界を制限したモデルを作ることで，「結論へジャンプする」ことができ，またそれによってときに失敗もする．またそれはAI研究において，「サーカムスクリプション」と呼ばれるものと一致する(McCarthy, 1980)．サーカムスクリプションとは，「事実Aからある性質（プロパティ）Pを持つことが推論できたオブジェクトだけが，その性質Pを満たすオブジェクトだと推測する」ためのルールのことである．McCarthyは次のようにも述べる．「より一般的には，関係P（ x, y, …, z ）を満たすことを示すことができたタプル< x, y, …, z >を，この関係を満たすタプルのすべてだと推測する．」適切なタプル，つまり（n項の）関係の集合がサーカムスクライブされる．[43]

Prolog自体にもある種の閉世界仮説が用いられている．（エルブラン空間全体を探索する代わりに，）導出に失敗するゴールはその否定が真であるとみなされる．節として登録されていないことは，質問されても成功せず，偽であるとみなされる．[44] これは失敗による否定（NAF）と呼ばれる．

また常識推論は，古典的な論理学の推論（三段論法）が持つ単調性が損なわれるため，非単調推論とも呼ばれる．サーカムスクリプション概念は非単調推論の一種として提唱された．McCarthyによるサーカムスクリプションの形式化について，本論文では詳しく論じないが，上述のサーカムスクリプションの概念自体は，むしろPrologの行う推論に近いことはほぼ明らかである．オブジェクトBについて，Aという事実から「オブジェクトBが性質Pを持つ」ことが，あるルールによって示されたとしよう．これはPrologでは，P（B）を頭部として持つルール「P（B）:-A」があって，ゴールP（B）が成功したということに対応する．そして，X＝Bだけ がP（X）を成功させる単一化であり，またB中の各変数に対する単一化代入をタプルとして，データベース述語のfindall/3 あるいは setof/3 で収集したリストが，サーカムスクライブされた内容である．

具体例を示そう．表4のA列とB列のペアから作られる関係の集合Ωとする．

Ω＝{（クジラ，ホニュウ動物），（イルカ，ホニュウ動物），（猫，ホニュウ動物），
（カエル，両生類），（メダカ，魚類），
（雨，降っている），（雨，止んでいる），（雪，降っていない），
（太郎，悪い），（太郎，悪くない）}．

対応するPrologプログラムは，以下のように書くことができる．

表4中の関係(クジラ，ホニュウ動物).
表4中の関係(イルカ，ホニュウ動物).
表4中の関係(猫，ホニュウ動物).
表4中の関係(カエル，両生類).
表4中の関係(メダカ，魚類).
表4中の関係(雨，降っている).
表4中の関係(雨，止んでいる).
表4中の関係(雪，降っていない).
表4中の関係(太郎，悪い).
表4中の関係(太郎，悪くない).

```
表4中の関係全体( Ω ):-
        findall( ( X, Y ), 表4中の関係( X, Y ), Ω ).

表4の知識にかんするサーカムスクリプション( _関係, _関係の集合 ):-
        _関係 = ( X, Y ),
        _ゴール = 表4中の関係( X, Y ),
        setof( _関係, _ゴール, _関係の集合 ).
```

以下に実行例を示す.

```
(q.96) ?- 表4中の関係全体( Ω ).
Ω = [(クジラ, ホニュウ動物), (金魚, 魚), (カエル, 両生類)].

(q.97) ?- 表4のサーカムスクリプション( ( Γ, ホニュウ動物), _関係の集合 ).
_関係の集合 = [(イルカ, ホニュウ動物), (クジラ, ホニュウ動物), (猫, ホニュウ動物)].
```

Ωに相当する背景知識を持っている言語使用者が,「クジラがホニュウ動物だ」という文について, クジラだけがホニュウ動物であると考えることができる, すなわち総記の解釈に伴う意識的な除外を必要としない(中立叙述の解釈が可能である)のは, すべての行のうち, 2行目と3行目を削除した後, その任意の行を集めた部分集合である. 2行目と3行目の代替的関係をDと置く.

$D=\{(A, \text{ホニュウ動物}) \in \Omega \mid A \neq \text{クジラ}\}=\{(\text{イルカ}, \text{ホニュウ動物}), (\text{猫}, \text{ホニュウ動物})\}$.

中立叙述が可能な最小集合は, $K_{min}=\{(\text{クジラ}, \text{ホニュウ動物})\}$であり, また最大集合は, $K^{max}=\Omega-D$である(ここで記号ーは集合差を表す).

当面の談話知識Γとすると, $\Gamma \supseteq K_{min}$の場合が既知, $\Gamma \cap K_{min}=\varnothing$の場合が未知であり, また$\Gamma \cap D \neq \varnothing$の場合, $\Gamma \cap D$を言語使用者が自分自身で意識的に阻止することができれば, すなわち$\Gamma \cap D$内の要素の除外を意識できれば, 総記の解釈が成立する.

対比や総記は, 少なくとも当面の間, 違和感の原因Dを除去するように, 代替的関係(X, Y)の全体の集合Ωから関係(A, B)を含む部分集合を選び直す認知過程である. 反対に, 総記や対比の認知効果が生じない, すなわちΓの選び直しの必要がない場合が, 中立叙述や主題である.

10 認知モデルとして残された課題

前節では, 5節のアプローチを拡張し, 既知性を述語がassert済か否かに対応させ, 情報を質問文への応答パタンの変化をもたらす言語刺激と考え, 総記や対比の意味合いの発生についても, 簡潔なPrologプログラムによって, 曖昧さなしに, 計算的に表現することができた. しかし前節のアプローチではまだ日本語の柔軟な表現のほんの一部分を論理プログラミングで再現できたにすぎず, 現実の日本語の豊かな表現力とのギャップを十分に埋めることができたと言うにはほど遠い.

本節では，論理プログラミングと自然言語との間のギャップについて，認知科学と意思決定に関連するより広い領域の知見を参照しつつ考察する．論点が拡散するきらいはあるが，あえて余剰として削ることはせず，あえてありのままに示す．以下では，まず言語学で論じられていた情報構造に潜在するパラドックスを指摘する(10.1節)．次に，Prologの誕生に関連する文法理論との関係について振返り，文や文章における隠れた再帰性について論じる(10.2節)．またAIにおける知識表現手法の中で，とくに論理形式が，自然言語と隔てられた人工言語であるという性質の背景を探る．不可能性定理のように論理は単純だが推論は複雑である場合もその一つだ（10.3節)．さらに心理学におけるルール（条件文）推論の研究における否定パラダイムと行動経済学の接点に言及する（10.4節)．最後に意思決定支援のような現実の情報システムにおける応用可能性を論じる(10.5節)．

10.1 情報のパラドックス

例えば，情報の新旧について，もう一度考えよう．言語学では「文脈からの予測可能性」(久野，1973)から説明されており，また文の情報構造については，旧から新，新から新，および新から旧の3通りの流れのパタンのいずれかであって，旧から旧のパタンは，「新情報」を持たない話であり，語用論の観点から排除されるとされた(井上，1989 p.161)．

久野や井上の情報にかする議論は生成文法よりもむしろ機能言語学の影響を受けたものと考えられる．また旧から旧への流れが阻止される原因を，文自体が予測可能で，情報を伝えないとすると，Griceの協調原則，とくに量の公理の違反であり，また確率論的な情報概念を用いると，先行文脈を条件として次に現れる単語の予測確率が事前確率と変わらない．またもしそれがシステムのふるまいを変化させたならば，情報が得られたと認識される．しかしながら，これは情報が認知されるプロセスの結果であり，そのメカニズムの説明ではない．さらに，この解釈には論理的なパラドックスが隠れている．

関係（X,Y）の新規性は，XやYの既知性とは独立である．XとYがともに既知であっても，関係（X,Y）が新規であれば情報を生じる．本節の解釈もこの流儀に従った．ところがもし関係としても既知である受信されないはずの(旧,旧)が受信されたなら，意外である．意外性のあるメッセージは新しい情報である．一般化すれば，{旧,新}^{n} の系列中に一つの予測不可能な「新」があると新情報である．しかし「新」の位置が最後しかないとすると（論理的にそうなるが），それは予測可能であり，「旧」である．かといってメッセージが来ないと信じると，メッセージは送信されて意外性，つまり「新」を生む．これは論理学における自己言及パラドックス，あるいは意思決定論やゲーム理論において抜き打ち試験（バックワードインダクションパラドックス）[45]として知られる問題の変種である．いずれにせよ完全な予測はむずかしいため，確率的な予測の問題として考察される余地がある．

10.2 文法と再帰

論理プログラミング言語Prologはその開発の経緯[46]から，自然言語処理と関連し，とくに確定節文法（DG）とかかわりが深い． ColmerauerがPrologに関連して確定節文法（DG）をより柔軟にしたメタモーフォシス文法（MG）を提唱したことをきっかけに，さまざまな論理メタ文法が考案された（Dahl, 1990）．MGの規則は Ｓα --> β で，左辺の非終端記号Sを，右辺の終端または非終端記号の系列α，βに書き換える．ただしβについては，手続き呼び出しのこともある．DGの規則はαの部分のないMGである．ただし実際のPrologのDCGパーサでは， S--> α β のように書かれ，αはカンマで区切られた終端または非終端記号の系列，βは注括弧で囲まれたゴール節である．

Dahlは英語の“The man that John saw”を例にとる．この関係節は“The man [John saw the man]”の関

係節化であり，パースツリーの一部にループが生じる．Prologとλ計算はともにTuring完全であり，計算の普遍的モデルと解釈されている万能Turingマシンを模倣する．上述の関係節の元の形には“the man”が再帰的に現れる．λ計算風に書くと，“λx.(λy. y saw x)”の形式であり，2引数の関数 f(x, y) のカリー化と同じである．ちなみに，1節の冒頭で紹介されたプログラミング言語の定義は潜在的に再帰を含む．「プログラムを表現するための人工言語」には「プログラム」という別の用語が用いられており，常識的にそれは「プログラミング言語を用いてコンピュータに実行させる作業内容を指示する文や命令」である．

このように，英語の関係節を一例として，自然言語の表現には明示的に現れない再帰が埋め込まれていることが予想できる．指示代名詞，人称代名詞などもそうであろう．AIプログラミングとは離れて，自然言語文から論理表現を抽出する試みとして，モンタギュー文法（自然言語断片の構成原理に基づく可能世界意味論への翻訳）が知られる．[47]

10.3 知識表現と論理モデル

自然言語理解へのAI的アプローチの特徴は，言葉の意味に対応した知識表現を抽出することである．知識表現の種類は，論理表現の他に，ルールベース，意味ネットワーク，フレーム，ニューラルネット，確率学習モデルなどである．法的推論の研究では，法律文章から格文法に基づくモンタギュー意味論を伴う論理表現（PRL）に翻訳し，さらに法的推論が可能な論理形式（CPF）を抽出する手法が提案されている（岡田，2000; 吉野，2000）．

論理もまた人工言語であり，プログラミング言語と同様に，自然言語との間に溝が生じうる．経済学者もっとも，一定の条件のセットから論理的に妥当な推論によって導かれる定理について，人間が直ちに自明と信じることができなくても不思議はない．そうでなければ数学など存在しないはずであろう．例えばKenneth Arrowの一般不可能性定理のように，たとえ一つひとつのルールについては論理的に説明可能であり，かつ直観的に理解できたとしても，ルールの全体（公理系）としては矛盾しており，反直観的な論理的結論を導くということもありうる．Arrowは完備的で推移性な選好順序を仮定し，その任意のn組からそれ自体への関数で，定義域の無制限性，推移性，全会一致性，無関係な代替案からの独立性，および非独裁性の5条件を満たすものは存在しないことを証明した（Arrow, 1951/1963）．ちなみにIndo(2007)は一般不可能性定理の基礎的ケースについて自動証明できる簡潔なPrologプログラムを提示している．

10.4 ルールはどこからくるのか？

推論や計算の複雑さが，人間の合理性を制限するという発想は，Herbart Simonによって限定合理性(bounded rationality)と名付けられた．[48]しかしながら，人間の思考の非論理性がすべて限定合理性によって説明できるわけではない．事実，たった一つの，複雑でもないルールですら，人間は正しい判断ができないこともある．認知心理学者が古くから扱ってきた実験の一つに，Wason(1968)の選択課題（いわゆる4枚カード問題）がある．実験参加者に「PならばQである」といった単純なルールの正しさを検証するため，一方の面にPまたはPの否定，他方の面にQまたはQの否定が示された4枚のカードを机の上に並べて，裏返して調べなければならないカードを選ばせると，論理的に正しい選択パタンをする割合は1割程度ときわめて低くなる．

現実の人々は，Wasonの選択課題（その抽象的な肯定表現の場合）において，裏返してみる必要のあるQの否定を示すカードを正しく選択できない．また裏返す必要のないQを示すカードを選んでしまう．ところが，問題表現を「PならばQではない」に変えるだけで，9割方正答パタンを選ぶようになることも知られる．これは否定パラダイムとして知られる．否定パラダイムは，行動経済学でフレーミングと呼ばれる現象（肯定

表現ではリスク回避，否定表現ではリスク愛好に切替わる）の一例と考えられる．[49]

Marvin Minskyの提唱したフレームもまた，たんに典型的な知識の集まりとして理解されるべきものではなく，論理プログラミングと関連した新たな視点から再検討されてもよいだろう[50] それは環境に適応した知能が適用できる認知機能のアプリケーションであり，逆に，知能は自分の持つ認知機能アプリケーションをあまり変えずに適応できる環境を選ぼうとするだろう．ちょうど，自然界に暮らす生物たちが都市に進出してその生態系を創出するのと似ている．

包摂アーキテクチャのように，知識表現を持たないAI手法も提案されている．Brooks(1991)の包摂アーキテクチャ（SA）は，あまり高等でない生物の動作原理を模したロボットの動作原理であり，障害物を避けながら環境内を動き回って探索し，未知の世界の地図を作る．SAは惑星探査機や家庭用掃除ロボットに応用されており，特定分野において実用的な成功を収めた．

環境に適合した知能，その知識表現という観点は，認知科学で探求され続けているテーマであるが，現在の機械学習を基礎とするAIにも，潜在的な関連はあるはず．その性質上，分析者はデータを集めるだけで，どのような特徴量が学習によって抽出されるかについて明確にコントロールしないので，結果的に環境に適応した知能が手掛かりとして無意識的に利用できる信号を学習するというのと似ている．これについても明確に議論されないままである．

10.5 意思決定と社会

そもそも，自然言語表現はしばしば曖昧であったり，不正確であったりするものである．またそのことが自然言語プログラミングのデメリットとして古くから指摘されてきた．また顧客や開発者の双方とも，さまざまな行動経済学的な（あるいは認知的な）バイアスによって判断自体が歪められている可能性があり，言語表現（フレーミング効果）もまたそのバイアスを誘発する要因の一つである．

自然言語インタフェースとは独立に，意思決定支援システム（DSS）の領域においても，意思決定を行う業務担当者や管理者の認知的バイアスの問題は意識される．しかしながらこの弱点は，翻訳された確定節の動作から，対応する元の自然言語表現に遡ってチェックすることができるというメリットでもある．それはデバッグやシステムの定義自体の問題点を診断するのに役立ち，適切に使用すれば，顧客の言うことに逆らうことなく助言するための言語表現技術の一部にすらなりうるはずである．自然言語表現を確定節（ルール節）に翻訳することは，それを用いたシミュレーションにより，ルール表現に対する理解のメンタルモデルを意味し，またその判断のバイアスを検出できれば，行動科学的な見地から診断と助言を行うために役立つ情報が得られるはずである．

11 組織とエージェンシー

本節は，前節の終わりで述べた論理プログラミングと自然言語との間のギャップについて，とくに意思決定と社会との関連で述べた観点の延長であり，いわば残された課題についてのさらなる補遺である．

11.1 合理性の追求としての情報システム

組織や個人は，少なくとも合理的にふるまおうとしているかぎり，ある特定の目的を達成するために，現状維持のオプションを含み，その資源の制約の下で実行できる手段の組み合わせの中から一定の基準を満たすような行動を選択する．現実をこのように抽象化した記述は，現実の問題解決手法においても隠れた共通知識である．

コンピュータは，プログラム化された大量の反復を含み，人手でそれを行うのが事実上不可能であるような作業を効率化する．それは，かつては誰かが誰かに依頼された仕事であったかもしれないし，また必ずしもルーチンな作業だけではなかったかもしれない．組織や個人をクライアントとするインターネット技術（IT）や人工知能（AI）も，冒頭で述べたような合理性をもつクライアントを前提とする（コンシューマ向けITとの対照をなす）．

実際，最適化[51]やAIは，人間が行ってきた業務を改善するための複雑な作業を，（非）実行可能解を探索する問題として解決するための技術であり，ビジネスでのインターネット活用が本格化する以前から実用化されていた．最適化が解くべき問題を目的関数と制約条件の組として数式表現し，適切な求解アルゴリズム（とそのソフトウェア化であるソルバー製品）を適用してコンピュータで解を見出すのに対し，AIの場合，人間が行っていた知的作業ないし業務的機能を（システム要件として）満足するため，ルールベースやニューラルネットといった，知識表現のタイプとそのアルゴリズムを選択し，コンピュータで実行できるAIシステムとして設計する．[52]

11.2 仕事を依頼するための言語

言うまでもなく，プログラム化された仕事はコンピュータが実行できる部分であり，またそれを依頼するための言語的コミュニケーションは，プログラム売買やスマートコントラクト（契約の自動化）のように，それ自体がプログラム化されるまでは人間が行ってきた．これらは本来どのような関係にあるものだろうか．自明な答えは「仕事の依頼」と「仕事の遂行」のペア，経済学的な契約の理論でいうエージェンシー関係である．あるいは形式的に言うと，Montagueの言う普遍文法（UG）であり，自然言語とその意味論モデルとの対応関係（準同型写像）である．情報システムの構築は，伝統的に，「依頼」部分に自然言語を用い，また「遂行」部分に手続的なプログラミング言語を用いてきた．論理プログラミングや関数プログラミングは，その依頼の部分に力点が置かれる．

人が誰かに何かを頼む場合，言葉を使って依頼したり，その作業内容についてときに言葉だけでなく図表を使って指示したりする必要がある．しかし仕事の依頼に，最初から数式やコンピュータプログラムだけ送ってくるということは考えにくい．手紙の前置きの後，「このプログラムを使って，かれこれのマシンで作業してくれ」と言った自然言語表現，つまり日常人が話したり書いたりする言葉が付け加えられ，末尾を丁寧な依頼や感謝の言葉と署名で締めくくる．またコミュニケーションが成り立つための背景として，単語や文の理解以外に，作業を依頼される側の情報環境やスキル，作業を依頼するための契約の条件などについての知識の共有や一定の信頼関係が必要である．例えば，IT/AIベンダーが，最初から数式（最適化問題）やコンピュータ用のプログラム（AIシステム）だけクライアントに送りつけるということは考えにくく，例えば情報提供依頼書，提案依頼書，サービスレベル合意書などの文書をやり取りする慣行や，法律上，契約を成立させるための諸手続きと様々な図表を含む複雑な言語的コミュニケーションが必要である．

11.3 自動化によって仕事はなくなっても言語はなくならない？

現代は，Google，Apple，Facebook，Amazon，Microsoft，IBM，Spotifyといった，巨大データを扱うIT企業がAIを標榜したビジネスへと舵を切り，そのAIプラットフォーム化を展開している．現在注目されている機械学習は，人によるプログラミング作業の代わりにデータに学習アルゴリズムを適用してAIシステムを作る技術であり，画像理解，ボードゲーム，自然言語処理，音楽や絵画などの創作などに適用されている．自然言語処理では，単語間の類似性判定や単語系列の予測に応用され，GoogleのBERT，OpenAIのGPT-2などが話題になった．チャットボットはビジネス分野において広範に用いられている．

他方，一般の人々から得たパーソナルデータを活用するIT企業や政府に対する不信感を背景に，AIにかんする倫理的問題（デジタルエシックス）が議論されるようになった(Floridi, 2019)．Floridiのあげる4つのデジタルエシックスは，1）倫理ショッピング，2）倫理ブルーウォッシング，3）倫理ロビイング，4）倫理ダンピング，5）倫理縮小である．これらの背景は，倫理の標準の欠如，欺瞞的な倫理対策，法的規範導入に対する抵抗への濫用，非倫理的な研究開発の輸出，倫理的作業への責任の曖昧さである．またこれらは「信頼できるAI」に対する脅威であり，それによってAIは人々の脅威となる．AI脅威論は，自然言語処理や自然言語プログラミングの問題が潜在的に提起してきたことでもある．これまでの研究の反面として，人間自身は言葉を自然に使うことができても，どのようにして使っているかをルールとして書くことは，たとえ平均的な人々よりもすぐれた知能をもった研究者であってもむずかしいということである．もし自然言語プログラミングにおけるトレードオフ制約が緩和できないのであれば，訓練用データに機械学習アルゴリズムを適用してプログラムを自動生成する最近のAI技術を使った方が，初級レベルの人間のプログラマーを訓練するよりも生産性が高くなるだろう．

オックスフォード大の研究者による「10年後になくなる仕事」のランキング[53]では，米国人の702種類の仕事のうち47%が機械学習やモバイルロボティックスによる自動化される可能性が高いとされた(Frey & Osborne, 2013)．これを「AI活用によるディスラプション」の懸念と捉えることもできるだろう．同ランキングで自動化のリスクが最低の職種はレクリエーション療法士(0.3%)であり，仕事がプログラム化される確率50%以上が403職種（57%），90%以上が264職種（38%）である．プログラマーは293位で自動化の確率48%である．Osborneらの自動化の確率推定は，機械学習の一種[54]を用いているが，教師データとして70職種が選択されている．この確率は，直観的に言うと，つまりほぼ確実に自動化されると見込まれる職種グループと，ほぼ確実に自動化されないと見込まれる職種グループの，どちらのデータパタンに似ているかという判別関数から計算される．つまり自動化確率1/2に近い予測結果は，プログラマーの仕事をデータ化したとき，両グループへの判別がどっちつかず（平均値）だということを意味するにすぎない．

Frey and Osborne(2013)は自動化の阻害要因を，「知覚と操作」「創作的知能」「社会的知能」の3群の変数にまとめた（表1）．このうち社会的知能は，交渉，説得，他者への支援と配慮といったコミュニケーションを含む．ただし，前述のようにこの予測モデルには教師ラベルが貼られた70職種による主観的バイアスが含まれている．ちなみに，企業の経済学では，企業は契約や諸コミュニケーションによる取引コストを削減するために組織化されると考えられている．外注やクラウドソーシングによって組織に属さずに行われるプログラミング作業のための取引コストが内製化のコスト上回れば，それらは再び組織内にとりこまれ，自動化されていくと予想できる．すなわち，契約にともなうコミュニケーションの煩雑さは，現在は自動化の阻害要因の一つと考えられるが，内製化や契約自体のスマート化の進展の両面から克服されうる．

現実的な自動化の傾向は，自動プログラム合成によってプログラミングがAIにとってかわられる置換プロセスと，デジタル変革推進による雇用拡大の影響が相克するだろう．また近年のAIで注目されている機械学習アプローチは，大量の言語的データに基づき言語表現を数値ベクトルに変換し，ネットワークの重みを適当な（最適化）アルゴリズムを適用して学習させる．その結果得られる知識表現は，言葉による動作説明を可能にする論理形式やルール形式を伴わなずに，結果的に人間のパフォーマンスに近い，あるいはそれを超えるものである（それゆえ前述したような現実の人間に近いとも言えるが）．しかしAIの社会活用では，その倫理的側面として説明の責任を伴う（例えば自動運転の車が人身事故を起こした場合や資産運用の自動取引プログラムが違法な売買を繰り返した場合など）．

12 結論

人間は不自由なく言葉を使用することができるにもかかわらず，コンピュータと自由にやりとりすることができないのはなぜか．もしPrologこの質問をすると失敗するだろう．前半部分が論理的に偽であるからだ．実際，人間はなぜ自分たちが闊達に言葉を使用できるのかを説明できないことがある．言葉で人間とやりとりできるコンピュータを目指したAI研究は，コンピュータがビジネスと産業に現実適用され始めた1950年代には，早くも独立した研究分野として確立されていたが，それは人間知能の学際的研究領域である認知科学を成立させるとともに，現実の意思決定や管理業務に役立つ経営情報システムのコアになることが期待された．2度のAIブームとそれに続くAI冬の時代を経験し，現代はまたAIが社会的に注目され，かつまたその社会的な脅威が喧伝される時代である．これらは11節で論じた本論文の背景にあるより広い文脈に通じる．一方では，スマホやタブレットを自然言語で操ることは部分的に実現され，日常の中に溶け込んでいるが，他方では，自然言語プログラミングの実現はむずかしいとされたままである．しかし本論文の9節における助詞の考察から推察されるように，日本人は毎日Prologを喋っている可能性が指摘されうる．またもしそれが正しいとすると，日本の第5世代コンピュータプロジェクトがPrologに着目し，またそれについて海外の研究者が違和感を持ったとしても不思議ではない．

本論文では日本語プログラミングは以下の2条件を満たすものと考え，その簡便な実行方法を提案した．

1．日本語で，領域に固有でないプログラミングが可能である．つまりあらかじめ定められた限定された範囲の機能の遂行とそれに対応する特定の自然言語表現パタンにとどまることがない．

2．たんに汎用性のある既存のプログラミング言語のコード（断片化された英語の表現に近い）の日本語化にとどまらず，原理的に表現に制限のない，自然な日本語表現でプログラミングできること．

これまでの日本語プログラミングの現状に照らして，上述の2条件の両立は容易でないと考えられるため，本論文では一方で，助詞をファンクターとみなすことで比較的容易に実現できる論理プログラミングの日本語化という実用的な簡便法と並んで，助詞の言い回しによって生じる微妙な意味の変化に着目しながら，日本語の論理プログラム化を目指すアプローチを採用した．本研究では，その実用性の追求の事例として，組込述語の日本語化と文書からの自動プログラム生成を試みた．前者については付録Bにそのプログラムを掲載した．後者については，相続税の計算についてのサンプル文書から確定節を自動抽出した結果を示した．具体的には，アウトラインの抽出と，数式部分の抽出であり，相続税に関する領域知識は用いられていない．つまり，特定の目的に限定されずに書かれた日本語文章から，確定節へ翻訳するPrologプログラムを自動生成した．もちろん，これはかつてのSHRDLUと同じように，成功した例をとりあげた事例にすぎない．実用レベルと言わないが，多少の修正により軽程度の業務適用は可能と思われる．

また本論文では，助詞「は」と「が」の言い回しを巡って，既知と未知，および情報の新旧，総記と対比といった論点をPrologプログラム化によって明確化し，そのメカニズムを説明した．直観的には，「XはY」や「XがY」の形の日本語文の意味は導出の類似物であることが推察された．すなわち「XはY」は”(X, Y) :- (X', Y'); (X'', Y''); … ;(X''', Y''')”，「XがY」は”(X, Y) :- ￢(X', Y), ￢(X'', Y), …, ￢(X''', Y)”における導出の実行とみなせて，また対比や総記はその失敗の検出，バックトラックの試行に相当する．またこれらの意味的な変化は，ネットワークモデルの安定性(Ostrovsky, 2006)，および常識推論の研究におけるサーカムスクリプション(McCarthy, 1980)と関連することが論じられた．またPrologプログラムによる対比と総記の再現を通じて，知識の教授法にかんする知見が得られた．総記と対比は，競合ないし並行する代替的関係を意識させ，それを除外ないし確認するよう言語使用の主体を促す．またそれ

によって知識状態を不安定にする情報の追加を防いでいると考えられる．それによって教示の順序に制約があり，新規の知識の導入において，最初に定義を示し，次の事例を教える必要がある．日本語プログラミングとしては若干不自由である．しかし人間の言語理解の模倣でもある．とくに馴染みのない概念の導入において現実の教示の場面では経験上実践されている．本論文ではもっとも単純な形の文例しか扱わなかったが，理解容易性向上目的で広く応用の可能性がある．この他，本論文では取り上げなかったが，筆者は文章で述べられた定理証明の抽出も試みている．また8節で述べた方法は，構造モデル化(Pearl, 1997)に応用される可能性がある．これらの発展的な研究については，今後の課題としたい．

付録A

Prologは異なるAI分野が合流して誕生した．自然言語処理と最初のProlog言語は，1972年にマルセイユでColumerauerとRousselによって自然言語質問応答システム向けに開発された．自動定理証明を研究していたKowalskiが参加し，エジンバラに持ち帰って一階述語論理に合わせて書法を修正し，導出原理(Robinson, 1965)と関連付けしたものが，現在の標準的なPrologである(Colmerauer & Roussel, 1996)．プログラミング言語としての国際標準はISO/DEC-10 Prologである．Lispの影響を残した処理系もいくつかあったが， PrologをC言語などで実装できるように仮想マシンとして記述したWarren仮想マシン(Warren Abstract Machine; WAM)が公開された後は，標準的な記法が普及した．後にWarrenはSLD導出[55]にtabling機能を追加して計算を効率化し，多くの処理系がこれを取り入れた(Colmerauer & Roussel, 1996; Swift & Warren, 2012)．SLD導出の意味論はLoyd(1987)にまとめられている．

ISO標準準拠の商用Prolog処理系としては，SICStus， Amzi!， Visual， Strawberry などがある．GNU-PrologはISO標準準拠だが，(Windows版では)日本語文字を扱いにくい．日本語文字を扱えて現時点で利用可能な処理系として，SWIの他，AZ-Prologは，Prolog-KABAのコードが動作すると謳われている．SWISHやJekejeke-Prologはブラウザ上から実行できるサービスとして提供されている．B-PrologはUTF-8に対応し，制約処理に強みがある．ちなみに，B-Prologをベースに関数プログラミングとアクターモデル(並列処理)などを融合した言語にPicatがある．

なお本付録と本文8節で採用している組込述語term_string/2はISO標準ではない．代替的方法は，プログラムをソースコードとしてファイルに書き出し再度consultすることが考えられる．ファイル書き出しのためのtell/1とtold/0もISO標準ではない(GNU-Prologにはある)．ISO標準でのファイル書き出しは，ストリームを指定する必要があり，open/3とclose/1を組み合わせることによって行う．詳しくは処理系のヘルプやマニュアルを参照されたい．なお，述語の性質は predicate_property/2(ISO標準)を用いて調べることができる．

以下はDahl (1990)の簡潔な説明に従って，Prologの構文を紹介する．Prologの表現の基本要素は項(term)である．項は，変数，定数，または $f(t_1, \cdots, t_k)$ の形式である．fはk引数のファンクターであり，各t_iは項である．1階述語論理に即して，論理的な意味を持つ基本要素は述語(predicate)と呼ばれ，$P(t_1, \cdots, t_n)$の形式を持つ．ここでPはn個の引数(アリティ)をもつ述語の記号，t_i は項である．P/nはアリティnの述語の意味である．リテラルは否定を前置できる述語で，自由変数のないものを言う．節はリテラルの選言(OR結合)であり，確定節は肯定リテラルが一つからなる節である．事実とルールは確定節である．事実は否定リテラルを持たない．ゴール節は否定リテラルのみからなる．事実、ルール、ゴールを総称してホーン節と呼ぶ．

事実は一つの述語Pの形式であり，最後にピリオドがつく．述語Pの中に現れる変数に一貫した方式で

項の組を代入したとき，その述語Pが真になれば事実の真の事例(instance)が得られる．ルールは P :- P_1, …, P_n.のように述語を並べた形式で，最後にピリオドをつける．ルールの頭部P は，本体の述語の系列 P_1, …, P_n.の中に現れる変数に一貫した方式で項の組を代入したとき，本体の述語がすべて真になるとき，Pも真になり，各述語に対応する事実の真の事例の列が得られる．ルールは，論理式としては P ∨ ¬ P_1∨ …∨ ¬P_n である．記号∨は「または」の意味（選言，OR結合）で，&を「かつ」（連言，AND結合）とすると，P ← P_1 ， … , P_n と論理的等価である．つまり事実は頭部のみで本体がないルールであり，その論理的意味はモデル化対象の世界において，述語変数にどのような項を代入しても，真の事例(instance)が得られるという主張(assertion)である．ゴール（質問）は， ？－ Q_1, …, Q_n. という形式で，論理式は¬（¬ Q_1∨ …∨ ¬Q_n ）＝Q_1 & … & Q_n, すなわち本体のみで頭部がないルール（の否定）としての意味をもつ．直観的には「各述語Q_i の変数に代入すると，対象世界で真の事例になる項の組を見つけよ（そうすることによってゴールの否定の矛盾が示される）」という意味になる．

付録B

本付録はPrologの組込述語の抜粋を，日本語で利用できるようにするアプリケーションを提示する．動作の概要としては，以下の述語名ペアの対応表をCSVファイルから読み込んで，日本語組込述語の設定を行う．

1，ファイル書出しを開始します/1, tell/1
2，ファイル書出しを終了します/0, told/0
3，アトムの文字コードリスト/2, atom_codes/2
4，文字のコード/2, cha_code/2
5，区間内の数/3, between/3
6，アトム的リストの結合/3, atomic_list_concat/3
7，改行します/0, nl/0
8，書き出す/1, write/1
9，強制的に失敗します/0, fail/0
10，ここから先は失敗すると後戻りすることはできません/0, !/0
11，ユニブ/2, (=..)/2
12，節を参照する/2, clause/2
13，ゴール収集袋/3, bagof/3
14，成功事例の収集/3, bagof/3
15，成功事例すべて収集/3, findall/3
16，成功事例をソートして収集/3, setof/3
17，リストのソート/2, sort/2
18，リストの要素/2, member/2
19，リストの結合/3, member/3
20，部分文字列/5, sub_atom/5
21，節の登録/1, assert/1
22，節の削除/1, retract/1

23，節の廃止リスト/1, abolish/1
24，プログラムの読み込み/1, consult/1
25，または/2, (;)/2

上述の内容のカンマ区切りのテキストデータを，カレントディレクトリ下に'組込述語名読替辞書.csv'というファイル名で保存する．各行の先頭は行番号，2要素目は日本語述語名，3要素目は日本語述語名に対応するシステムの組込述語である．もちろん両者のアリティは一致していなければならない．

日本語述語名を設定するプログラムの概要を以下に示す．

```
%%%%%%%%%% ここから プログラム %%%%%%%%%%
述語名対応表をファイルから読み込む( J, F / N, G / N ):-
        File = '組込述語名読替辞書.csv',
        csv_read_file( File, Data, [ ] ),
        nth1( J, Data, row( J, X, Y ) ),
        term_string( F / N, X ),
        term_string( G / N, Y ).

組込述語の読替え( J, F, G, N, L, P, Q ):-
        述語名対応表をファイルから読み込む( J, F / N, G / N ),
        length( L, N ),
        P =.. [ F | L ],
        Q =.. [ G | L ].

日本語組込述語の設定( J, N, P, Q ):-
        組込述語の読替え( J, _ , _, N, _, P, Q ),
        ¥+ clause( P, Q ),
        assert( ( P :- Q ) ).

日本語組込述語の初期化:-
        組込述語の読替え( _, _, _, _, _, P, Q ),
        clause( P, Q ),
        retract( ( P :- Q ) ),
        fail
        ;
        true.

日本語組込述語の設定:-
        日本語組込述語の初期化,
        fail.
```

```
日本語組込述語の設定:-
        日本語組込述語の設定( _, _, _, _ ),
        fail
        ;
        true.

:- 日本語組込述語の設定.
%%%%%%%%%% プログラム ここまで %%%%%%%%%%
```

上記のプログラムを例えば’jpx.pl’として保存し，システムに読み込む．すると，組込述語 member/2の代わりに，18行目の「リストの要素/2」を用いて，以下のような日本語の質問を実行できるようになる．

```
(q.98) ?- リストの要素( _リストの要素, [ リンゴ, バナナ, みかん ] ).
_リストの要素 = リンゴ ;
_リストの要素 = バナナ ;
_リストの要素 = みかん.
```

もちろん，変数名は元のように半角英数大文字で開始してもよい．

```
(q.99) ?- リストの要素( X, [ リンゴ, バナナ, みかん ] ).
X = リンゴ ;
X = バナナ ;
X = みかん.
```

また，ファンクターをユーザ定義し，独自書式の言語に仕立てることもできよう．

```
:- op( 500, yfx, jpx ).

_和述語 jpx L :-
        組込述語の読替え( _, _和述語/_, _, _, _組込述語, L ),
        call( _組込述語 ).

(q.100) ?- リストの要素 jpx [ X, [ リンゴ, バナナ, みかん ] ].
X = リンゴ ;
X = バナナ ;
X = みかん ;
false.
```

述語名の対応などは，「組込述語の読替え/7」で確認することができる．

(q.101) ?- 組込述語の読替え(18, _, _, B, _, C, D).
B = 2,
C = リストの要素(_24640, _24646),
D = member(_24640, _24646).

逆に, この「組込述語の読替え」を用いて, Prologプログラムを日本語で説明することができる. 例えば4.3節の「アルファベットの小文字とその文字コードをファイルに保存/0」のルール本体の各述語を, 日本語で説明してみよう.

準備として, ルールの本体中の述語を取り出すツールを用意する.

```
述語リストとゴール結合の相互変換( [ A ], A ):-
        ¥+ A =.. [ ',' | _ ],
        ¥+ A =.. [ ';' | _ ].

述語リストとゴール結合の相互変換( [ X | L ], ( X, Y ) ):-
        述語リストとゴール結合の相互変換( L, Y ).

述語リストとゴール結合の相互変換( L , ( A ; B ) ):-
        述語リストとゴール結合の相互変換( L1 , A ),
        述語リストとゴール結合の相互変換( L2 , B ),
        append( L1, [ ';' | L2 ], L ).
```

以上の準備の下, まず次の「」を用いて, 元のプログラム内の組込述語を列挙しよう.

```
'ルール本体のK番目の述語'( _ルール名, K, _述語 ):-
        clause( _ルール名, _ルール本体 ),
        述語リストとゴール結合の相互変換( _リスト, _ルール本体 ),
        nth1( K, _リスト, _述語 ).
```

(q.102) ?- 'ルール本体のK番目の述語'(アルファベットの小文字とその文字コードをファイルに保存, J, _述語), nl, write(J; _述語), fail.

```
1;tell(alphabet.txt)
2;atom_codes(az,[_38972,_38978])
3;between(_38972,_38978,_38994)
4;char_code(_38998,_38994)
5;atomic_list_concat([_38998,_38994],,,_39020)
6;nl
7;write(_39020)
8;fail
```

```
9;(;)
10;told
false.
```

「ルール内の組込述語の説明/1」を用いて，日本語組込述語としてルール本体を完成させる.

```
ルール内の組込述語の説明( _ルール名 ):-
        'ルール本体のK番目の述語'( _ルール名, J, _述語 ),
        組込述語の読替え( _, _, _, _, _, _日本語述語, _述語 ),
        nl,
        write( [ J ] ),
%       numbervars( _日本語述語, 0, _ ),
        write( _日本語述語 ),
        fail
        ;
        true.
```

(q.103) ?- ルール内の組込述語の説明(アルファベットの小文字とその文字コードをファイルに保存).

```
[1]ファイル書出しを開始します(alphabet.txt)
[2]アトムの文字コードリスト(az,[_38600,_38606])
[3]区間内の数(_38600,_38606,_38622)
[4]文字のコード(_38626,_38622)
[5]アトム的リストの結合([_38626,_38622],,,_38648)
[6]改行します
[7]書き出す(_38648)
[8]強制的に失敗します
[9]または(前,後)
[10]ファイル書出しを終了します
true.
```

最後に，対応リスト内の日本語化した組込述語を一覧しよう.

(q.104) ?- 組込述語の読替え(J, _, P, Q), current_predicate(X, Q), numbervars(P, 0, N), nl, write([J : X/N] : P :- Q), fail.

```
[1:tell/1]:ファイル書出しを開始します(A):-tell(A)
[2:told/0]:ファイル書出しを終了します:-told
[3:atom_codes/2]:アトムの文字コードリスト(A,B):-atom_codes(A,B)
[4:char_code/2]:文字のコード(A,B):-char_code(A,B)
```

[5:between/3]:区間内の数(A,B,C):-between(A,B,C)
[6:atomic_list_concat/3]:アトム的リストの結合(A,B,C):-atomic_list_concat(A,B,C)
[7:nl/0]:改行します:-nl
[8:write/1]:書き出す(A):-write(A)
[9:fail/0]:強制的に失敗します:-fail
[10:!/0]:ここから先は失敗すると後戻りすることはできません:-!
[11:(=..)/2]:ユニブ(A,B):-A=..B
[12:clause/2]:節を参照する(A,B):-clause(A,B)
[13:bagof/3]:ゴール収集袋(A,B,C):-bagof(A,B,C)
[14:bagof/3]:成功事例の収集(A,B,C):-bagof(A,B,C)
[15:findall/3]:成功事例すべて収集(A,B,C):-findall(A,B,C)
[16:setof/3]:成功事例をソートして収集(A,B,C):-setof(A,B,C)
[17:sort/2]:リストのソート(A,B):-sort(A,B)
[18:member/2]:リストの要素(A,B):-member(A,B)
[20:sub_atom/5]:部分文字列(A,B,C,D,E):-sub_atom(A,B,C,D,E)
[21:assert/1]:節の登録(A):-assert(A)
[22:retract/1]:節の削除(A):-retract(A)
[23:abolish/1]:節の廃止リスト(A):-abolish(A)
[24:consult/1]:プログラムの読み込み(A):-consult(A)
[25:(;)/2]:または(A,B):-A;B
false.

改良として，変数名を含む書き換え対応表を用いて，変数名についても日本語で理解可能なソースコードを出力することが考えられる．例えば，以下のようにルールを書き出す．

```
ファイル書出しを開始します( _ファイル名 ):- tell( _ファイル名 ).
文字のコード( _文字, _文字コード ):- char_code( _文字, _文字コード ).
```

これは読者の練習問題とする．

付録C

本付録は，本文の8節で論じられた「相続税の計算」についてのサンプル文書中の数式を含む行を抽出し，疑似的なルール形式に翻訳するために用いるプログラムを抜粋して紹介する．

前提として，サンプル文書全体を，述語名「サンプル文書/1」の引数に，’(ここにサンプル文書全体)’のように引用符で囲まれた文字列データとして格納する．

ちなみに，当該URLの文書は，次のコードを実行してブラウザで表示させることができる．

```
(q.105) ?- URI = 'https://www.nta.go.jp/taxes/shiraberu/taxanswer/sozoku/4152.htm', win_shell( open, URI ).
```

なお，上記文書に含まれる図としての数式表示は，以下のように特殊鍵括弧付きの文字列とした．

(q.106) ?- サンプル文書(X), nth1(J, X, A), ¥+ ¥+ sub_atom(A, _, _, _, ″〔″), nl, write(J : A), fail.

36:〔各相続人等の税額〕 ＋ 〔相続税額の2割加算〕 － 〔暦年課税分の贈与税額控除〕 － 〔配偶者の税額軽減〕 － 〔未成年者控除〕 － 〔障害者控除〕 － 〔相次相続控除〕 － 〔外国税額控除〕 ＝ 〔各相続人の控除後の税額〕(赤字の場合は0になります)
37:〔各相続等の控除後の税額〕 － 〔相続時精算課税分の贈与税相当額(外国税額控除前の税額)〕 － 〔医療法人持分税額控除〕 ＝ 〔各相続人等の納付すべき税額(※)〕
40:〔赤字となった金額(マイナスは付けません)〕 － 〔相続時精算課税分の贈与税の計算をする際、控除した外国税額〕 ＝ 〔還付を受けることができる金額〕
false.

また文書の内容によっては，特殊な文字を”¥”を前置することでエスケープしておく必要がある(上述のデータ内には該当する文字は含まれていない)．

サンプル文書内の数式行を項として再表現する

ここでは本文の実行例[67]において用いられた述語「サンプル文書内の数式行を項として再表現する/2」の全コードを説明文書付きで紹介する．

まず，以下の「サンプル文書内のn行目/2」では，サンプル文書を行に分割し，n行目を取り出す．その際，タブ，それゆえまた空行も削る．また「サンプル文書内の数式行/2」では，全角等号に着目して数式行を抽出する．

```
サンプル文書内のn行目( _行番号, _行 ):-
        サンプル文書( _文書全体 ),
        _区切り = '¥n¥t',
        split_string( _文書全体, _区切り, _区切り, _行のリスト ),
        nth1( _行番号, _行のリスト, _行 ),

サンプル文書内の数式行( _行番号, _行 ):-
        サンプル文書内のn行目( _行番号, _行 ),
        sub_atom( _行, _, _, _, ″＝″ ).
```

次に3ステップで「サンプル文書内の数式行/2」で取り出したサンプル文書内の数式行を項に翻訳する．まず最初のステップでは以下の「文字列から算術演算への変換/2」を用いて，通貨の単位「万円」は演算子定義し，カッコ内の数式や注釈を引数とするファンクターとして認識させる．便宜上，半角数値内のカンマ，読点「、」については，後の処理でのエラー対応を簡単にするため削除しておく．

```
文字列から算術演算への変換( _文字列, _算術 ):-
```

全角算術演算記号のリスト(D),
prologの算術演算記号(E),
文字列をリスト内の語で置換する(_文字列, D, E, S0),
全角括弧を半角に変える(S0, S1),
カンマを削除する(S1, S2),
読点の除去(S2, _算術).

「文字列から算術演算への変換/2」の実行例を示す.

(q.107) ?- _行番号 = 30, サンプル文書内の数式行(_行番号, _行), 文字列から算術演算への変換(_行, _算術).
_行番号 = 30,
_行 = ″相続税の総額 × 各人の課税価格 ÷ 課税価格の合計額 ＝ 各相続人等の税額″,
_算術 = ’相続税の総額 * 各人の課税価格 / 課税価格の合計額 =:= 各相続人等の税額’.

続く2ステップでは「数式表現を疑似コードに変換する/2」および「数式表現を項に変換する/2」を用いる. なお通貨単位「万円」に便宜的なファンクターを定義しておく.

:- op(200, yf, 万円).

数式表現を疑似コードに変換する(_数式を含む行, _疑似コード):-
特別の区切り文字を削除する(_数式を含む行, B),
文字列から算術演算への変換(B, _疑似コード).

数式表現を項に変換する(_数式を含む行, _項):-
数式表現を疑似コードに変換する(_数式を含む行, _疑似コード),
項と文字列の変換(_項, _疑似コード).

それぞれの実行例を示す.

(q.108) ?- _算術 = ’ 課税価格の合計額 - 基礎控除額(3000万円 + 600万円 * 法定相続人の数)’, 数式表現を疑似コードに変換する(_算術, _疑似コード), 数式表現を項に変換する(_算術, _項)

_算術 = ’ 課税価格の合計額 - 基礎控除額(3000万円 + 600万円 * 法定相続人の数) ’,
_疑似コード = ’課税価格の合計額-基礎控除額(3000万円+600万円*法定相続人の数) ’,
_項 = (課税価格の合計額-基礎控除額(3000 万円+600 万円*法定相続人の数)).

上述の各ルール本体に使われている諸条件については, 後述する.「サンプル文書内の数式行を項として再表現する/3」を用いて, 以上の3ルールを用いて, サンプル文書内の数式行を項に翻訳する. またこれによって本文中でデモンストレーションを再現することができる.

```
サンプル文書内の数式行を項として再表現する( _行番号, _行, _項 ):-
        サンプル文書内の数式行( _行番号, _行 ),
        文字列から算術演算への変換( _行, _行1 ),
        数式表現を項に変換する( _行1, _項 ).

上述のプログラムで用いられている各種ツールを以下に掲載する. 詳しい説明は省く.

全角算術演算記号のリスト( D ):-
        D = [ "＋", "×", "－", "÷", "＝" ].

prologの算術演算記号( E ):-
        E = [ "+", "*", "-", "/", "=:=" ].

全角の括弧( D ):-
        D = [ "（", "）", "｛",  "｝" ].

半角の括弧( E ):-
        E = [ "(", ")", "(", ")" ].

特別の区切り文字( [ _半角空白, _全角空白, _特殊鍵括弧開, _特殊鍵括弧閉 ] ):-
        _半角空白 = "¥s",
        char_code( _全角空白, 12288 ),
        % _特殊鍵括弧開 = "〔",
        char_code( _特殊鍵括弧開, 12308 ),
        % _特殊鍵括弧閉 = "〕",
        char_code( _特殊鍵括弧閉, 12309 ).

特別の区切り文字を削除する( A, B ):-
        特別の区切り文字( D ),
        atomic_list_concat( D, F ),
        split_string( A, F, F, P ),
        atomic_list_concat( P, B ).

語による文字列の分割( A, B, A1, A2 ):-
        sub_atom( A, I, _, K, B ),
        sub_atom( A, 0, I, _, A1 ),
        sub_atom( A, _, K, 0, A2 ).

語による文字列の分割と置換( S, W, V, [ S1, V | L ] ):-
```

```
        語による文字列の分割( S, W, S1, S2 ),
        !,
        語による文字列の分割と置換( S2, W, V, L ).

語による文字列の分割と置換( S, _, _, [ S ] ).

語の置換( S, W, V, T ):-
        語による文字列の分割と置換( S, W, V, L ),
        atomic_list_concat( L, T ).

文字列をリスト内の語で置換する( A, [ ], [ ], A ).

文字列をリスト内の語で置換する( A, [ B | Y ], [ C | W ], F ):-
        語の置換( A, B, C, D ),
        文字列をリスト内の語で置換する( D, Y, W, F ).

半角カンマの除去( S, T ):-
        文字列をリスト内の語で置換する( S, [ ',' ], [ '' ], T ).

読点の除去( S, T ):-
        文字列をリスト内の語で置換する( S, [ '、' ], [ '' ], T ).

全角括弧を半角に変える( S, T ):-
        全角の括弧( D ),
        半角の括弧( E ),
        文字列をリスト内の語で置換する( S, D, E, T ).

項と文字列の変換( T, S ):-
        term_string( T, S ).
```

以上のプログラムをファイルとして保存し，SWI-Prologに読み込むことで，本文中の「「サンプル文書内の数式行を項として再表現する/2」のデモンストレーション結果を再現することができる．以下では本文中に示した項を，write_canonical/1で前置演算子の形に清書する．

```
(q.109) ?- サンプル文書内の数式行を項として再表現する( J, X, Y ), nl, write( [J] ), tab(1),
write_canonical( Y ), fail.
```

[6] =:=(-(+(-(+(相続または遺贈により取得した財産の総額,みなし相続などにより取得した財産の価額),非課税財産の価額),相続時精算課税に係る贈与財産の価額(注1)),債務及び葬式費用の額),純資産価額(赤字のときは0))

[7] =:=(+(純資産価額,相続開始前3年以内の贈与財産の価額(注2)),各人の課税価格(千円未満切捨て))
[14] =:=(各相続人の課税価格の合計,課税価格の合計額)
[16] =:=(-(課税価格の合計額,基礎控除額(+(万円(3000),*(万円(600),法定相続人の数)))),課税遺産総額)
[23] =:=(*(課税遺産総額,各法定相続人の法定相続分),法定相続分に応ずる各法定相続人の取得金額(千円未満切り捨て))
[25] =:=(*(法定相続分に応ずる各法定相続人の取得金額,税率),算出税額)
[27] =:=(各法定相続人ごとの算出税額の合計,相続税の総額)
[30] =:=(/(*(相続税の総額,各人の課税価格),課税価格の合計額),各相続人等の税額)
[36] =:=(-(-(-(-(-(-(+(各相続人等の税額,相続税額の2割加算),暦年課税分の贈与税額控除),配偶者の税額軽減),未成年者控除),障害者控除),相次相続控除),外国税額控除),各相続人の控除後の税額(赤字の場合は0になります))
[37] =:=(-(-(各相続等の控除後の税額,相続時精算課税分の贈与税相当額(外国税額控除前の税額)),医療法人持分税額控除),各相続人等の納付すべき税額(※))
[40] =:=(-(赤字となった金額(マイナスは付けません),相続時精算課税分の贈与税の計算をする際控除した外国税額),還付を受けることができる金額)
false.

こうして得られた数式表現に対応するPrologの項を，以下の「演算を分解する/2,3」を用いて，項目分解する.

```
演算を分解する( A, D, [ A | D ] ).

演算を分解する( A, L, D ):-
        A =.. [ F | P ],
        nth1( J, P, B ),
        演算を分解する( B, [ ( F, J ) | L ], D ).

演算を分解する( A, D ):-
        演算を分解する( A, [ ], D ).
```

「演算を分解する/2,3」はユニブを再帰的に用いて，ファンクターとその引数位置を組で数式中の位置を識別する. 実際に，40行目の数式を分解してみる.

(q.110) ?- サンプル文書内の数式行を項として再表現する(40, _, A), 演算を分解する(A, D).

A = (赤字となった金額(マイナスは付けません)-相続時精算課税分の贈与税の計算をする際、控除した外国税額=:=還付を受けることができる金額),
D = [赤字となった金額(マイナスは付けません)-相続時精算課税分の贈与税の計算をする際、控除した外国税額=:=還付を受けることができる金額]

上述の質問結果は，2個の成功したゴールを出力している．変数Aは元の文章の16行目のPrologの項への変換，Dはその項目と識別位置である．最初に成功した代入組（A，D）のDは数式全体を唯一の要素とするリストである．ここで，バックトラックさせるためセミコロン;を入力すると，以下の出力を得る．

A = （赤字となった金額(マイナスは付けません)−相続時精算課税分の贈与税の計算をする際、控除した外国税額=:=還付を受けることができる金額),
D = [赤字となった金額(マイナスは付けません)−相続時精算課税分の贈与税の計算をする際、控除した外国税額，((=:=), 1)] .

バックトラック後に見つかった2番目の代入組のDは，数式内の項目とその識別位置からなるリストで，識別位置((=:=), 1)は，その項目が数式の双方向評価=:=/2の左辺であることを示す．

次に16行目の数式の全項目を分解してみよう．

(q.111) ?− サンプル文書内の数式行を項として再表現する(16, _, A)，prologの算術演算記号(E)，演算を分解する(A, [F | X]), X ¥= [ ], nl, write(F), tab(1), write(path: X), fail.

課税価格の合計額−基礎控除額(3000 万円+600 万円*法定相続人の数) path:[((=:=),1)]
課税価格の合計額 path:[((−),1),((=:=),1)]
基礎控除額(3000 万円+600 万円*法定相続人の数) path:[((−),2),((=:=),1)]
3000 万円+600 万円*法定相続人の数 path:[(基礎控除額,1),((−),2),((=:=),1)]
3000 万円 path:[((+),1),(基礎控除額,1),((−),2),((=:=),1)]
3000 path:[((万円),1),((+),1),(基礎控除額,1),((−),2),((=:=),1)]
600 万円*法定相続人の数 path:[((+),2),(基礎控除額,1),((−),2),((=:=),1)]
600 万円 path:[((*),1),((+),2),(基礎控除額,1),((−),2),((=:=),1)]
600 path:[((万円),1),((*),1),((+),2),(基礎控除額,1),((−),2),((=:=),1)]
法定相続人の数 path:[((*),2),((+),2),(基礎控除額,1),((−),2),((=:=),1)]
課税遺産総額 path:[((=:=),2)]
false.

上記出力の全11行のうち，4行目から10行目までのデータは全て識別位置（path）内に「基礎控除額」を含んでおり，そのファンクター内の引数の一部であることを意味する．3行目の先頭に，ファンクター「基礎控除額」が括弧内の引数「3000 万円+600 万円*法定相続人の数」付きで出力されている．数式内の識別位置はpath:「((−),2),((=:=),1)」と表示されている．リストの第2要素((=:=),1)は数式全体の中の等号の左辺，また第1要素((−),2)はその左辺の減算の右側の項の位置に，「基礎控除額」が位置することを意味する．

次のルールは，数式行内のファンクター（括弧内に引数）を取り出す．

算術演算子(P):−
　　prologの算術演算記号(E),

```
        member( Y, E ),
        term_string( P, Y ).

直前が算術演算でなくかつそれ以前のパス内に算術演算しかない( A, F, P, K, X ):-
        演算を分解する( A, [ F, ( P, K ) | X ] ),
        ¥+  算術演算子( P ),
        ¥+ ( member( ( Q, _ ), X ), ¥+  算術演算子( Q ) ).

数式内の注釈やファンクター( J, F, P, K, X ):-
        サンプル文書内の数式行を項として再表現する( J, _, A ),
        直前が算術演算でなくかつそれ以前のパス内に算術演算しかない( A, F, P, K, X ).
```

「数式内の注釈やファンクター/5」の説明は本体の条件から自ずと推察されよう．次のルールでそれらを列挙する．

```
数式内の注釈やファンクター:-
        数式内の注釈やファンクター( J, F, P, K, X ),
        write( [ J ] : F : ( P, K ) : X ),
        nl,
        fail
        ;
        true.

(q.112) ?-  数式内の注釈やファンクター.

[6]:注1:(相続時精算課税に係る贈与財産の価額,1):[((+),2),((-),1),((=:=),1)]
[6]:赤字のときは0:(純資産価額,1):[((=:=),2)]
[7]:注2:(相続開始前3年以内の贈与財産の価額,1):[((+),2),((=:=),1)]
[7]:千円未満切捨て:(各人の課税価格,1):[((=:=),2)]
[16]:3000  万円+600  万円*法定相続人の数:(基礎控除額,1):[((-),2),((=:=),1)]
[23]:千円未満切り捨て:(法定相続分に応ずる各法定相続人の取得金額,1):[((=:=),2)]
[36]:赤字の場合は0になります:(各相続人の控除後の税額,1):[((=:=),2)]
[37]:外国税額控除前の税額:(相続時精算課税分の贈与税相当額,1):[((-),2),((-),1),((=:=),1)]
[37]:※:(各相続人等の納付すべき税額,1):[((=:=),2)]
[40]:マイナスは付けません:(赤字となった金額,1):[((-),1),((=:=),1)]
true.
```

16行目の数式内で，最初に成功したゴールが，括弧内の数式「3000 万円+600 万円*法定相続人の数」(変数F)を引数として持つファンクター名が「基礎控除額」(変数P)である．ファンクターの位置Xについては前述の通りである．

上述のファンクターをルールとして独立させ，また各項目の先頭に半角下線を付与してatom_concatし，term_stringで項に変換する．詳しい説明は省略するが，プログラムとその結果を示す．

```
リスト内の各要素を変数化する( _リスト, _結果 ):-
        findall( _変数名, (
                member( _要素, _リスト ),
                カンマを削除する( _要素, _要素1 ),
                atom_concat( '_', _要素1, _変数名 )
        ), _結果 ).

ファンクターからルール頭部を作る( _ファンクター名, _頭部 ):-
        atom_concat( '_', _ファンクター名, _引数名 ),
        _頭部 =.. [ _ファンクター名, _引数名 ].

ファンクター引数からルール本体を作る( _算術, _本体, _変数 ):-
        prologの算術演算記号( E ),
        atomic_list_concat( E, D ),
        term_string( _算術, F ),
        split_string( F, D, D, P ),
        リスト内の各要素を変数化する( P, _変数 ),
        文字列をリスト内の語で置換する( F, P, _変数, _本体 ).

数式内の注釈やファンクターをルール化する( J, _ルール, _変数 ):-
        数式内の注釈やファンクター( J, _算術, _ファンクター, _, _ ),
        ファンクターからルール頭部を作る( _ファンクター, _頭部 ),
        ファンクター引数からルール本体を作る( _算術, _本体, _変数 ),
        _ルール = ( _頭部 :- _本体 ).
```

以下の実行例は，16行目のカッコ内の数式を独立させたルールを作成する．

```
(q.113) ?- 数式内の注釈やファンクターをルール化する( 16, _ルール, _変数 ).
_ルール = (基礎控除額('_基礎控除額'):-'_3000 万円+_600 万円*_法定相続人の数'),
_変数 = ['_3000 万円', '_600 万円', '_法定相続人の数'] .
```

注釈行についても機械的に実行するため，意味的に理解しにくい行もあることをお詫びする．

参考文献

[1] 有本卓(1980).『確率・情報・エントロピー』森北出版.

[2] Arrow, K. J. (1963). Social choice and individual values (2nd ed.). New York, NY: John Wiley & Sons.

Retrieved from https://cowles.yale.edu/sites/default/files/files/pub/mon/m12-2-all.pdf（長名寛明訳『社会的選択と個人的評価』東京：日本経済新聞社, 1977）
[3] Aumann, R. J. (1995). Backward induction and common knowledge of rationality. *Games and Economic Behavior,* 8, 6-19. doi:10.1016/S0899-8256(05)80015-6
[4] Blackwell, D. (1953). Equivalent comparisons of experiments. The Annals of Mathematical Statistics, 24, 265-272. Retrieved from https://www.jstor.org/stable/2236332
[5] 馬場祐人・筧 捷彦(2011). 日本語で一貫したプログラミングの実践:プロデルを用いて. 研究報告ソフトウェア工学(SE) 17, 1-9. Retrieved from http://id.nii.ac.jp/1001/00074113/
[6] Basu, K. (1994). The traveler's dilemma: Paradoxes of rationality in game theory. *The American Economic Review,* 84, 391-395. https://www.jstor.org/stable/2117865
[7] Bicchieri, C. (1989). Strategic behavior and counterfactuals. Synthese, 76, 135-169. doi:10.1007/BF00869644
[8] Blackburn, P., Bos, J., & Striegnitz, K. (2003). Learn Prolog now! (Version 1.2.5). Retrieved from http://www.academia.edu/download/31015322/pro.pdf
[9] Bond, F., Isahara, H., Fujita, S., Uchimoto, K., Kuribayashi, T., & Kanzaki, K. (2009). Enhancing the Japanese WordNet. In The 7th Workshop on asian language resources, in conjunction with ACL-IJCNLP 2009, Singapore, pp.1-8. doi:10.5555/1690299.1690300
[10] Bratko, I. (2012). Prolog programing for artificial intelligence. Addison-Wesley（『Prologへの入門(PrologとAI)』[第2版の翻訳]近代科学社,1994）
[11] Brooks, R. A. (1991). Intelligence without representation, *Artificial Intelligence* 47, 139-159. doi:10.1016/0004-3702(91)90053-M
[12] Clark, K.L., & McCabe, F.G. (1984). Micro-Prolog: programing in logic. Prentice Hall.(山田眞一訳『人工知能のためのmicro-Prologプログラムコレクション』東京：サイエンス社, 1986）
[13] Clocksin, W. (1997). Clause and effect: Prolog programing for the working programmer. Berlin: Springer-Verlag.
[14] Colmerauer, A., & Roussel, P. (1996). The birth of Prolog. In Bergin, T. J., and Gibson, R. G. (eds.), History of programming languages II. New York, NY: ACM, pp.331-367. doi:10.1145/234286.1057820
[15] Croskerry, P., Cosby, K.S., Graber, M.L., & Singh, H. (2017). Diagnosis: Interpreting the shadows.（綿貫聡・徳田安春監訳『「誤診」はなくせるのかー実践知としての診断エラー学の世界』東京：医学書院, 2019）
[16] Dahl, V.(1990). 構文解析器のカジュアルな記述者からの複雑性の隠蔽. In Dahl, V. & Saint-Dizier, P.(eds.) Natural language understanding and logic programing, Elsevier, 1985.（西田豊明・松本裕治・上原邦明(訳),『自然言語理解と論理プログラミング』 東京：近代科学社, 1990, pp. 1-24）
[17]太細孝・鈴木克志・伊草ひとみ・佐藤裕幸(1984).『Prolog入門』 東京：啓学出版.
[18] Deransart, P., Ed-Dbali, A., & Cervoni, L. (1996). Prolog: The standard reference manual. Berlin: Springer-Verlag.
[19] Evans, J. St. B. T., & Over, D. E. (2004). If. Oxford, UK: Oxford University Press.
[20] Floridi, L. (2019). Translating principles into practices of digital ethics: Five risks of being unethical. Philosophy & Technology, 32, 185-193. doi:10.1007/s13347-019-00354-x

[21] Frey, C. B., & Osborne, M. (2013). The future of employment, University of Oxford. Retrieved from http://sep4u.gr/wp-content/uploads/The_Future_of_Employment_ox_2013.pdf

[22] Gaertner, W. (2001). Domain conditions in social choice theory. Cambridge, UK: Cambridge University Press.

[23] Gärdenfors, P. (1986). Belief revisions and the Ramsey test for conditionals. *The Philosophical Review,* 95, 81-93. doi:10.2307/2185133

[24] Halliday, M. A. K., & Matthiessen, C. M. (2013). Halliday's introduction to functional grammar. New York, NY: Routledge.

[25] Halpern, J. Y., & Pearl, J. (2005). Causes and explanations: A structural-model approach. Part I: Causes. *The British Journal for the Philosophy of Science,* 56, 843-887. doi:10.1093/bjps/axi147

[26] Halpern, J. Y., & Moses, Y. (1986). Taken by surprise: the paradox of the surprise test revisited. *Journal of Philosophical Logic*, 15, 281-304. doi:10.1007/BF00248573

[27] Hatfield, J.W., Kominers, S.D., Nichifor, A., Ostrovsky, M. and Westkamp, A. (2019), Full substitutability. *Theoretical Economics*, 14: 1535-1590. doi:10.3982/TE3240

[28] 林周二(1973).『統計学講義第2版』東京:丸善.

[29] Hooker, J. (2000). Logic-based methods for optimization: Combining optimization and constraint satisfaction. New York, NY: John-Wiley.

[30] Horgan, T. (1981). Counterfactuals and Newcomb's problem. *The Journal of Philosophy,* 78, 331-356. doi: 10.2307/2026128

[31] Hurwicz, L. (1973). The design of mechanisms for resource allocation. *The American Economic Review*, 63, 1-30. https://www.jstor.org/stable/1817047

[32] Indo, K. Proving Arrow's theorem by Prolog. *Computational Economics* 30, 57-63. SCRIPT Press. doi:10.1007/s10614-007-9086-2

[33] Indo, K. (2009). Modeling a small agent society based on social choice logic programing. In Terano, T., Kita, H., & Takahashi, S. (eds.), Agent-based approaches in economic and social complex systems V (pp. 93-104). Tokyo: Springer-Verlag. doi:10.1007/978-4-431-87435-5_8

[34] Indo, K. (2014). Parallel possibility results of preference aggregation and strategy-proofness by using Prolog. In Proceedings of the 6th International Conference on Agents and Artificial Intelligence - Volume 2: ICAART (pp. 243-248). SciTePress. doi:10.5220/0004913302430248

[35] 犬童健良(2016). 論理と確率の推論を「心の中のゲーム」としてモデル化する. 認知科学 23, 153－169. doi:10.11225/jcss.23.153

[36] 犬童健良(2018, January 22). 日本語でPrologプログラミング. Retrieved from https://xkindo.net/cog_dec/wp/jl2p.pdf

[37] 犬童健良(2018). 日本語文の語彙的ネットワークの安定性.日本認知科学会第35回大会ポスター発表.

[38] 犬童健良・松田紀之・太田究三郎(1989) 日本語の「は」と「が」の使い分け:フレーム理論による形式化. *Institute of socio-economic planning discussion paper series* 406, University of Tsukuba.

[39] 井上和子(1989).『日本文法小辞典』東京:大修館書店.

[40] Jacobs, B. (2019). The mathematics of changing one's mind, via Jeffrey's or via Pearl's update rule. *Journal of Artificial Intelligence Research* 65, 783-806. doi:10.1613/jair.1.11349

[41] 木村明・片桐明(1988). 日本語プログラミング言語『Mind』についてその概要と、日本語プログラミングの実用性. 情報処理学会研究報告プログラミング(PRO), 27, 25-32. Retrieved from http://id.nii.ac.jp/1001/00031091/
[42] 小谷善行(1993). 日本語に基づく論理プログラム表現, 情報処理学会論文誌, 34, 973-984. Retrieved from http://id.nii.ac.jp/1001/00014478/
[43] 久野障(1973).『日本文法研究』東京:大修館書店
[44] Kuroda, S.-Y. (1965). Generative grammatical studies in the Japanese language, MIT. Retrieved from Retrieved from http://dspace.mit.edu/handle/1721.1/13006
[45] Laffont, J.-J. (1989). Economie de l'incertain et de l'information. New York, NY:MIT Press.(佐藤公敏訳『不確実性と情報の経済学』東京:東洋経済新報社, 1992)
[46] Lewis, D. (1976). Counterfactuals. Oxford, UK: Basil Blackwell.
[47] Lloyd, J. (1987). Foundations of logic programing. New York, NY: Springer-Verlag. (佐藤雅彦・森下真一訳『論理プログラミングの基礎』東京:産業図書, 1987).
[48] Marschak, J., & Miyasawa, K. (1968). Economic comparability of information systems. *International Economic Review*, 9, 137-174. doi: 10.2307/2525472
[49] Maskin, E. (1999). Nash equilibrium and welfare optimality. *The Review of Economic Studies*, 66, 23-38. doi:10.1111/1467-937X.00076
[50] McCarthy, J. (1980). Circumscription: A form of non-monotonic reasoning, *Artificial Intelligence*, 13, 27-39. doi:10.1016/0004-3702(80)90011-9
[51] 三上章(1963).『日本語の論理:ハとガ』くろしお出版.
[52] Minsky, M. (1975) A framework for representing knowledge. In P. Winston ed., Psychology of Computer Vision. New York, NY: McGraw-Hill, pp. 211-277.(知識を表現するための枠組. 白井良明・杉原厚吉訳『コンピュータビジョンの心理』東京:産業図書, 1979, pp.237-335)
[53] 宮沢光一(1971).『情報・決定論序説』東京:岩波書店.
[54] 持橋大地・大羽成征(2019).『ガウス過程と機械学習』東京:講談社.
[55] Montague, R. (1974). Formal philosophy: selected papers of Richard Montague. Edited and with an introduction by Richmond H. Thomason. New Haven, CT: Yale University Press.
[56] 中島秀之(1984). 知識表現用言語としてのProlog/KR. 情報処理学会論文誌, 25(2), 180-186. Retrieved from http://id.nii.ac.jp/1001/00015957/
[57] 中島秀之・浅田稔・松原仁他編(2019).『AI事典第3版』東京:近代科学社.
[58] 岡田光弘(2000). 法的推論の論理による表現. 吉野(2000, pp.169-181))所収.
[59] Ostrovsky, M. (2008). Stability in supply chain networks. *American Economic Review*, 98, 897-923. doi: 10.1257/aer.98.3.897
[60] Pearl, J. (1997). On the identification of nonparametric structural models. UCLA Cognitive Systems Laboratory, Technical Report R-207. Retrieved from https://escholarship.org/uc/item/4bq5r0c4
[61] Perry, J, & Barwise, J. (1983). Situations and attitudes. Cambridge: MA: MIT Press. (土屋俊・鈴木浩之・白井英俊・片桐弘訳『状況と態度』産業図書, 1992)
[62] Pettit, P., & Sugden, J. (1989). The backward induction paradox. *The Journal of Philosophy*, 86, 169-182. doi: 10.2307/2026960
[63] Robinson, J. A. (1965). A machine-oriented logic based on the resolution principle. *Journal of the*

ACM, 12, 23-41. doi:10.1145/321250.321253
[64] 坂井秀寿(1979).『日本語の文法と論理』東京:勁草書房.
[65] 坂原茂(1985/2007).『日常言語の推論』東京:東京大学出版会.
[66] Sen, A. (1982/1997). Choice, welfare and measurement. Oxford, UK: Basil Blackwell / Cambridge, MA: Harvard University Press.
[67] 柴谷方良・影山太郎・田守育啓(1982).『言語の構造―理論と分析―:意味・頭語篇』東京:くろしお出版.
[68] 白井賢一郎(1985).『形式意味論入門:言語・論理・認知の世界』東京:産業図書.
[69] 白井賢一郎(1991).『自然言語の意味論:モンタギューから「状況」への展開』東京:産業図書.
[70] Starling, L., & Shapiro, E. (1994). The art of prolog: advanced programing (2nd ed.). Cambridge, MA: MIT Press. (松田利夫訳『Prologの技芸』東京:構造計画研究所, 1988)
[71] Swift, T., & Warren, D. S. (2012). XSB: Extending Prolog with tabled logic programing. *Theory and Practice of Logic Programing*, 12, 157-187. doi:10.1017/S1471068411000500
[72] 高野真(2014). Prologで学ぶAI手法:推論システムと自然言語処理. Retrieved from https://www.az-Prolog.com/books/Prolog
[73] 田村明久(2013). 安定結婚からサプライチェーンネットワークの安定性へ. オペレーションズリサーチ, 58, 325-331. Retrieved from http://www.orsj.or.jp/archive2/or58-06/or58_6_325.pdf
[74] 東京大学教養学部統計学教室(1991).『統計学入門』東京:東京大学出版会.
[75] Tversky, A. (1994). Contingent preferences: Loss aversion and tradeoff contrast in decision making. *Japanese Psychological Research*, 36, 3-9. doi:10.4992/psycholres1954.36.3
[76] Wason, P. (1968). Reasoning about a rule. *Quarterly Journal of Experimental Psychology*, 20, 273-281. doi:10.1080/14640746808400161
[77] Winograd,T. (1972). Procedures as a representation for data in a computer program for understanding natural language. *Cognitive Psychology*, 3, 1-191. doi:10.1016/0010-0285(72)90002-3
[78] Winograd,T. (2008). SHRDLU. Retrieved from http://hci.stanford.edu/~winograd/shrdlu/
[79] 吉田民人(1967). 情報科学の構想: エヴォルーショニストのウィーナー的自然観. 加藤秀俊・ 竹内郁郎・ 吉田民人『社会的コミュニケーション』東京:培風館, pp.1-287.

---

[1] AI の基本的手法としては, 論理学に基づく方法の他, ルールベース, 事例ベース, 常識推論, 進化アルゴリズム, ファジイ推論, 統計学習, 深層学習, エージェントシミュレーションといった基礎技術, およびそれらを応用した擬人化エージェント(チャットボット, ヒト型ロボット, ネットゲームのキャラクターなど)などがある. AI 各分野の詳細については, 中島・浅田・松原他(2019)を参照されたい.

[2] 日本産業規格 情報処理用語 基本用語 JIS X 0001 01.05.10/同プログラミング JIS X 0007:2001 07.02.01

[3] 日本産業規格 情報処理用語 プログラミング JIS X 0007:2001 01.05.01

[4] 日本産業規格 情報処理用語 プログラミング JIS X 0007:2001 07.01.12

[5] ISO 10646/Unicode が 1991-1993 に策定されて以降, マルチバイトコードへの対応が進み, 例えば VBA も日本語の文字列をコメント部分以外に, 変数名や関数名として使えるように改良された. それ以前のプログラミング言語では日本語文字種を使うため特別の工夫が必要であった(中川 et al., 1994).

[6] Prolog プログラムは, フレームを潜在的に表現する. フレームは Minsky(1975)によって提唱された. AI の文献では, フレームを論理や意味ネットワークと異なり, より構造化された典型的な知識の表現であると説明され, 後のオブジェクト指向に近いものと理解されてきた. しかし Minsky によるオリジナルの記述では, 適用状況に応じて迅速に相互変換され, 再結合可能な質問の集まり, すなわちメタレベルで統制された節プログラムとみなすのがより正確であろう.

[7] 日本産業規格 情報処理用語 プログラミング JIS X 0007:2001 07.02.09

[8] Wikipedia(英語版)"Prolog"によるとPrologの計算機構はチューリング完全であり，またクイズ番組で人間に勝ったIBMワトソンや企業のウェブアプリケーション，スマートデバイス，マイクロソフトWindowsプラットフォームなど多岐に渡って適用されている知識表現ベース開発ツールに応用されるなど，現在まで実用に耐えている．https://en.m.wikipedia.org/wiki/Prolog (2020年3月1日閲覧)

[9] ISO標準Prologにかんしては，Deransart, Ed-Dbali & Cervoni(1996)に詳述されている．

[10] 日本産業規格．情報処理用語―人工知能―基本概念及びエキスパートシステム JIS X 0028:1999 28.04.14.

[11] ウェブサイト上のリソースとしては，神戸大学田村研究室(http://bach.istc.kobe-u.ac.jp/prolog/)が他の文献や関連サイトへのリンクを含めて簡潔に示されている．また同研究室ではJava言語への翻訳を実装したProlog Cafeが開発・提供されている．ウェブサイト「Haskellが数学だとすれば、Prologは国語だ」(https://nojiriko.asia/prolog/)には用例集として日本語化されたPrologプログラムが多数掲載されている．また気軽に読めるウェブ教材(英語)として，例えば，"Learn Prolog now!"(Blackburn, Bos, and Striegnitz, 2003)がある．

[12] 犬童(2018, January 22)のスライド中の事例を参照．

[13] 本論文中の5章から9章まで，および付録Cのソースコードを，フェアユースの範囲で引用される場合は，本論文を出典として挙げられよ．犬童健良(2020)．日本語による論理プログラミングの試み．関東学園大学経済学紀要第46集，pp.31-121．なお本論文の著作権は関東学園大学に譲渡されている．このことは本論文で示されたアイディアやプログラムを研究教育や実務一般に実践活用される自由を妨げるものではないが，現実適用の際には著作権にご留意の上，筆者およびその所属先は，本論文に掲載されたプログラムの正確性や実用性を保証せず，これらの利用によって生じた，あるいは生じうる損害の一切の責任を負わないという条件に合意の上，読者自身の責任で行って頂けると幸いである．

[14] swi-prolog.orgから2条項BSDライセンスで配布されている．Windows版SWI-Prologはマルチバイトコードに対応している．それゆえ基本的に日本語でプログラムを書くことができる．しかしWindowsではshift_jis(cp932)における非対応文字種(括弧など)があるので，それらの文字を含む日本語文字列の述語名は引用符で囲み，変数名の場合は日本語名の先頭に半角下線を付加し，非対応文字の使用を避ける必要がある．ファイルからの読込みにcsv_read/3，ファイル書き出しにtell/1, told/0が便利だが， utf-8で読み込み，shift_jis(cp932)で出力される(オプションはあるが効かないようである)．そこで日本語文字を含むファイルはテキストエディターやnkfコマンドで文字コード変換する必要がある．マルチバイトコード未対応の処理系(例えばGNU-Prolog)では，文字コードで処理して外部ファイルへ出力，あるいはブラウザで表示するといった代替的方法が考えられる．

[15] ルールの頭部は一つの述語であり，本体は複数の述語をカンマでつなぐことができる．各述語はカッコ内に引数を持つことができ，述語名と引数の数(アリティ)によって区別される．本体の述語は否定オペレータ(¥+)を前置できる．ルールの論理的意味は，本体の述語の連言(AND結合)が，頭部の述語の十分条件となるということである．

[16] 導出は背理法の証明手続きを機械化したものであり，未代入の変数に適切な代入を見つけ出すこと，すなわち単一化アルゴリズムが内蔵される．実行の制御は，節の頭部から本体の条件に順にツリー状に辿る方式(SLD導出)を採用し，途中で代入に失敗すると自動的に後戻りして別の代入する値を探す仕組み(バックトラック)を備える．これによって適切に問題をプログラミングすると，Prologはその問題の論理的に可能な解を系統的に調べつくす．

[17] 単一化(ユニフィケーション;unification)は論理推論を自動化するresolution(導出)に伴い変数に代入する項を決定するアルゴリズムである．項のペア u と t に対し， $u\sigma = t\sigma$ となる項の代入 σ を単一化子(unifier)と言う．たとえば文字aをXに，数字3をYに，f(Y, X)をZにそれぞれ代入することを， σ ={a / X, 3 / Y, f(Y, X) / Z}などと書く．u = f(X, Y)， t = Z とすると，$u\sigma = t\sigma$ = f(a, 0)と単一化できる．代入は一般に複数あるので，Prologの単一化はそれらのうち最も簡潔なものを選ぶ．ちなみに，類推などの帰納推論に応用されるのは，逆単一化の操作である．

[18] Prologでは一階述語論理の記述力を拡張して，差分リスト，末尾再帰，カットとfailによるバックトラック打ち切り，データベース述語における存在限定子，DCGパーサなど，高階論理ないし非論理的な述語がよく利用される．またプログラマーは手続型プログラミングと同様，必要に応じてtrace/0やspy/1などのデバッグ用のツールを使いこなして，導出に沿ったルールの実行順序と変数代入の経緯を把握する能力を要求される．

[19] SWI-Prologの組込述語のヘルプファイルは，プロンプト?- に"help."と改行(ENTER)キーを押下げ，下方のテキストボックスに述語名を入力してsearchボタンを押すと読むことができる(ただし英文)．配布元

のウェブサイト https://www.swi-prolog.org/pldoc/のリファレンスマニュアルなども参照できる.
[20] またこれらを組み合わせると, Web-API からのデータ取得やWebスクレイピングが可能できる. より具体的に言うと, 筆者が 2019 年に試みた例は, EDINET タクソノミー分析, CiNii Books API のオープンデータ取得, Google Books 店頭販売価格のスクレイピングなどである.
[21] メモ帳(notepad.exe)では UTF-8(BOM 付き)になる.
[22] 前出のウェブサイト nojiriko.asia/prolog/の手法は, 本文中の4節と付録 B で示した方法, 6 節で述べた手続き SS に近いものと考えられる. また助詞をファンクターとする方法も取り入れられている.
[23] op/3 についての SWI-Prolog のヘルプ(?- help(op).)による.
[24] SWI-Prolog では, (X, Y) はユニブによって [',', X, Y] と単一化される. つまり, (X, Y) = ',' (X, Y), (X, Y, Z) = (X, (Y, Z)) = ',' (X, ',' (Y, Z)) などとなる.
[25] 主題化後のタプルに元のタプル全体を残すアイディアは, ゲーム理論的メカニズムデザインにヒントを得ている. タプル(選好プロフィール)を情報空間に吐かせることによって, Hurwictz(1972)は分権的資源配分メカニズムに対して privacy preserving の条件を課し, またナッシュ遂行理論の基本定理(Maskin, 1999)ではそれによって古典的な不可能性定理(Gibbard-Satterthwaite 定理)を免れている.
[26] ちなみに, 関係 (X, Y) を述語 X(Y) のプログラム上の表現とする(非エジンバラ記法の)Prolog 処理系として, Prolog-KR(中島, 1984)や micro-Prolog(Clark &McCabe, 1984)が知られる.
[27] 犬童・松田・太田(1989)はフレーム(Minsky, 1975)による説明を試みている.
[28] 文の意味の安定性とサプライチェーンネットワークの安定性との関連性は, 犬童(2018)によって最初に論じられた(予稿論文非公開). またこれは犬童(2016)で提案された「心の中のゲーム」アプローチの一つの展開と考えることができる.
[29] 井上(1989, IV 2)
[30] ベイズの定理は, 関心の対象となる事象 Y の確率が, シグナル X を受信前と受信後でどのように変化するのかを計算する. すなわち, 事後確率 P(Y|X), 事前確率 P(Y)とすると, P(Y|X)= P(XY) /P(Y)である. 情報理論では Shannon のビット情報量および統計熱力学のエントロピー概念を基礎として確率に基づく情報概念を用いて, 通信経路へのノイズ混入の影響などを評価し, 確率分布自体の変化をダイバージェンス(カルバック=ライブラー情報量)として把握する(有本, 1980). 不確実性下の意思決定者の行動を論じる諸分野, 統計的意思決定論やゲーム理論(Marschak & Miyasawa, 1968; 宮沢, 1971)では期待利得(期待効用)を鎖大化する意思決定者が, メッセージ受信前と受信後で行動を変化させる場合に, 情報の価値があったとみなされる. とくに Blackwell の定理(Laffont, 1985; Blackwell, 1953)は, 情報システム(情報構造)をn個のシグナルとm個の代替案に一般化し, マルコフ行列を用いて表現することにより, 情報システムのペア間での「より情報が多い」という関係を特徴付けする(証明は, 例えば). Blackwell の定理は情報経済学や経営情報学などに幅広く応用されている.
[31] 情報の旧/新の概念に見られる異論理性を, 総記や対比は自然言語理解で生じる信念改訂と解釈することも可能と思われる. Ramesy test は科学哲学において, 自明性定理(仮説の正しさをデータ観察後の事後確率とすると事前確率と一致し無情報となる)を軸に長く議論されている. David Lewis の自明性定理(Lewis, 1976)は, 自然言語の反事実条件文(counterfactuals)ないし仮想法(subjunctives)の意味論における確率論と論理学の融合の困難を示している. Gärdenfors(1986)は自明性定理の様相体系(認識論理)バージョンを示した. また条件文理解と関連するバックワードインダクションパラドックスは, 人々の行動や意思決定を論じる統計数理, 社会科学分野, およびAIに広く共通する基礎的問題である. 確率論や論理学の素朴な適用は意外性(surprise)をうまく扱えないという問題は, 意思決定論におけるニューカム問題(Horgan, 1981), ゲーム理論における均衡洗練(Bicchierri, 1986), あるいは共通知識問題(Aumann, 1995)などにおいて生じる. また因果推論と密接に関連する(Halpern and Peral, 2005; Jacobs, 2019). Lewis の提案した反事実条件文の最近傍不可能性世界としての意味論モデルは, サーカムスクリプション(McArthy, 1980)の言語理解における一つの事例と考えられる. ちなみに, 犬童(2009)は Dodgson 投票ルールを Condorcet 投票ルールに対する最近傍不可能性世界の社会選択版と解釈し, その Prolog プログラムを示している.
[32] 情報概念を, 認知・評価・指令の3局面から区別するものとして吉田(1998)がある. 吉田は社会学の立場から, サイバネティクスを参考にした情報概念を論じた.
[33] 虚偽情報や曖昧情報を含む社会的機能の例として, 例えば, インターネットの SNS サービスにおけるフェイクニュースやユーザ間でやりとりされる煽りがある. それらは情報の受信者に信念や感情・気分の変化(不安, 希望, 憤りなど)を与え, 特定の行動, 同一情報を発信すること(情報の拡散), 炎上と呼ばれる集合的情報発信, あるいは選挙における特定候補者への投票などへ誘導することを狙いとして大量に流布される. まずユーザ自身の自己意識において, 特定の感情や気分が自覚される. またそれがネットワーク

効果によって増幅される，すなわち情報発信者に対してフィードバックされることで新たにコピーされた情報の発信者を生む結果，集合的な行動として観察される．

[34] 誤伝のない情報は，宮沢の言うⅠ型情報（あるいは実験）に相当する（宮沢，1971，p.84）．Ⅰ型情報は実験で観察されたデータを，ある特定の確率分布から抽出された標本として知ることができる．そうでないのが誤伝のある情報，Ⅱ型の情報である．Ⅱ型の情報はn個の状態とm個のシグナルに一般化し，マルコフ行列を用いて表現される（p.85）．また表1の4区分は，統計学的検定（林，1973；東京大学教養学部統計学教室，1991）におけるエラーの種類の区分を模している．αは第1種の誤り（正しい帰無仮説が棄却されて有意性ありと判定されるケース）の確率，βは第2種の誤り（対立仮説が正しいのに帰無仮説が棄却されず有意性なしと判定されるケース）に相当する．αは危険率（有意水準），1－βは検定力とも呼ばれる．なお，臨床検査分野(Cosby，2017)では，非感染者を陽性と誤判定するのが偽陽性（false positive），感染者を陰性と誤判定するのが偽陰性(false negative)であり，それぞれ第 1 種と第2種の誤りに相当する．また特異度，感度が1－α，1－βに相当する．

[35] 以下にそのプログラム断片を示す．

```
現在Bに割当てられているAの代わりにCを割当てる案( _A, _B, _C ):-
        左側リストの要素( _C ),
        割当案( _A, _B ),
        _C ¥= _A,
        ¥+ 割当案( _C, _B ).
```

[36] 領域条件は，常識推論の研究におけるサーカムスクリプション(McCarthy，1980)と関連する．

[37] 脚注 28 参照．

[38] Ostorovsky(2008)，田村(2013)参照．これらの代替性条件は，契約のゲーム理論モデルの文脈において一般化され，プレイヤーの間接効用関数の劣モジュラー性と関連付けられる(Hatfield et al.，2018)．

[39] Ostorovsky(2008)．

[40] 登録された各述語（の真の事例）を列挙すると次のようである．

```
?- 述語名( X ), current_predicate( X, P ), P, writeln( P ), fail.
クジラ(ホニュウ動物)
金魚(魚)
カエル(両生類)
false.
```

関係としては，以下のように登録される．

```
?- 関係( _述語名, _引数 ), nl.
_述語名 = クジラ,
_引数 = ホニュウ動物 ;
_述語名 = 金魚,
_引数 = 魚 ;
_述語名 = カエル,
_引数 = 両生類.
```

[41] 阻止条件は条件文理解を説明に際して鍵となる．条件文理解については，Frank Ramsey の先駆的業績（いわゆる Ramsey test）以降，論理学，科学哲学，言語学，心理学に分散して長らく研究されている（脚注 31 も参照）．坂原(1985/2007)は言語学寄りからの日本語文献である．条件文研究を哲学と心理学を中心にレビューした文献として，Evans & Over (2004).をあげておく．また総記や対比（黒田，1963；久野，1973）は，代替的関係やその不在を（中立叙述や主題の）阻止条件として成立するが，とくに対比は談話文脈における既定値（default）に対して感応的である（久野，1973，p.31）．これは Wason が条件文推論課題についての認知心理学実験において見出した否定パラダイムと同様，行動経済学における参照点依存やフレーミング効果に類似することが興味深い．ちなみにプロスペクト理論における損失回避を応用して，ブランド選択における文脈効果を説明するための「トレードオフ対比」が提案されている(Tversky，1994)．

[42] 総記では，文の表す基本的な関係（X，Y）に対し，その代替的関係（Z，Y）をなす別の語 Z≠X によって，中立叙述の解釈が阻止される．一方，対比では，同じく基本的な関係（X，Y）に対し，その代替的関係（Z，W）が存在しないとき，それを言語理解者自身が考え出さない限り，文の意味が理解しにくいと感じられる．こうした一方の阻止条件が他方の阻止条件の否定という関係性は，総記と中立叙述，対比と主題，そして総記と対比のそれぞれの間に見られる．これは数学的に言うとブロッキングリレーション，脳神経科

学用語での側抑制にあたる.
[43] 社会選択理論における領域条件は，AI 研究者が気づいていないサーカムスクリプションの例である. 10.3 節で言及した一般不可能性定理(Arrow,1951/1963)は，5 条件のいずれかを緩和することで可能性の結果が得られる. このうち全域性（定義域の無制限性）の条件を緩和する可能性の結果として，Duncan Black による単峰的領域を嚆矢として，その後さまざまな領域制限(domain restrictions)ないし領域条件(domain conditions)が研究された(Sen, 1982; Gaertner, 2001). ちなみに，犬童(2014)では，Arrow の定理およびそのゲーム理論バージョンである Gibbard-Satterthaite 定理の Prolog プログラム(犬童，2007，2009)を応用して，両定理に対するより精密な領域条件を見出している.
[44] SWI-Prolog では未登録述語はエラーになるので，clause/2 を用いるか，あるいは dynamic 宣言をしてお必要があるく.
```
?- c.
ERROR: Undefined procedure: c/0 (DWIM could not correct goal)
?- clause( c, _ ).
false.
?- dynamic(c/0).
true.
?- c.
false.
```
[45] Pettit & Sugden(1989), Bicchieri(1988), Halpern & Moses (1986), Basu(1994).
[46] Colmerauer & Roussel (1996). 付録 A も参照.
[47] Montague (1974)の後継として最もよく研究されるのが，自然言語断片に限量化を適用して可能世界意味論（内包論理）に翻訳するアプローチ（PTQ）であり，述語論理とλ計算が駆使される. すでに 1970 年代末に日本語に適用されている（坂井，1979）. また Hinttika や Lorenzen の研究に刺激されてゲーム理論が応用された. 一般記号論としての UG を提唱するアプローチでは，述語論理や数学（代数システムの準同型写像）の知識を要請する（白井，1985）. また，状況意味論，談話表示意味論などの代替的理論が，モンタギュー文法の問題点を克服するため研究された（Perry & Barwise, 1992; 白井，1991）.
[48] Simon のアイディアをやや強引に要約してみると，次のようになるだろう. 限定合理性によって最適化ないし最大化の試みを挫かれた人々は，満足化原理(satisficing principle)にしたがって行動を選択する. その結果としての集団的相互作用は，準分解可能性(quasi-decomposability)という性質を帯びた階層システムとして組織や社会秩序といった人工物を形成する.
[49] 当初は P のカードを選ぶパタンが多くみられるため，確証バイアスの例と考えられたが，否定パラダイムの発見によって，マッチングバイアスであると考えられるようになった. 後の認知心理学者は実践的な文脈（許可スキーマ），あるいは環境に埋め込まれた知能（進化的プログラム）といった代替理論が提案された. また Wason に影響された条件文の心理学研究(Evans, 2004)から二重過程理論が提唱され，現代の行動経済学との接点にもなっている.
[50] Prolog プログラムは，フレームを潜在的に表現する. フレームは Minsky(1975)によって提唱された. AI の文献では，フレームを論理や意味ネットワークと異なり，より構造化された典型的な知識の表現であると説明され，後のオブジェクト指向に近いものと理解されてきた. しかし Minsky によるオリジナルの記述では，適用状況に応じて迅速に相互変換され，再結合可能な質問の集まり，すなわちメタレベルで統制された節プログラムとみなすのがより正確であろう.
[51] ただし，ここで言う最適化技術は，OR 手法のうち数理計画法と呼ばれるものを指す. 論理プログラミングにおける導出原理は，数理計画法において0－1整数計画法と呼ばれる特殊な組合せ最適化問題と密接な関係にある（Hooker, 2000）.
[52] 両手法は独立した研究領域だが，意思決定支援システムの構築やデータ分析の手法としては相互に乗り入れがある. ニューラルネットの重み学習アルゴリズムには最適化が含まれており，組合せ最適化のアルゴリズムにはメタヒューリスティックス（ルールベースシステムの一種）やニューラルネット学習も活用されている.
[53] Frey & Osborne(2013)の付録の表を参照. 職種ごとの仕事と作業者の特性や要求されるスキルのデータベース（O*NET 変数）を用いて自動化される確率を推定している.
[54] 70 職種を教師ラベルとしたガウス過程識別潜在変数を作り，指数的二次共分散行列のガウス過程分類器を適用している. ガウス過程およびその識別関数（判別関数）については，持橋・大羽(2019, p.97)を参照.
[55] 確定節に対する選択的線形導出（Selective Linear resolution for Definite clause）. 事実節を終端とし

て，ゴールに単一化された頭部をもつルールの本体の条件を順に下位ゴールとして質問していくため，導出木が一方方向に枝分かれしていく形になる．